SE Experiment-8

(Batch-A/A3)

Team Details

Dev Gaglani-2024300057
Stavan Desai-2024300042
T.E. Computer Engineering – A

Aim:

To implement and test at least two core modules of the chosen case study, the **Inventory and Stock Management System (ISMS)**. The two core modules are **Module 1: Authentication & Role-Based Access Control** and **Module 2: Voice-Enabled Inventory & Omnichannel Synchronization**. Both are implemented exactly as specified in the SRS, use case, class, sequence, activity and data-flow diagrams prepared in Experiments 1–7, and are verified with documented test cases and automated tests. The other five SRS feature modules (Supplier Management, Purchase Orders, Sales, Forecasting, Notifications) and the platform module were also built, so that the two core modules run inside a complete system.

Theory:

The **implementation phase** of the software development life cycle converts the design (use-case, class, sequence and activity diagrams and DFDs) into working source code. A large system is implemented **module by module**: each module covers a related set of functional requirements, has a clear interface (here, REST API endpoints and WebSocket channels), and can be built and tested independently before being integrated with the rest of the system.

Software testing checks that the implementation behaves as the requirements say. The levels of testing used in this experiment are:

- **Unit testing:** individual classes and methods (e.g. `VoiceInterpreter.interpret()`, BCrypt hashing) are tested in isolation.
- **Integration testing:** several components are tested together through the real REST API and a real PostgreSQL database (controller → service → entity → database), using JUnit 5, Spring Boot Test and MockMvc.
- **System testing:** complete end-to-end scenarios are executed in the browser exactly as an administrator, warehouse staff member or sales person would, and the results are captured as screenshots.

Test cases are designed using black-box techniques: **equivalence partitioning** (valid / invalid credentials, valid / invalid voice command), **boundary value analysis** (the 4th versus the 5th failed login, voice confidence just below and at the 0.75 threshold, stock of 1, 0 and −1 units), and **negative testing** (reused password-reset token, forged token, wrong role, duplicate SKU). **White-box** checks cover each branch of the activity diagram and sequence diagrams, for example both outcomes of the decisions “Credentials valid?”, “Account locked?”, “Use Voice Command?”, “Confidence ≥ threshold?” and the `alt` branch of SD-1 (“quantity < reorder point” versus “no alert required”).

Each test case records the **input**, the **expected result** derived from the SRS, the **actual result** observed, and a **pass / fail status**. The automated tests can be re-run after every change with one command (`./mvnw test`), which provides **regression testing**.

Problem Description

The Inventory and Stock Management System must track, control and optimise stock across multiple warehouses and sales channels. Only authenticated users may use it, and every action is restricted by role: Administrators manage users and system settings, Managers oversee suppliers, purchase orders and forecasts, Warehouse Staff keep stock accurate (also hands-free by voice) and confirm goods receipts, and Sales Personnel record physical sales and returns while marketplace orders arrive by webhook.

The SRS (`moodle/moodle.md`, Document A) groups the functional requirements into seven feature areas. For this experiment the two features on which all the others depend were selected:

1. **Authentication & RBAC:** SRS 4.1 `Auth.*`, SE-1 to SE-5, BR-1, BR-5; use cases "Manage Users and Roles" and "View System Reports & Audit Logs"; classes `User` and `Administrator`; DFD process 1.0 (and 2.0 for user management), data store D1.
2. **Voice-Enabled Inventory & Omnichannel Synchronization:** SRS 4.2 `Inventory.*`, `Voice.Command.*`, `Sync.Channel.*`, BR-3, PE-1, PE-2; use case "Manage Inventory (Stock)" with its «include» operations and the «extend» "Voice Command"; classes `Product`, `StockRecord` and `WarehouseStaff`; sequence diagram SD-1; DFD process 4.0 (Level 2 steps 4.1–4.5), data stores D3 and D4.

Both modules had to be implemented and tested, including their failure paths: a wrong password, the fifth failed login, a forged or reused reset token, a user opening a page outside their role, a duplicate SKU, deleting a product that still has stock, a negative quantity, two simultaneous sales of the last unit and a voice command below the confidence threshold.

Implementation:

ISMS was implemented as a full-stack web application in which every screen and API call is traceable to the SE documents from the earlier experiments. The finished system lives in the GitHub repository `2000stavan/lala`, and its history is rebuilt module by module in a new repository (one branch and one commit per module, see section 3 and `github.md`).

1. Technology Stack

Layer	Technology	Purpose in ISMS
Frontend	React 18 + Vite, TypeScript, Tailwind CSS, Recharts	Role-based web client with light and black dark themes; one page per module
Backend	Java 21, Spring Boot 3.3 (Web, Security, Data JPA, Validation, WebSocket, Mail, AOP)	REST API; one service class per class-diagram class
Domain layer	Java service + JPA entity classes	Implements the class diagram: `User`, `Product`, `StockRecord`, `Supplier`, `PurchaseOrder`, `SalesOrder`, `Forecast`, `Notification` …
Database	PostgreSQL 16, Flyway migrations V1–V6	Data stores D1–D8 of the DFD; CHECK constraints enforce BR-3 in the database
Security	JWT (30-minute sliding), BCrypt, RBAC with `@PreAuthorize`, HMAC-SHA256	Login, lockout after 5 failures, role checks on every endpoint, signed marketplace webhooks
Real time	Spring WebSocket (`/ws/stock`, `/ws/alerts`)	Live stock updates and alerts within the 5-second budget (PE-2)
Reporting	OpenPDF, in-process Holt-Winters forecasting	PDF invoices; demand forecast without an external ML service
Testing	JUnit 5, Spring Boot Test, MockMvc, Vitest, `smoke_test.sh`	70 backend tests, 1 frontend test and 53 live API checks

2. System Architecture

The system follows a layered (3-tier) architecture:

- **Presentation tier:** the React client. Each role sees only its own menu and a route guard blocks pages outside the role. The sidebar, theme switch, onboarding walkthrough and help button are shared by every page.
- **Application tier:** Spring Boot controllers → service classes → JPA entities. The service classes carry the names of the class-diagram operations (`approvePo`, `deductStock`, `captureVoiceCommand` …) and an AOP aspect (`CallTraceAspect`) records the order of the calls, so the sequence-diagram tests can compare the running code with SD-1, SD-2 and SD-3 message by message.
- **Data tier:** a PostgreSQL database created only by Flyway migrations. Tables map to the DFD data stores, and the schema is checked against `schema.md` by an automated test.

Project structure (3-tier: React client, Spring Boot services, PostgreSQL)

```
se-proj/
|-- backend/           Spring Boot 3.3 (Java 21)
|   |-- src/main/java/com/isms/
|   |   |-- api/       14 REST controllers
|   |   |-- domain/    13 services (one per class-diagram class + voice interpreter)
|   |   |-- model/     16 JPA entities (+ enums)
|   |   |-- repository/ 12 Spring Data repositories
|   |   |-- security/  JWT filter, RBAC config, lockout cache
|   |   |-- websocket/  /ws/stock, /ws/alerts handlers
|   |   `-- audit, error, trace, adapter, config
|   |-- src/main/resources/db/migration/   6 Flyway migrations (V1..V6)
|   `-- src/test/java/com/isms/            18 test classes
|-- frontend/          React 18 + Vite + TypeScript + Tailwind
|   `-- src/pages/     13 page components, light/dark theme
|-- docs/              traceability matrix, class-diagram mapping, SRS/DFD/UML sources
|-- scripts/           smoke_test.sh, load_test.sh, simulate_marketplace.sh, module_split.py
`-- docker-compose.yml  PostgreSQL 16, Mailpit, backend, frontend
```

Fig. 1: Project structure: the three tiers and the number of files in each package

psql isms - tables created by the Flyway migrations (data stores D1-D8 of the DFD)

```
isms=# \dt-style listing
table           | columns
------------------------+---------
 flyway_schema_history  |      10
 forecast_accuracy      |       8
 forecasts              |       5
 notification_preferences |      6
 notifications          |       8
 po_line_items          |       5
 products               |       6
 purchase_orders        |       7
 sales_order_line_items |       6
 sales_orders           |       6
 stock_records          |       5
 suppliers              |       7
 users                  |       6
(13 rows)
```

Fig. 2: Database tables created by the Flyway migrations (data stores D1–D8 plus the line-item and support tables)

psql isms - Flyway migration history

```
isms=# SELECT version, description, success FROM flyway_schema_history;
version |       description        | success
---------+--------------------------+---------
 1       | init                     | t
 2       | seed reference           | t
 3       | notification read status | t
 4       | fix sales line item pk   | t
 5       | notification preferences | t
 6       | forecast accuracy        | t
(6 rows)
```

Fig. 3: Flyway migration history: V1 schema, V2 reference users, V3–V6 notification status, sales line-item key, preferences and forecast accuracy

psql isms - database-level guards on stock_records (BR-3)

```
isms=# constraints of stock_records
constraint              |                                definition
------------------------------------+-------------------------------------------------------------------------
 stock_records_check                | CHECK ((quantity_reserved <= quantity_on_hand))
 stock_records_pkey                 | PRIMARY KEY (sku_id, location_id)
 stock_records_quantity_on_hand_check | CHECK ((quantity_on_hand >= 0))
 stock_records_quantity_reserved_check | CHECK ((quantity_reserved >= 0))
 stock_records_sku_id_fkey          | FOREIGN KEY (sku_id) REFERENCES products(sku_id) ON DELETE RESTRICT
(5 rows)
```

Fig. 4: Database-level guards on stock_records: quantity_on_hand >= 0 enforces business rule BR-3 even if the application were bypassed

3. Module Division and Teamwork

The seven modules are the seven feature areas of the SRS (section 4.1 to 4.7); Module 8 contains everything else. The work was **divided in alternating order**: Dev Gaglani implemented and pushed the odd-numbered modules (1, 3, 5, 7) and Stavan Desai the even-numbered ones (2, 4, 6, 8). Each module is one branch (`module-<n>-<name>`) with one commit, reviewed by the other team member through a pull request before it is merged, and every stage compiles on its own (verified for all eight stages).

#	Module (SRS feature)	What it covers	Done by	Branch
1	Auth & RBAC (4.1)	Secure login and role-specific access control for admins, managers, warehouse staff and sales personnel; also carries the project foundation (schema, entities, app shell)	**Dev**	module-1-auth-rbac
2	Voice-Enabled Inventory & Omnichannel Sync (4.2)	Full CRUD on products and stock through the standard UI or hands-free voice commands, real-time sync to all channels	**Stavan**	module-2-voice-inventory-sync
3	Supplier Management (4.3)	Vendor profiles, contact details, performance rating and procurement history	**Dev**	module-3-suppliers
4	Purchase Order Management (4.4)	Creates, approves and tracks inbound POs; stock rises automatically on receipt	**Stavan**	module-4-purchase-orders
5	Sales Tracking & Order Management (4.5)	Physical and digital sales, marketplace webhooks, invoicing, returns, automatic stock deduction	**Dev**	module-5-sales
6	Predictive Analytics & AI Forecasting (4.6)	Revenue and stock dashboards; demand forecast, seasonality and reorder points	**Stavan**	module-6-forecasting
7	Multimodal Notification System (4.7)	Real-time low-stock and shipping-delay alerts by e-mail, SMS and audio; inbox and preferences	**Dev**	module-7-notifications
8	Platform hardening (the rest)	Demo data, non-functional tests, class-diagram tests, load/smoke scripts, traceability and documentation	**Stavan**	module-8-platform-hardening

A few files travel with an earlier module because of real code dependencies (for example the notification engine ships with Module 2, because every stock change can raise a low-stock alert). These are listed in `github.md`.

4. How the System Works, Step by Step

The numbered flow below follows one typical working day and names the module and DFD process that handles each step. It is the order in which the modules are explained in the rest of this report.

1. **Sign in (Module 1, DFD 1.0).** Every user signs in with a user ID or e-mail and a password. The server checks the BCrypt hash, issues a 30-minute sliding token containing the role and shows only the menu items allowed for that role. Five wrong passwords lock the account.
2. **Administer users (Module 1, DFD 2.0, D1).** The Administrator creates users, assigns roles, unlocks accounts and reads the audit log; every action is logged.
3. **Maintain stock (Module 2, DFD 4.0, D3/D4).** Warehouse staff create and update products and change quantities through the form or by voice ("add 20 units of SKU-002 to location WH-01" → read-back → confirm). Every change is made under a database row lock and pushed live to every open screen.
4. **Get alerted (Module 7, DFD 7.0, D8).** When stock falls below the reorder point the system raises a low-stock alert by e-mail, SMS, audio and in the inbox.
5. **Buy goods (Modules 3 and 4, DFD 2.0/6.0, D2/D7).** Managers keep supplier records and create purchase orders; larger orders need an Administrator; Warehouse Staff confirm receipt, which raises stock (back to step 3).
6. **Sell goods (Module 5, DFD 3.0, D5).** Sales Personnel sell in the shop; marketplaces send signed webhooks. Stock is deducted atomically, a PDF invoice is produced and returns restore stock.
7. **Forecast (Module 6, DFD 5.0, D6).** Every sale feeds the forecast. The Manager sees predicted demand, reorder points and can turn a suggestion into a draft purchase order (back to step 5).
8. **Run and prove (Module 8).** Demo data, scripts, automated tests, the traceability matrix and this report make the system repeatable.

5. Traceability to the Earlier Experiments

Every module is traced to the requirement, use case, class, sequence diagram and DFD process it implements. The diagrams below are the ones submitted in Experiments 2 to 6; the complete requirement-by-requirement matrix is in `docs/traceability.md`.

Module	SRS feature	Use case (Exp. 2)	Classes (Exp. 3)	Sequence (Exp. 4)	DFD (Exp. 6)
1	4.1 Auth & RBAC	Manage Users and Roles; View System Reports & Audit Logs	User, Administrator	–	1.0, 2.0 · D1
2	4.2 Inventory, voice, sync	Manage Inventory (Add / Update / Adjust / Delete / Search); Voice Command	Product, StockRecord, WarehouseStaff	SD-1	4.0 · D3, D4
3	4.3 Supplier	Manage Suppliers	Supplier	SD-2 (first messages)	2.0 · D2
4	4.4 Purchase Order	Manage Purchase Orders; Receive Goods / Update Stock	PurchaseOrder, Manager, WarehouseStaff	SD-2	6.0 · D7
5	4.5 Sales	Process Sales Order; Process Returns / Cancellations	SalesOrder, SalesPersonnel	SD-3	3.0 · D5
6	4.6 Forecasting	Generate Demand Forecast; View Analytics Dashboard	Forecast	–	5.0 · D6

Module	SRS feature	Use case (Exp. 2)	Classes (Exp. 3)	Sequence (Exp. 4)	DFD (Exp. 6)
7	4.7 Notifications	Low Stock Alert, Shipping Delay Alert («extend»)	Notification	SD-1 alt, SD-2 opt	7.0 · D8

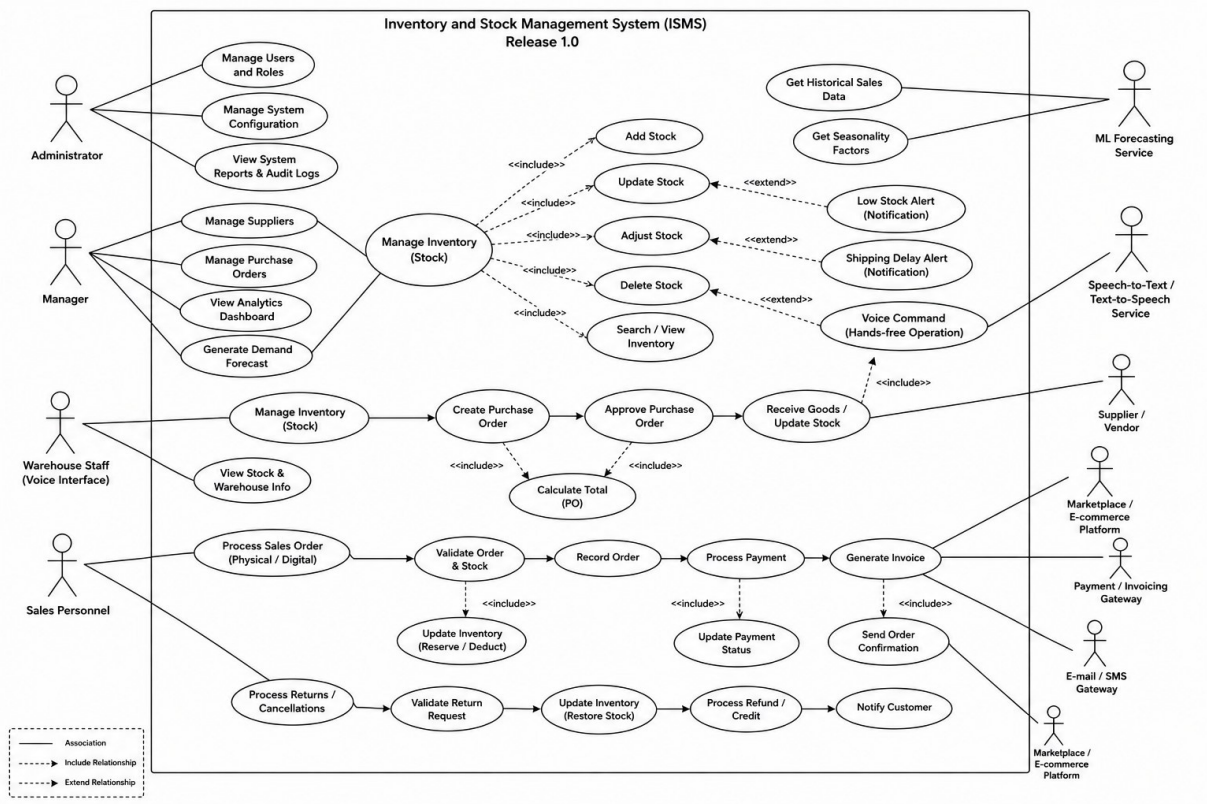


Fig. 5: Use-case diagram (Experiment 2): the use cases implemented by the seven modules

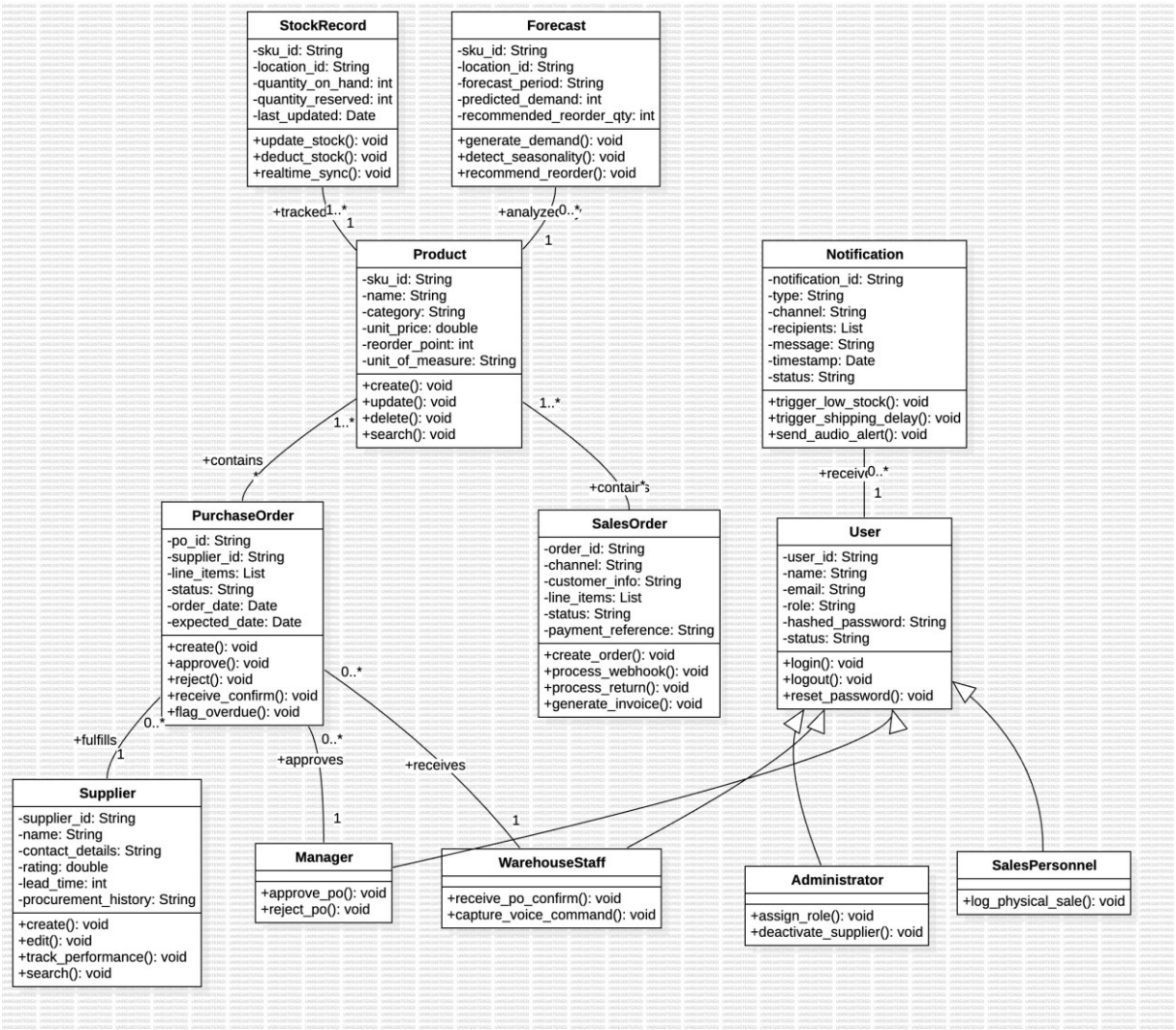


Fig. 6: Class diagram (Experiment 3): 12 classes, 43 operations and 9 relationships; each is checked by ClassContractTest and ClassDiagramStructureTest

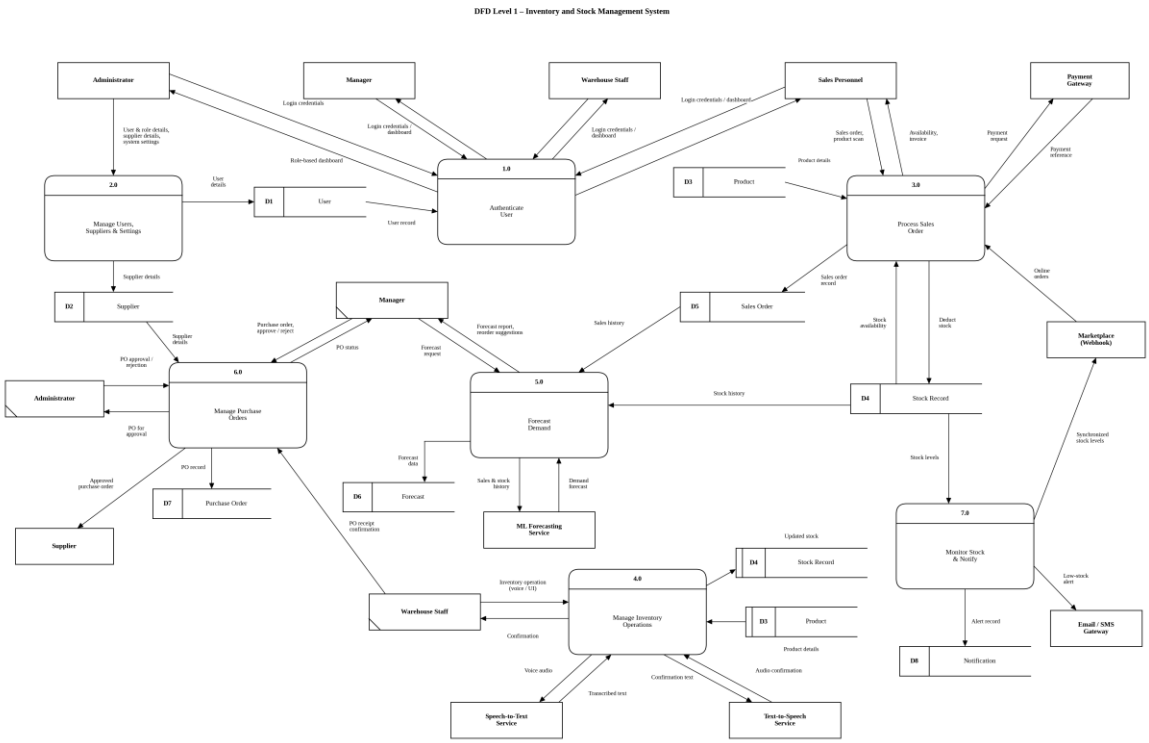


Fig. 7: DFD Level 1 (Experiment 6): processes 1.0–7.0 and data stores D1–D8

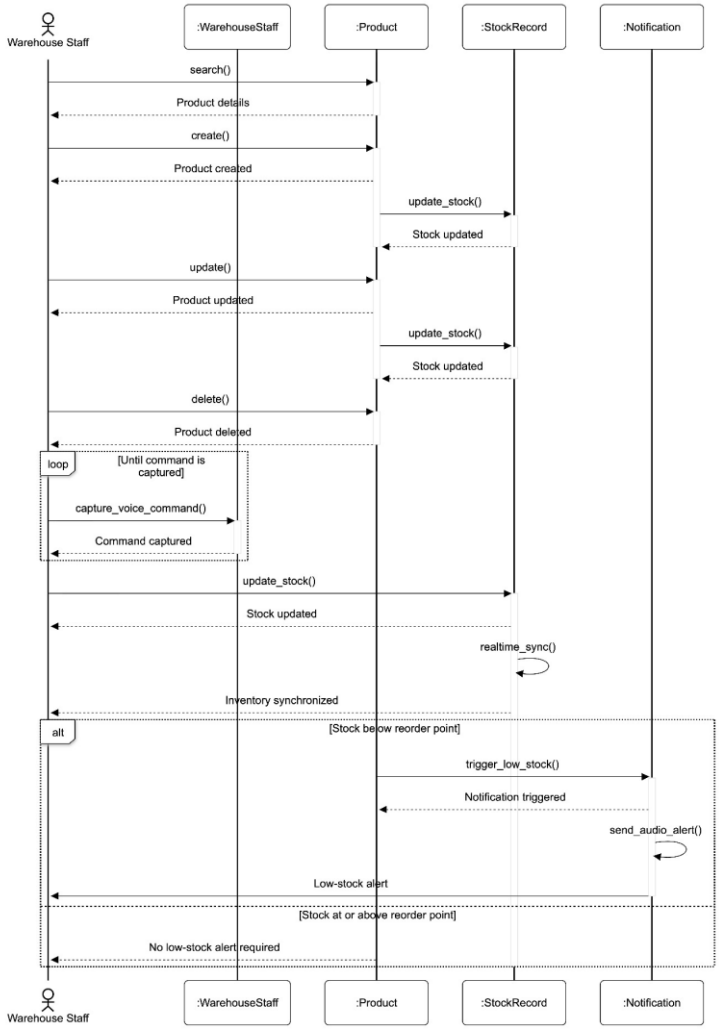


Fig. 8: Sequence diagram SD-1 (Experiment 4): inventory update and voice command, replayed message by message by Seq1InventoryTest

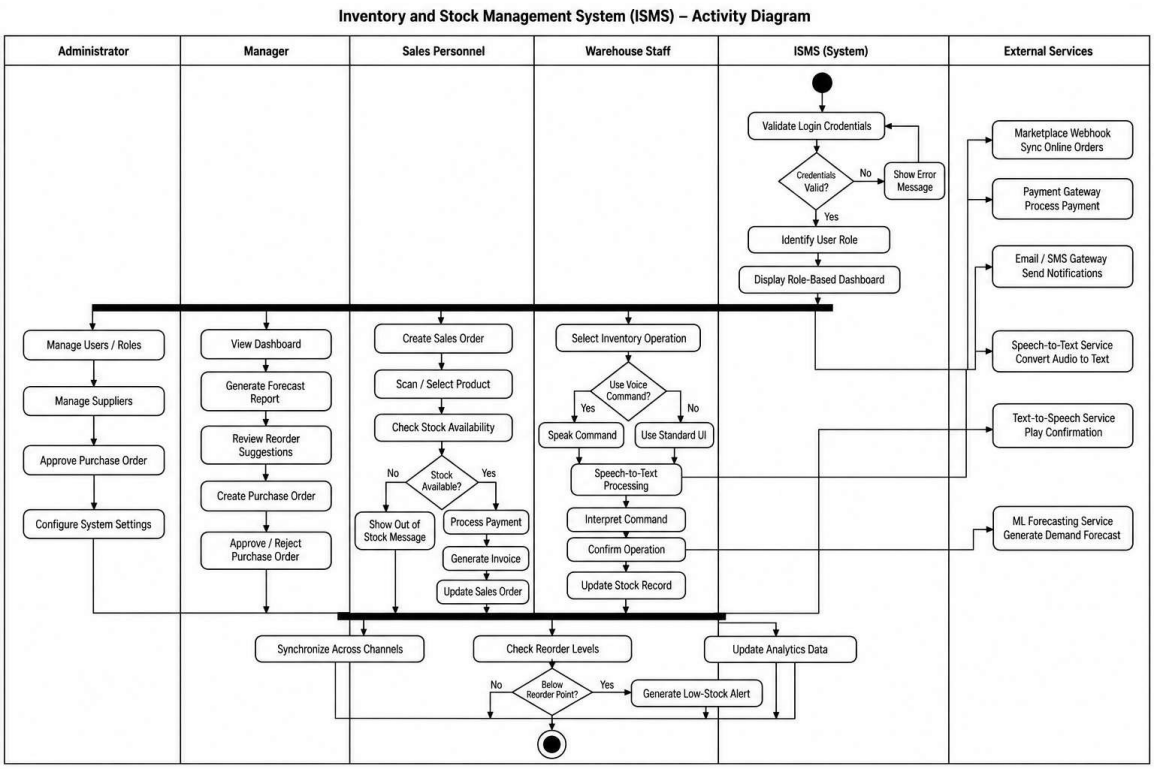


Fig. 9: Activity diagram (Experiment 5): the swimlane flow whose decisions are covered by the white-box test cases

3. Core Module 1: Authentication & Role-Based Access Control

Traceability: SRS 4.1 (`Auth.Login`, `Auth.Login.Fail`, `Auth.Password.Reset`, `Auth.RBAC.Roles`, `Auth.RBAC.Assign`, `Auth.RBAC.Enforce`, `Auth.Session.Timeout`, `Auth.Audit.Log`) · SE-1, SE-2, SE-3, SE-5, BR-1, BR-5, UI-1, UI-2, UD-1, UD-2 · Use cases "Manage Users and Roles", "View System Reports & Audit Logs" · Classes `User` (login, logout, reset_password) and `Administrator` (assign_role, deactivate_supplier) · DFD 1.0 Authenticate User (Validate Login Credentials → Identify User Role → Display Role-Based Dashboard) and 2.0, data store D1 · Activity nodes "Credentials valid?", "Account locked?".

Functionalities implemented

- **Login with generic failure:** user ID or e-mail plus password, BCrypt verification. A wrong user or a wrong password returns the same message, so an attacker cannot learn which user IDs exist.
- **Account lockout (SE-1):** five consecutive failures within 15 minutes lock the account. The lock is **stored in the database** (`users.status = locked`), so the Administrator sees it in the user list and unlocks it; even the correct password is refused while locked.
- **Password reset by e-mail:** a single-use, 30-minute token is e-mailed to the registered address; the request always answers the same generic message so existing accounts are not revealed.
- **Role-based access control:** four roles. The server checks the role on every endpoint (`401` without a valid token, `403` for the wrong role); the client hides menu items and shows an "Access Restricted" page if a user types a forbidden address.
- **Sliding 30-minute session:** every response carries a refreshed token (`X-Refreshed-Token`); after 30 idle minutes the next request is answered with `401` and the client returns to the login page.
- **Administration (class `Administrator`):** create users with inline validation, assign a role, deactivate or unlock accounts, and read the system limits (read-only).
- **Audit log (SE-5):** every login, failed login, lock, logout, user creation and role change is written as a structured, timestamped event and shown to the Administrator with filters.
- **Design system (UI-1, UI-2):** one visual language for all roles, a persistent help button that reopens the role walkthrough, and a light / black dark theme.

Key implementation code

Listing 1: The fifth failed login locks the account and the lock is committed (SE-1, Auth.Login.Fail)

```
// UserService.java
// noRollbackFor: the 5th failed login throws ACCOUNT_LOCKED *after* setting status=locked; that change must
be
// committed, otherwise the lock exists only in memory and the Administrator never sees (or can unlock) the
account.
@Transactional(noRollbackFor = ApiException.class)
public Map<String, Object> login(String identifier, String password) {
    if (identifier == null || identifier.isBlank() || password == null) {
        throw new ApiException(HttpStatus.BAD_REQUEST, "INVALID_INPUT", "Identifier and password are
required");
    }

    if (loginAttemptCache.isLocked(identifier)) {
        throw new ApiException(HttpStatus.FORBIDDEN, "ACCOUNT_LOCKED", "Account locked due to 5 failed
attempts in 15 minutes");
    }

    User user = userRepository.findByUserIdOrEmail(identifier).orElse(null);

    if (user == null || !passwordEncoder.matches(password, user.getHashedPassword())) {
        int failures = loginAttemptCache.recordFailure(identifier);
        if (user != null && failures >= 5) {
            user.setStatus(UserStatus.LOCKED);
            userRepository.save(user);
            auditLogger.log(user.getUserId(), "ACCOUNT_LOCKED", "users", Map.of("reason", "5_failed_logins"));
            throw new ApiException(HttpStatus.FORBIDDEN, "ACCOUNT_LOCKED", "Account locked due to 5 failed
attempts in 15 minutes");
        }
        auditLogger.log(identifier, "LOGIN_FAILED", "users", Map.of("failure_count", failures));
        throw new ApiException(HttpStatus.UNAUTHORIZED, "INVALID_CREDENTIALS", "Invalid user ID/email or
password");
  ...
```

Listing 2: Unauthenticated requests are 401 and wrong-role requests 403 (Auth.RBAC.Enforce, Auth.Session.Timeout; package names abridged)

```
// SecurityConfig.java
http
    ...
    // Unauthenticated / expired-token requests must be 401 (not Spring's default 403)
    // so the UI's session-timeout handling can redirect to the login page.
    .exceptionHandling(ex -> ex
        .authenticationEntryPoint(new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED))
        .accessDeniedHandler((req, res, e) -> res.setStatus(HttpStatus.FORBIDDEN.value())))
    .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
```

Listing 3: Sliding session: the token is renewed on every authenticated request

```
// JwtAuthFilter.java
if (claims != null && SecurityContextHolder.getContext().getAuthentication() == null) {
    String userId = jwtService.getUserId(claims);
    Role role = jwtService.getRole(claims);

    String roleName = "ROLE_" + role.name();
    SimpleGrantedAuthority authority = new SimpleGrantedAuthority(roleName);

    UsernamePasswordAuthenticationToken authentication =
            new UsernamePasswordAuthenticationToken(userId, null, Collections.singletonList(authority));
    authentication.setDetails(claims);
    SecurityContextHolder.getContext().setAuthentication(authentication);

    // Sliding expiration: send refreshed token in header
    String refreshed = jwtService.refreshToken(claims);
    response.setHeader("X-Refreshed-Token", refreshed);
}
```

Output screenshots: Module 1

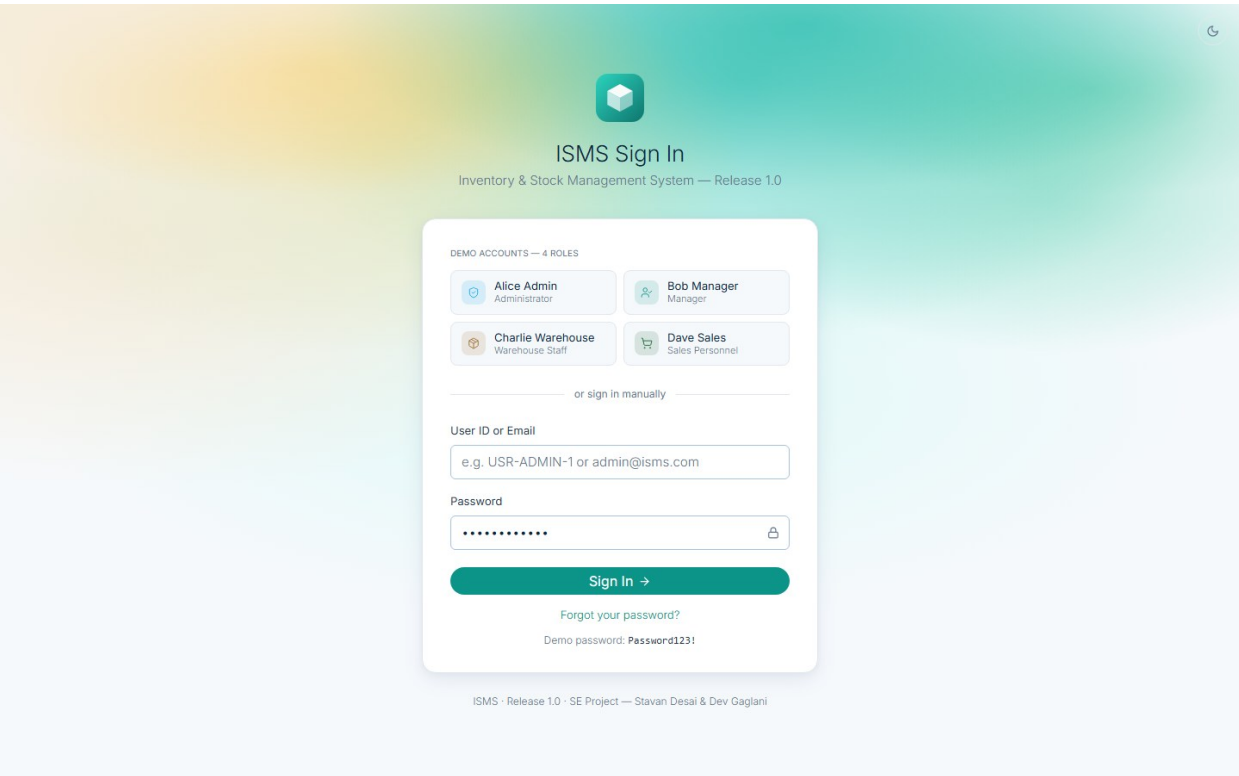


Fig. 10: Login page: demo accounts for the four roles, manual sign-in and the "Forgot your password?" link

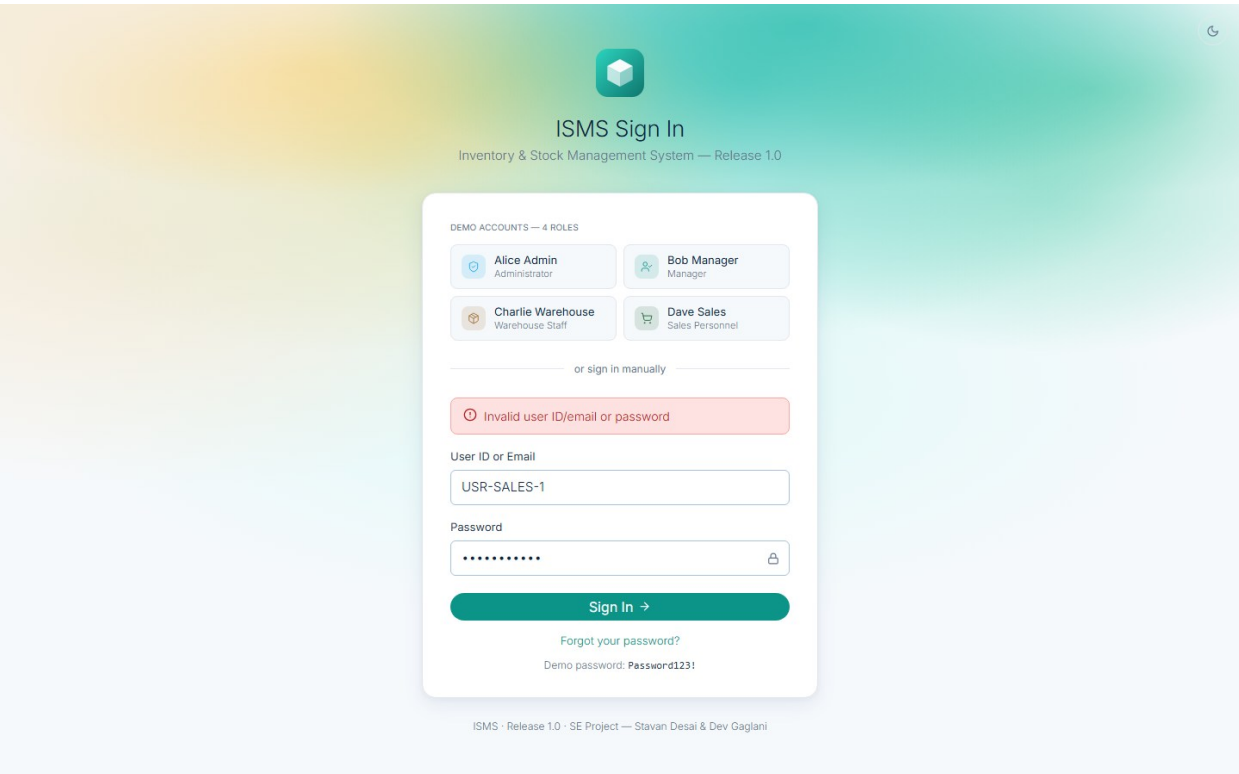


Fig. 11: Wrong password: a generic error that does not reveal whether the user exists (Auth.Login.Fail)

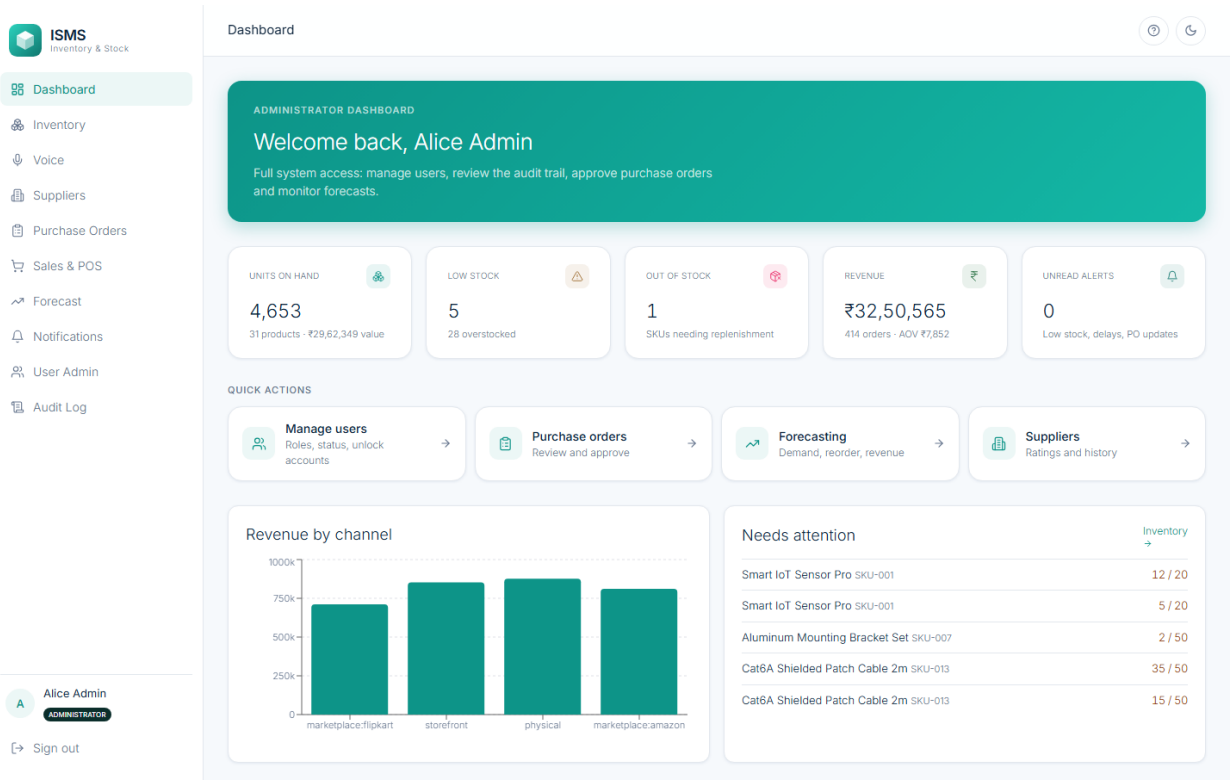


Fig. 12: First sign-in: the role-specific onboarding walkthrough (UD-1, UD-2); it can be reopened from the help button

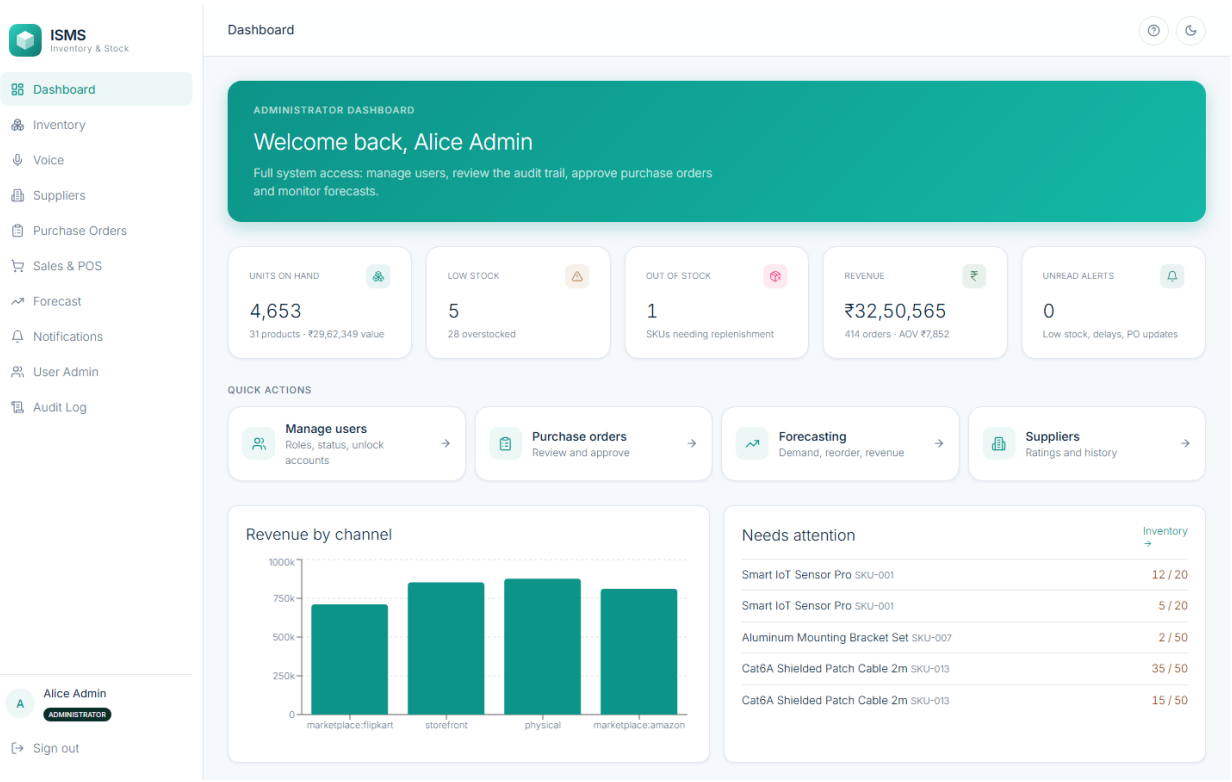


Fig. 13: Administrator dashboard (DFD 1.0 "Display Role-Based Dashboard"): live KPIs, quick actions and the full menu

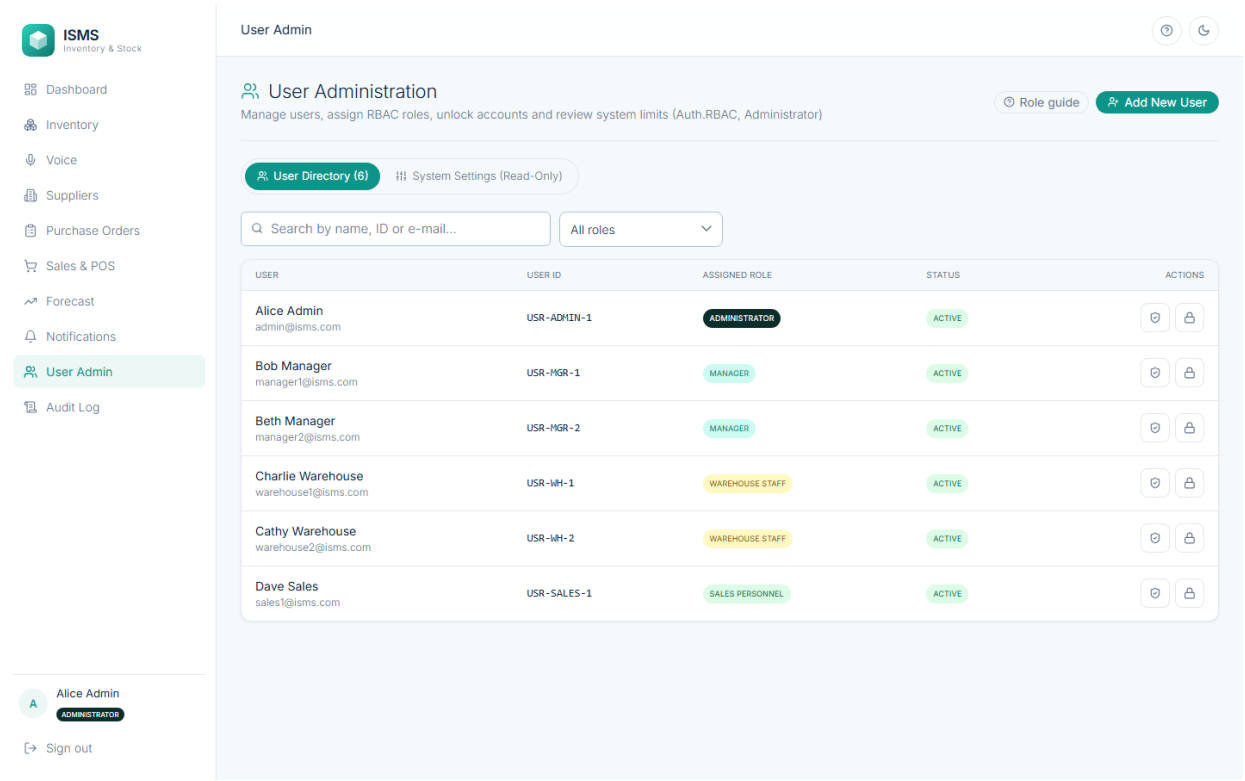


Fig. 14: User Administration: directory with role and status of every user (Administrator only)

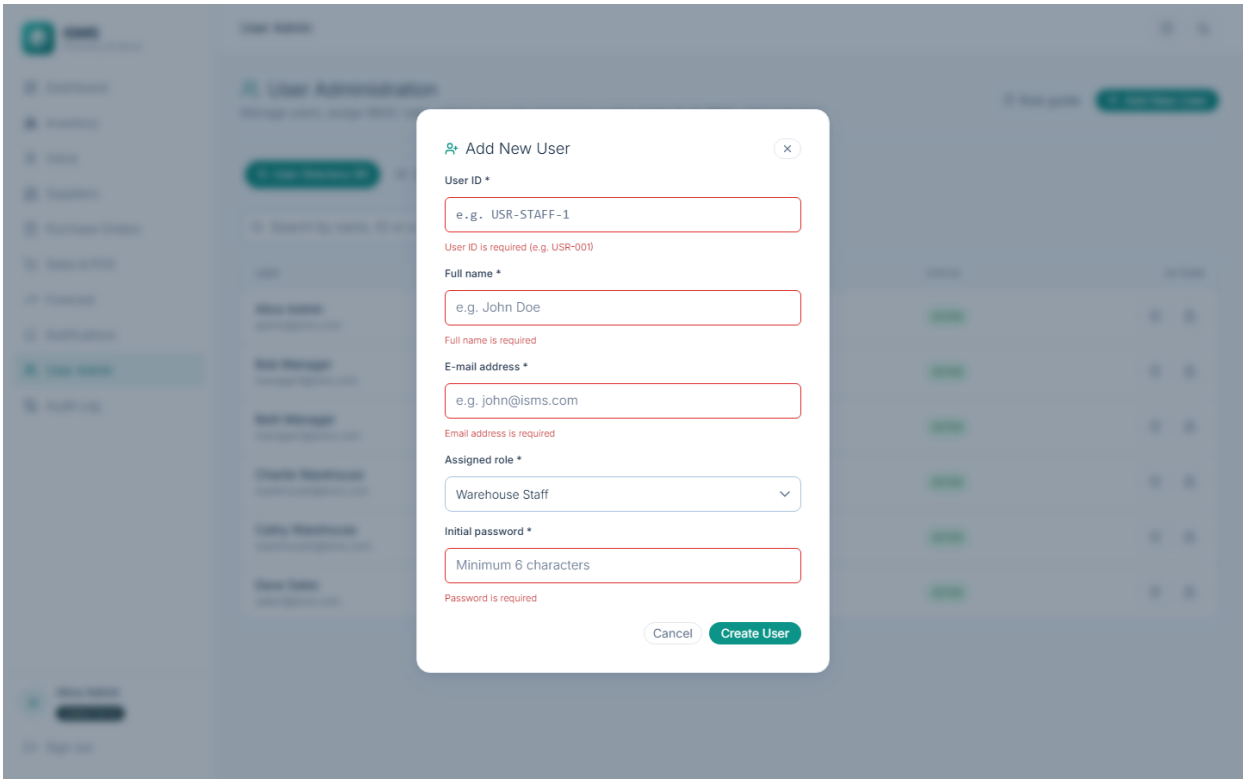


Fig. 15: Inline validation (UI-4): the empty form shows field-level messages before anything is sent to the server

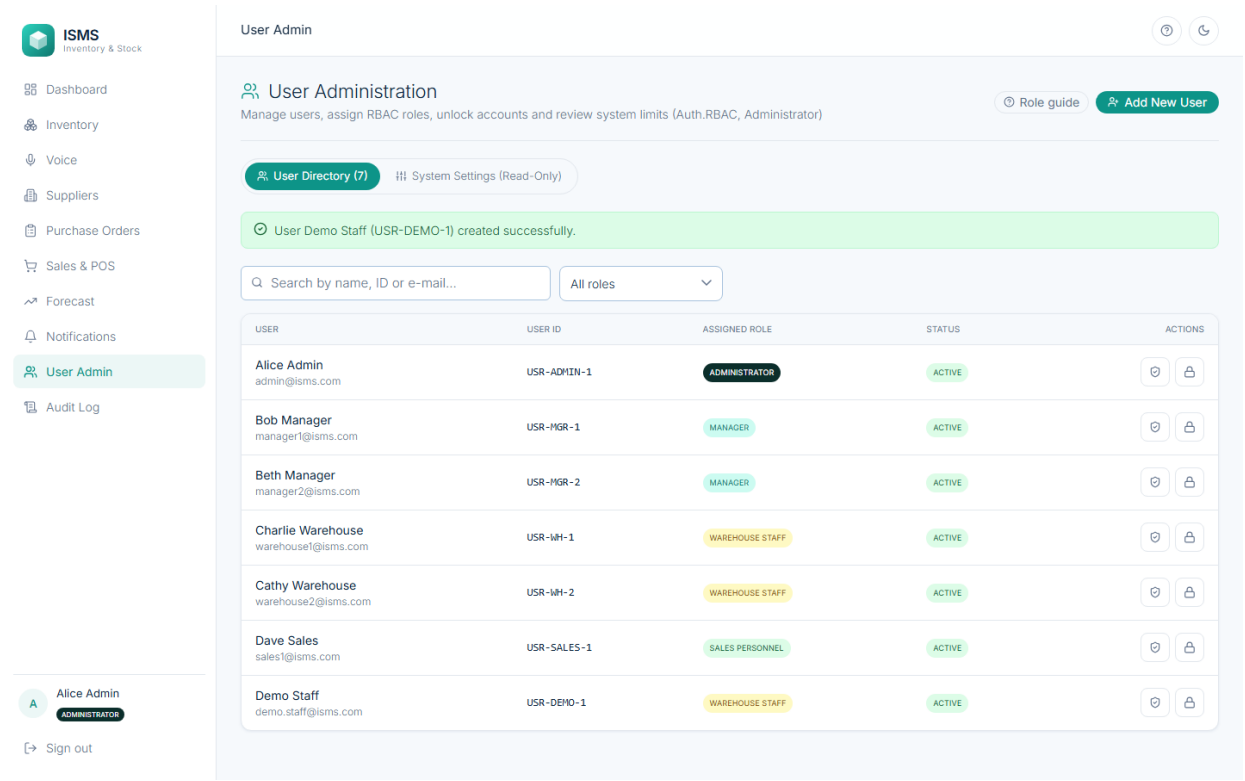


Fig. 16: User created: the new Demo Staff row appears with the Warehouse Staff role

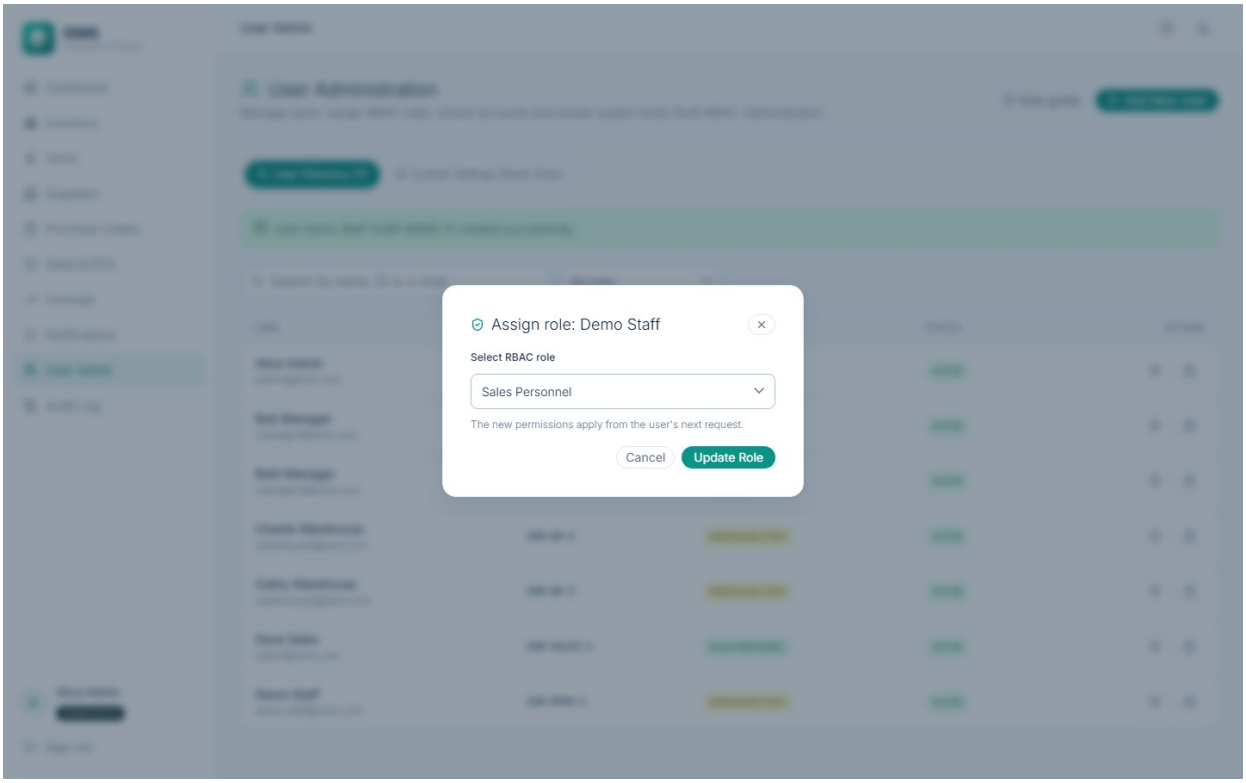


Fig. 17: Assign role (Administrator.assign_role): Demo Staff is changed to Sales Personnel

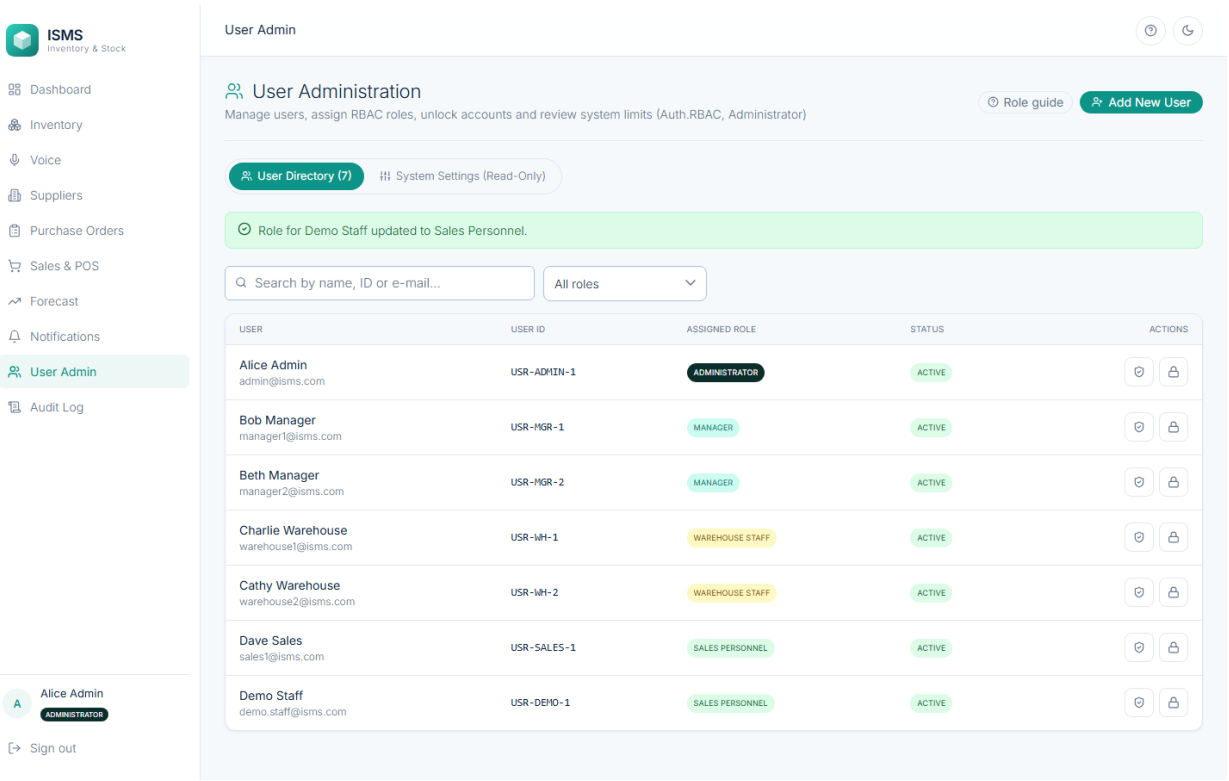


Fig. 18: Role updated; the change applies from the user's next request

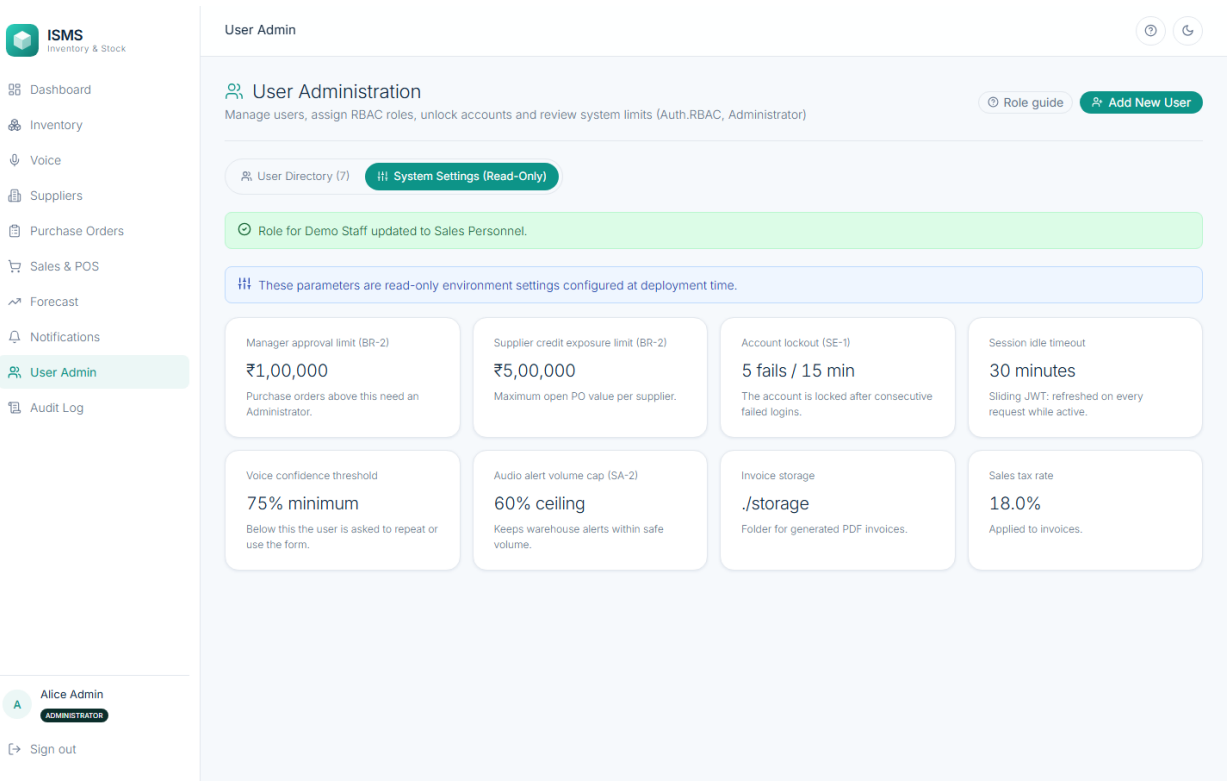


Fig. 19: System Settings (read-only): approval limit, credit limit, lockout policy, session timeout, voice threshold, volume cap

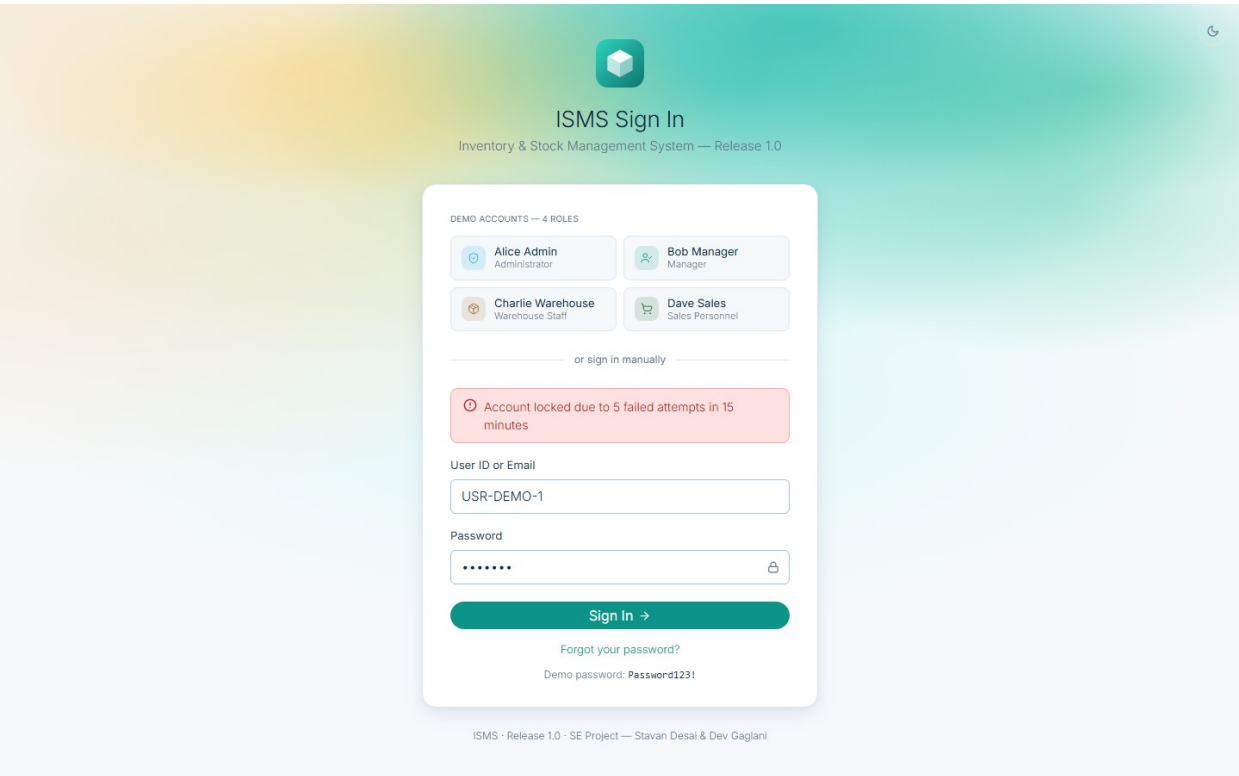


Fig. 20: Fifth consecutive wrong password for USR-DEMO-1: the account is locked (SE-1)

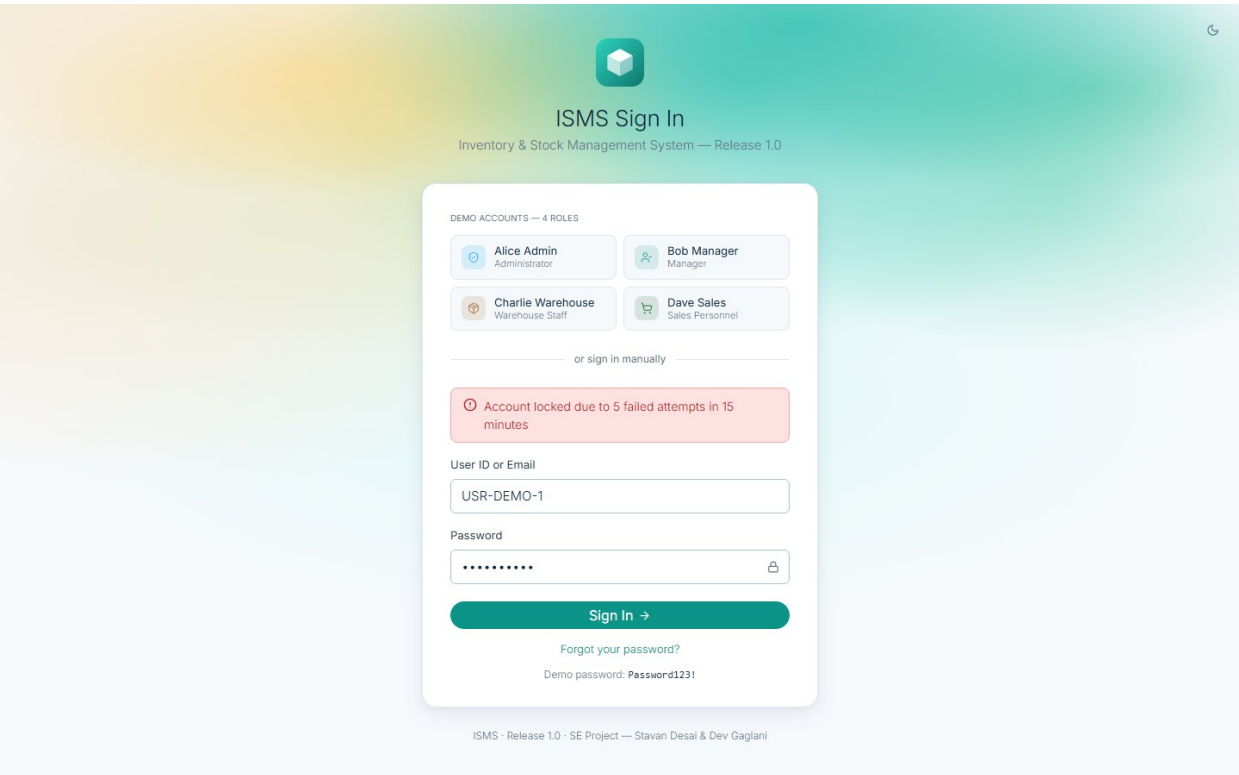


Fig. 21: While locked, even the correct password is refused

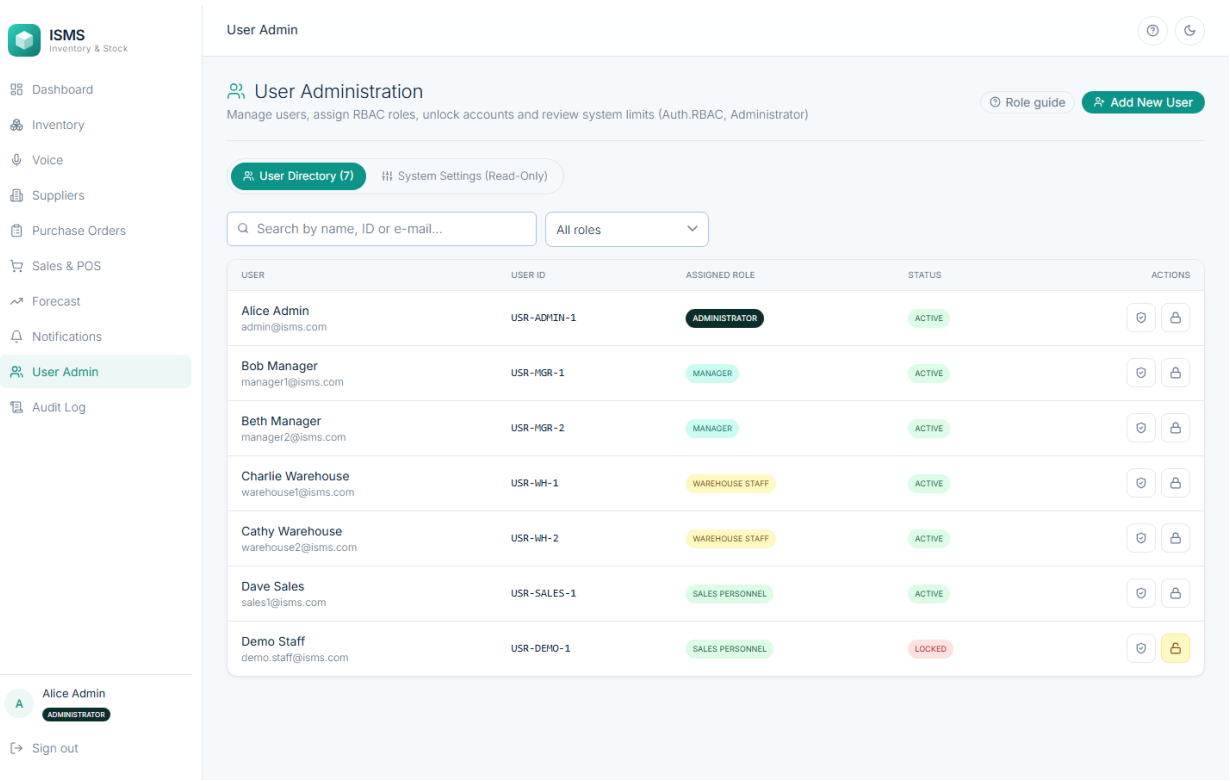


Fig. 22: Administrator view: the locked user is visible with a highlighted unlock action, because the lock is stored in users.status

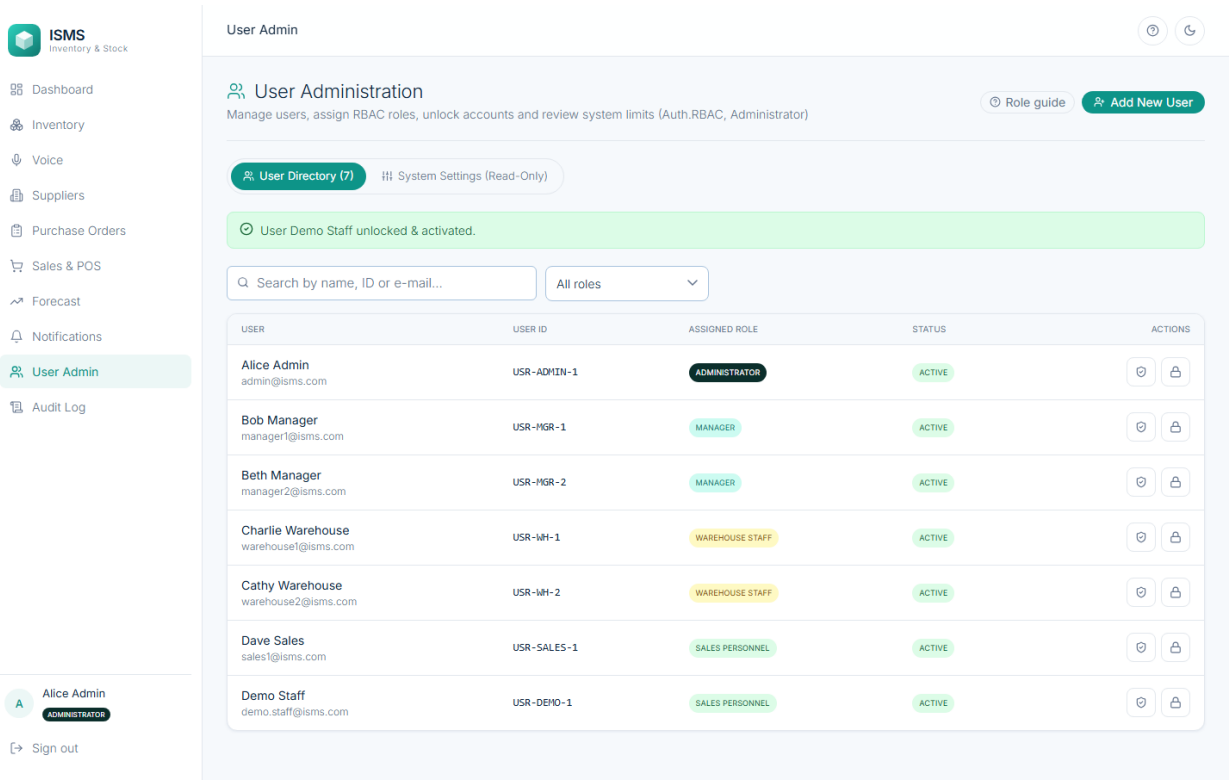


Fig. 23: After the Administrator unlocks the account, the user is Active again and can sign in

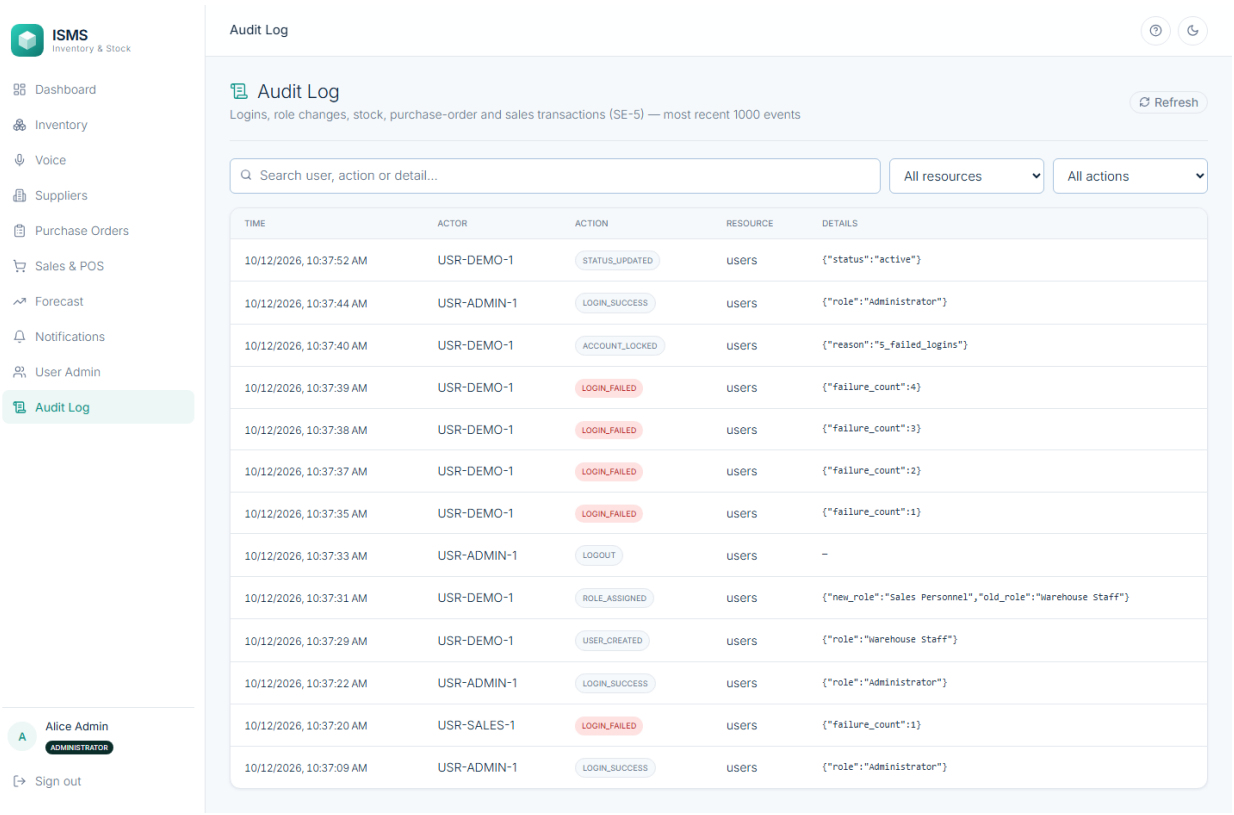


Fig. 24: Audit log (SE-5): LOGIN_SUCCESS, LOGIN_FAILED, ACCOUNT_LOCKED, LOGOUT, USER_CREATED and ROLE_ASSIGNED events with actor and time

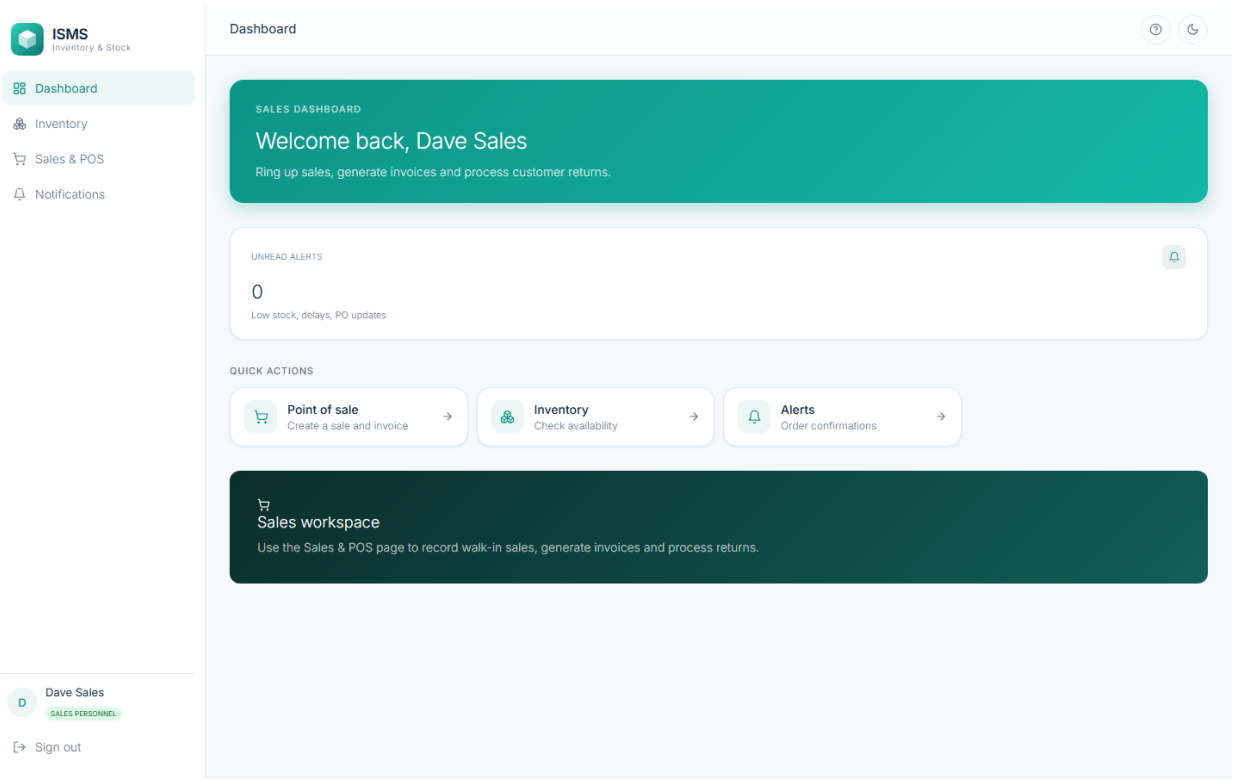


Fig. 25: Sales Personnel: only the menu items allowed for the role are shown (Auth.RBAC.Enforce)

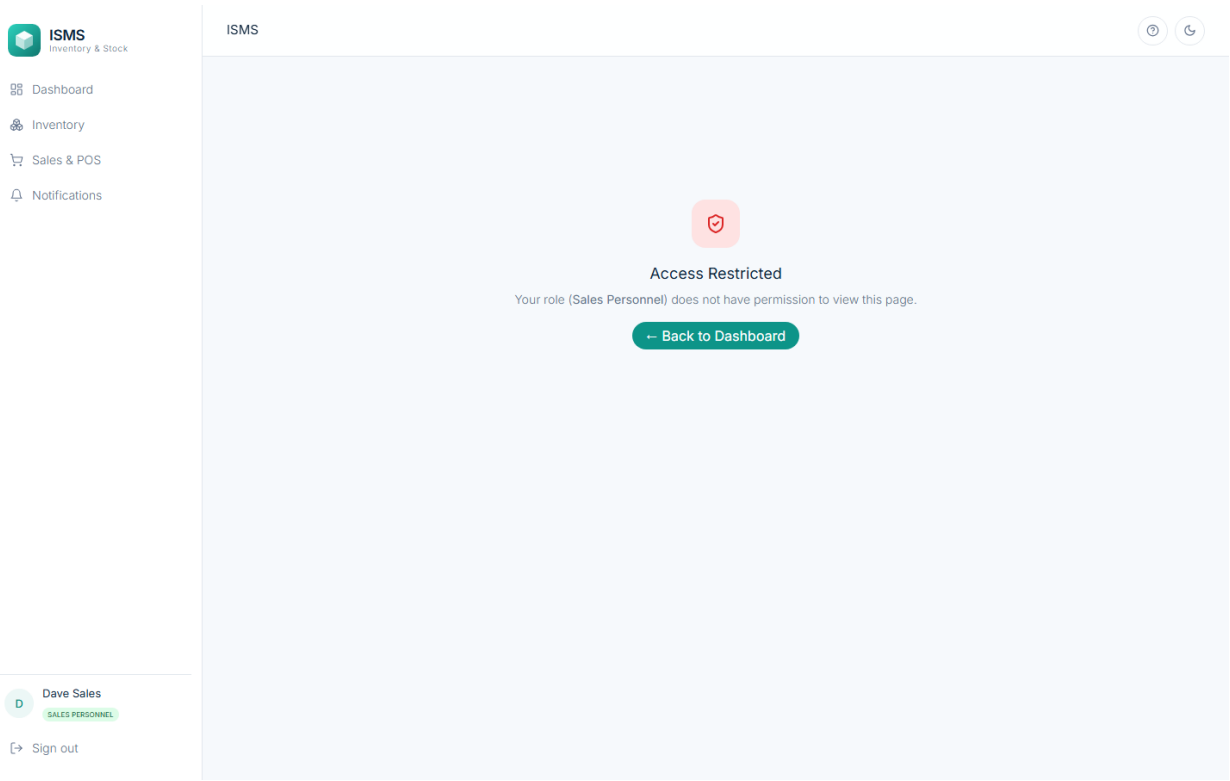


Fig. 26: A Sales Personnel user who types /users in the browser is shown "Access Restricted"

PowerShell - ISMS API: authentication and role checks (SE-2, SE-3)

```
$ curl -i -X POST /api/v1/auth/login  -d {"user_id":"USR-SALES-1","password":"********"}
HTTP 200  {"token": "eyJhbGciOiJIUzM4NCJ9.eyJzdWI...", "user": {"userId": "USR-SALES-1", "status": "active", "email": "sales1@isms.com", "name": "Dave Sale
s", "role": "Sales Personnel"}}

$ curl -i /api/v1/users                    (no token)
HTTP 401  <empty body>

$ curl -i /api/v1/users -H 'Authorization: Bearer not-a-jwt'
HTTP 401  <empty body>

$ curl -i /api/v1/users                    (Sales Personnel token)
HTTP 403  <empty body>

$ curl -i /api/v1/admin/audit-log          (Sales Personnel token)
HTTP 403  <empty body>

$ curl -i /api/v1/products                 (Sales Personnel token, allowed)
HTTP 200  31 products returned

$ curl -i /api/v1/auth/me
HTTP 200  {"userId":"USR-SALES-1","status":"active","email":"sales1@isms.com","name":"Dave Sales","role":"Sales Personnel"}

$ curl -i /api/v1/products   (sliding session, F1.6)
HTTP 200
X-Refreshed-Token: eyJhbGciOiJIUzM4NCJ9.eyJzdWIiOiJVU1ItU0FMRVMtM...
```

Fig. 27: The API enforces the same rules: 401 without a valid token, 403 for the wrong role, 200 for allowed calls, and a refreshed token on each request

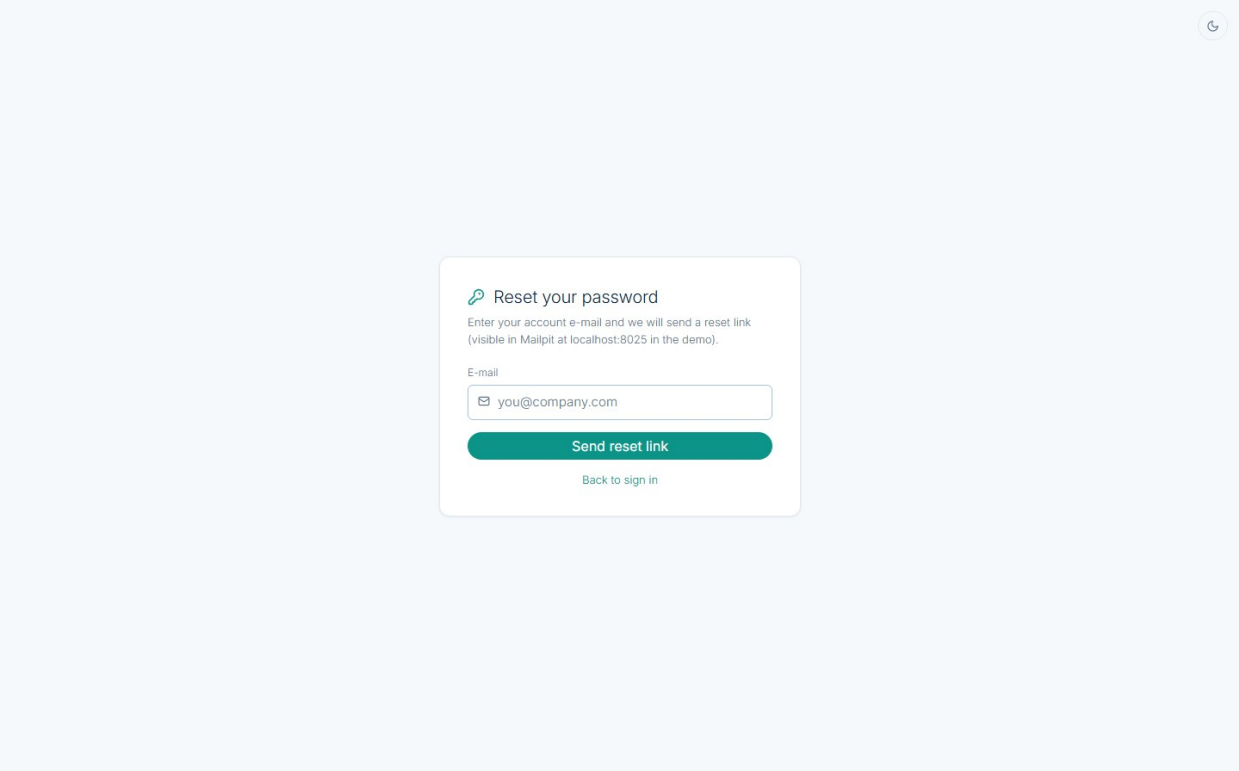


Fig. 28: Password reset, step 1: request a link for the registered e-mail address

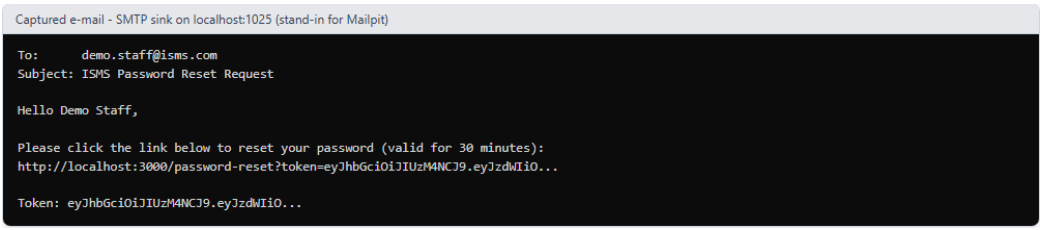


Fig. 29: Password reset, step 2: the e-mail received (captured by a local SMTP server on port 1025; Mailpit in the Docker set-up)

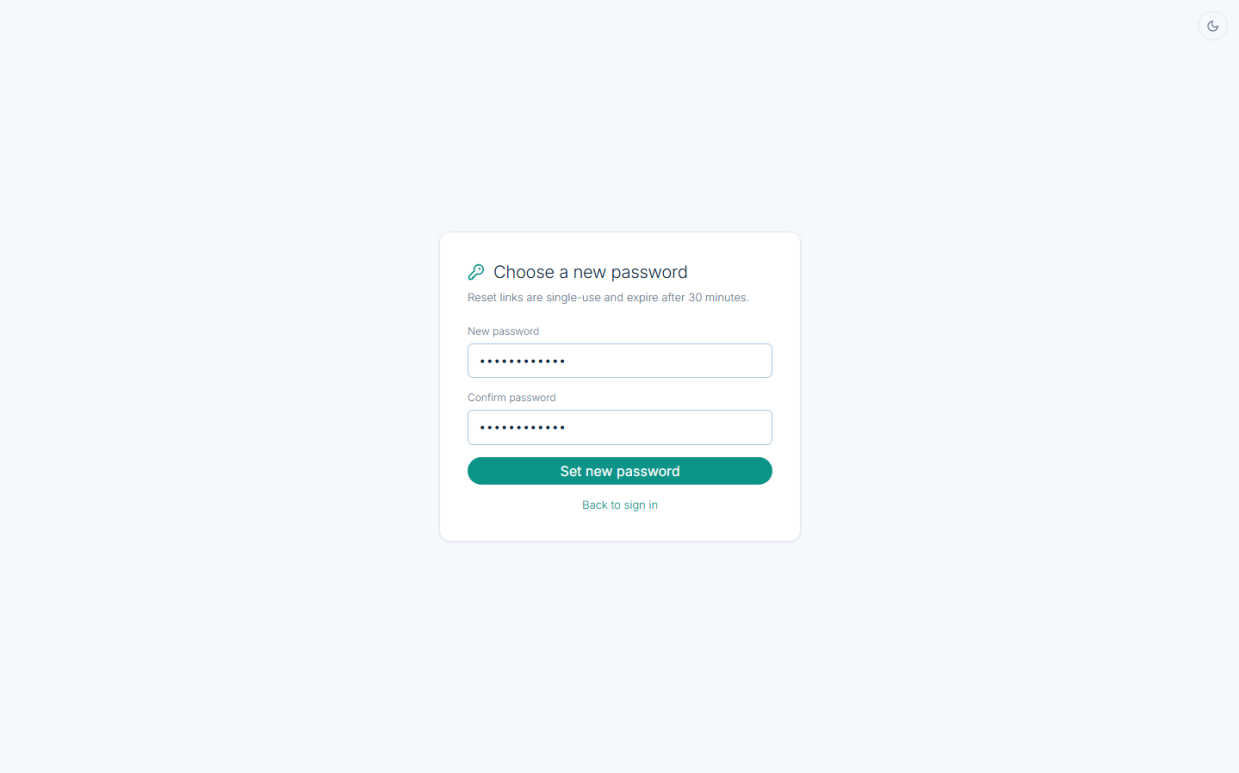


Fig. 30: Password reset, step 3: choose a new password using the single-use link

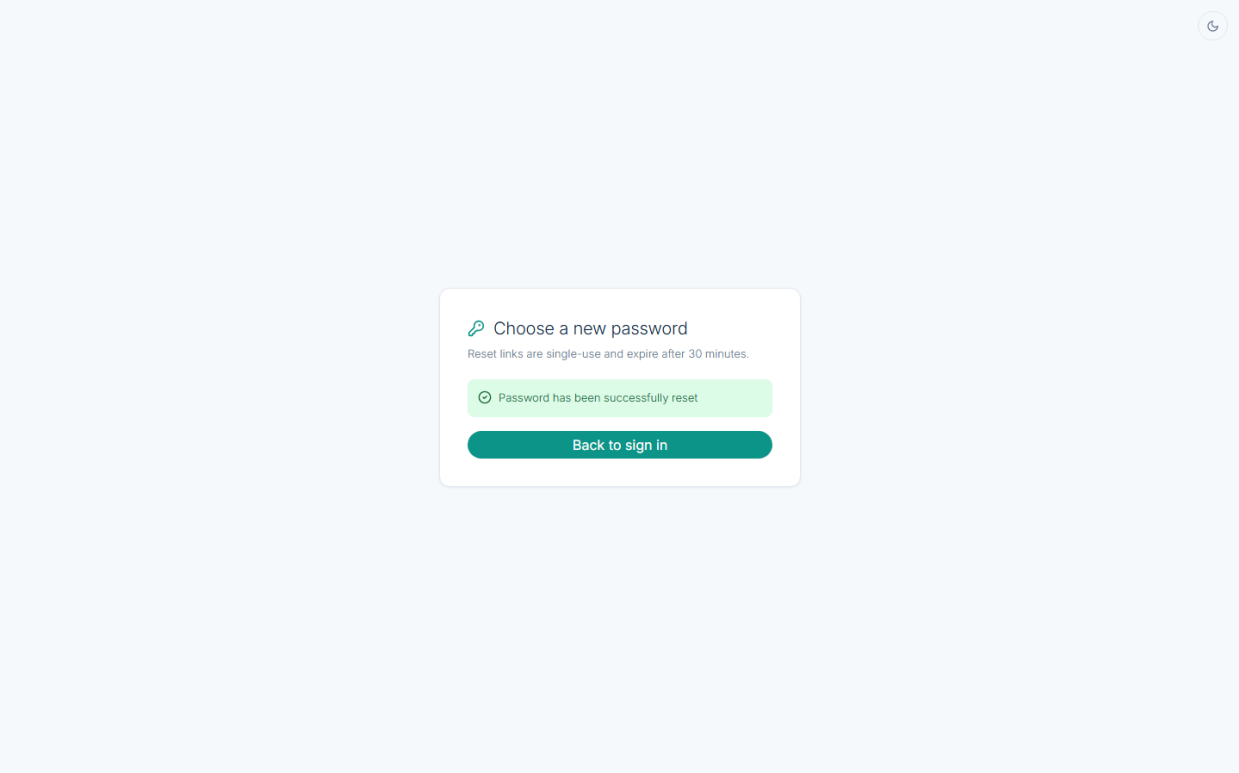


Fig. 31: Password changed successfully; the same link cannot be used a second time

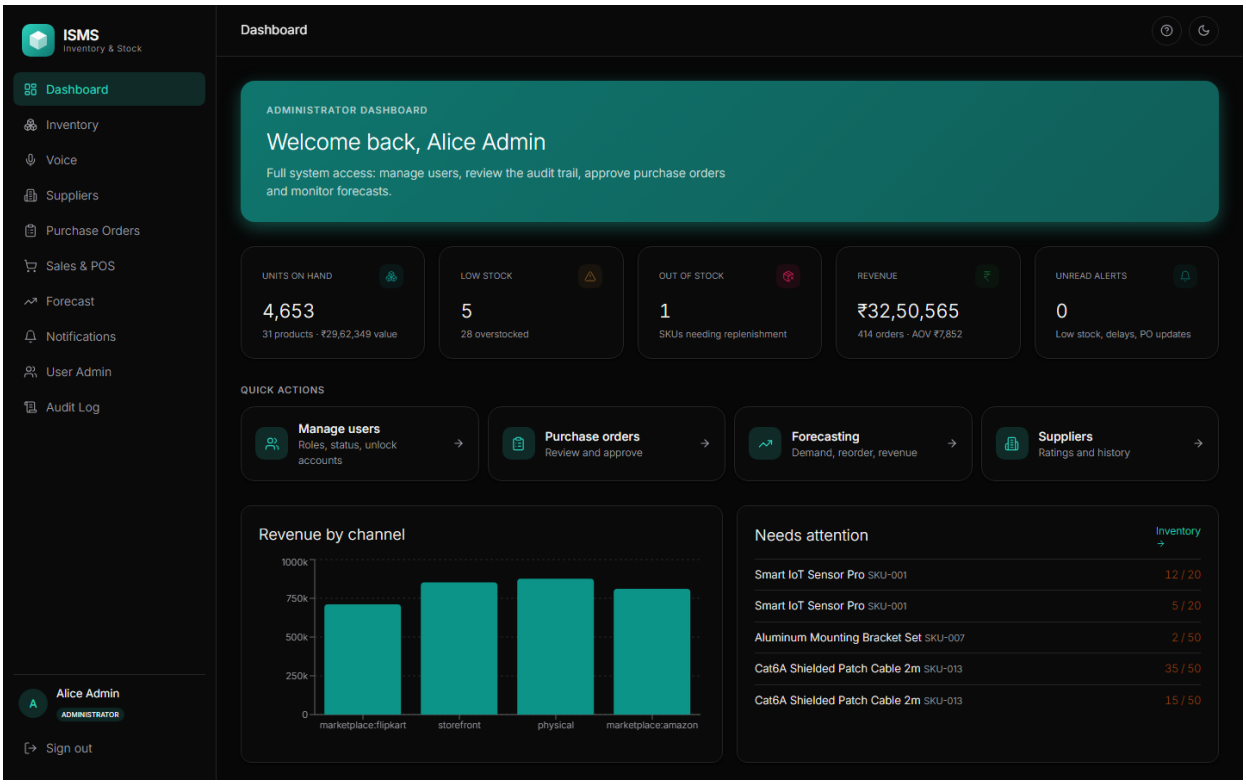


Fig. 32: Black dark theme of the same dashboard (switch at the top right; remembered per browser)

4. Core Module 2: Voice-Enabled Inventory & Omnichannel Synchronization

Traceability: SRS 4.2 (`Inventory.CRUD.Create / Update / Delete`, `Inventory.Search`, `Voice.Command.Capture / Interpret / Confirm / LowConfidence`, `Sync.Channel.Realtime`, `Sync.Channel.Conflict`) · BR-3, PE-1, PE-2, SA-2, Robustness-1, UI-3 · Use case "Manage Inventory (Stock)" «include» Add / Update / Adjust / Delete Stock and Search / View Inventory, with the «extend» "Low Stock Alert" and "Voice Command (Hands-free Operation)" · Classes `Product` (create, update, delete, search), `StockRecord` (update_stock, deduct_stock, realtime_sync) and `WarehouseStaff` (capture_voice_command) · Sequence diagram SD-1 (search → create → update_stock → update → update_stock → delete → loop capture_voice_command → update_stock → realtime_sync → alt trigger_low_stock / send_audio_alert) · DFD 4.0 Manage Inventory Operations, Level 2 steps 4.1 Select Inventory Operation → Use Voice Command? → 4.2 Speech-to-Text → 4.3 Interpret → 4.4 Confirm (Text-to-Speech) → 4.5 Update Stock Record · Data stores D3 Product, D4 Stock Record.

Functionalities implemented

- **Product CRUD:** create a product with SKU, name, category, unit, price and reorder point (an initial stock record is created); edit its attributes; delete it. A duplicate SKU is rejected with `409`, and a **delete is refused (`409 STOCK_EXISTS`) while any location still holds stock**.
- **Search and filter:** by SKU or name and by category, with stock on hand, reserved quantity and a status of In Stock / Low Stock / Out of Stock.
- **Stock updates under a row lock:** every change reads the stock row `FOR UPDATE`, so simultaneous requests are serialised. A change can never take stock below zero (BR-3); the same rule is also a database CHECK constraint.
- **Last-unit conflict (Sync.Channel.Conflict):** when two channels try to take the last unit at the same moment, the first confirmed transaction wins and the other receives an out-of-stock rejection.
- **Real-time synchronisation (PE-2):** after the commit, the new quantity is pushed over WebSocket `/ws/stock` to every open screen and to the channel adapters; an open Inventory page updates without reloading.
- **Low-stock alert (SD-1 `alt`):** after any stock change, if the quantity is below the reorder point a Low Stock notification and an audio alert are raised; otherwise the "no alert required" branch runs.
- **Voice commands:** the Warehouse Staff screen captures speech (or a typed fallback), interprets action, SKU, quantity and location (including spoken numbers such as "twenty"), reads the command back and waits for confirmation. Below the confidence threshold (0.75) the user is asked to repeat or use the form. Commands issued offline are queued and replayed idempotently by `client_id`.

Key implementation code

Listing 4: Stock deduction: row lock, fresh read, then the BR-3 check (StockRecord.deduct_stock)

```
// StockRecordService.java
StockRecord stock = stockRecordRepository.findBySkuIdAndLocationIdForUpdate(skuId, locationId)
        .orElseGet(() -> {
            StockRecord newStock = new StockRecord(
                    new StockRecordId(skuId, locationId),
                    product,
                    0,
                    0,
                    OffsetDateTime.now()
            );
            return stockRecordRepository.save(newStock);
        });


stock.setQuantityOnHand(quantityOnHand);
stock.setLastUpdated(OffsetDateTime.now());
```

Listing 5: Voice step of SD-1: the "Confidence ≥ threshold?" decision and the read-back before confirmation

```
// WarehouseStaffService.java
public Map<String, Object> captureVoiceCommand(String transcript, double confidence, String deviceId) {
    double threshold = appProperties.voice().confidenceThreshold();
    if (confidence < threshold) {
        return Map.of(
                "status", "low_confidence",
                "action", "repeat_or_use_ui",
                "confidence", confidence,
                "threshold", threshold
        );
    }


    VoiceInterpreter.InterpretationResult result = voiceInterpreter.interpret(transcript);
    if (!result.valid()) {
        return Map.of(
                "status", "low_confidence",
                "action", "repeat_or_use_ui",
                "error", result.error() != null ? result.error() : "Could not understand command"
        );
    }


    String token = "VCT-" + UUID.randomUUID().toString().substring(0, 8);
    pendingTokens.put(token, result);


    return Map.of(
            "status", "needs_confirmation",
            "token", token,
            "spoken_text", result.spokenText(),
    ...
```

Output screenshots: Module 2

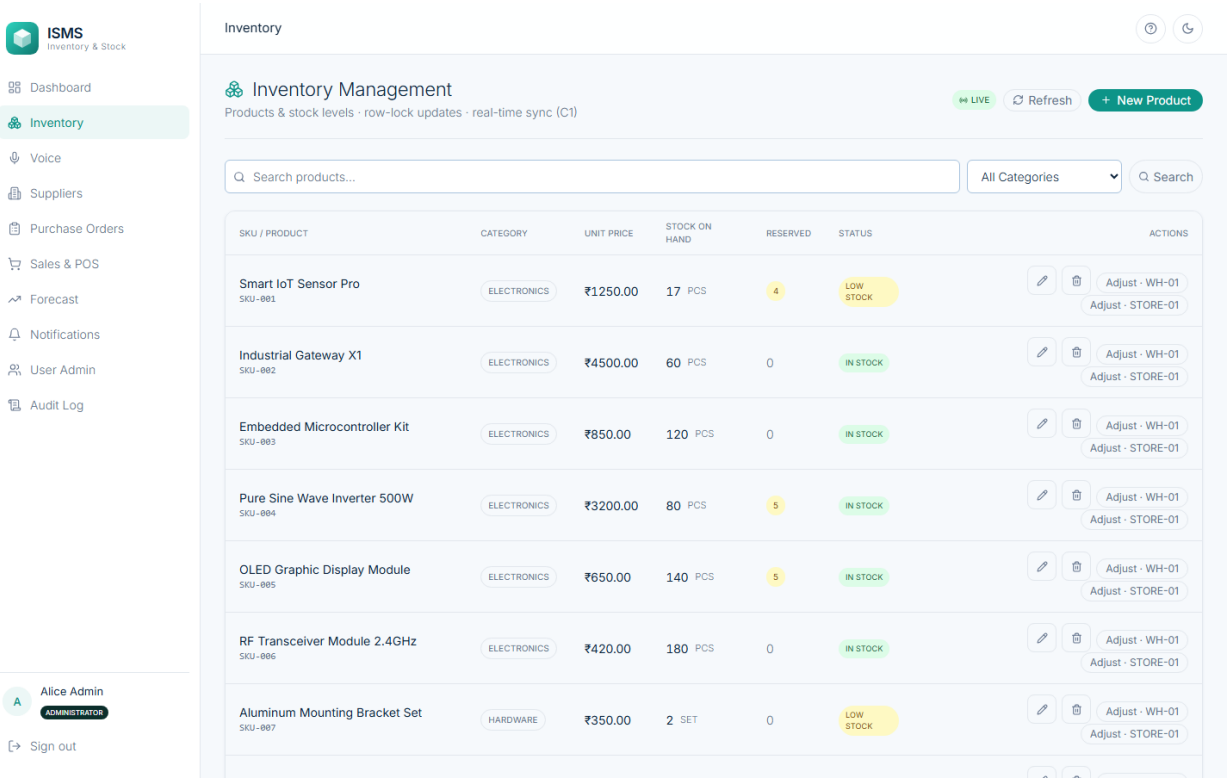


Fig. 33: Inventory list: products with stock on hand, reserved quantity and status; the LIVE badge shows the WebSocket connection

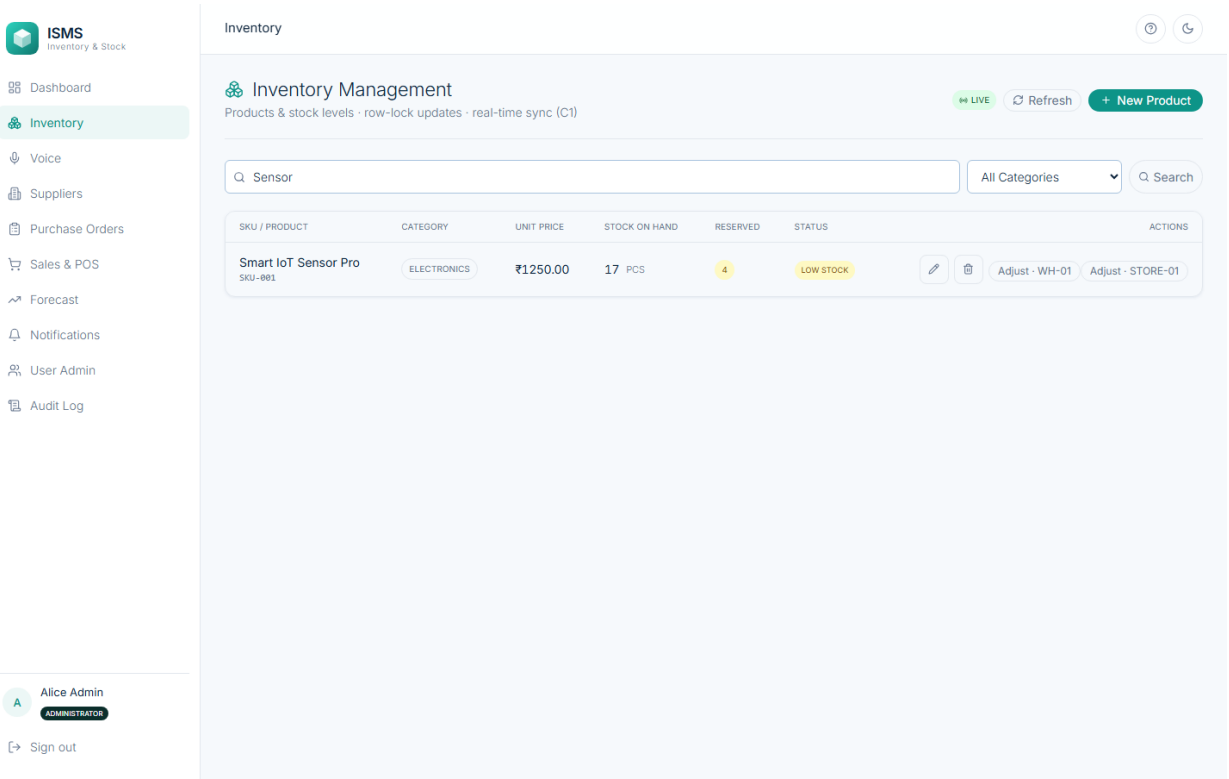


Fig. 34: Search by name: only the matching product is listed (Inventory.Search)

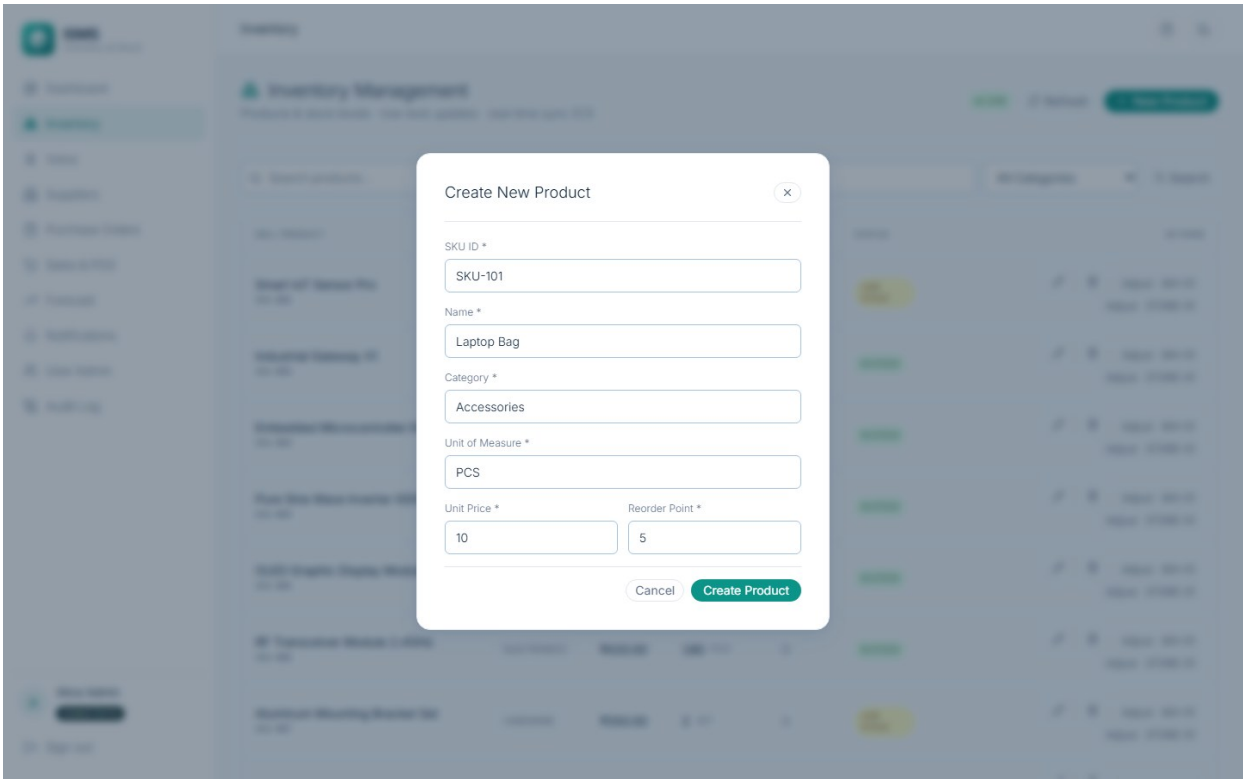


Fig. 35: Create a product: SKU, name, category, unit of measure, price and reorder point (Inventory.CRUD.Create)

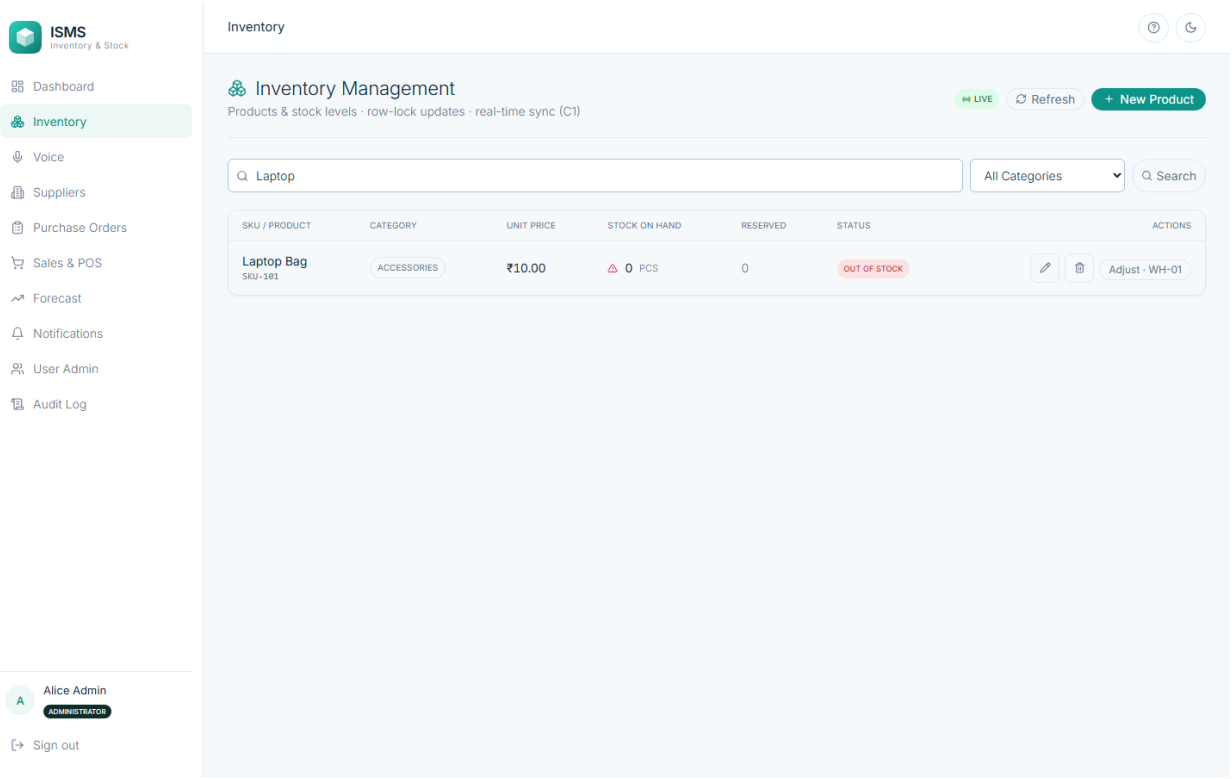


Fig. 36: Product SKU-101 created with an initial stock of 0

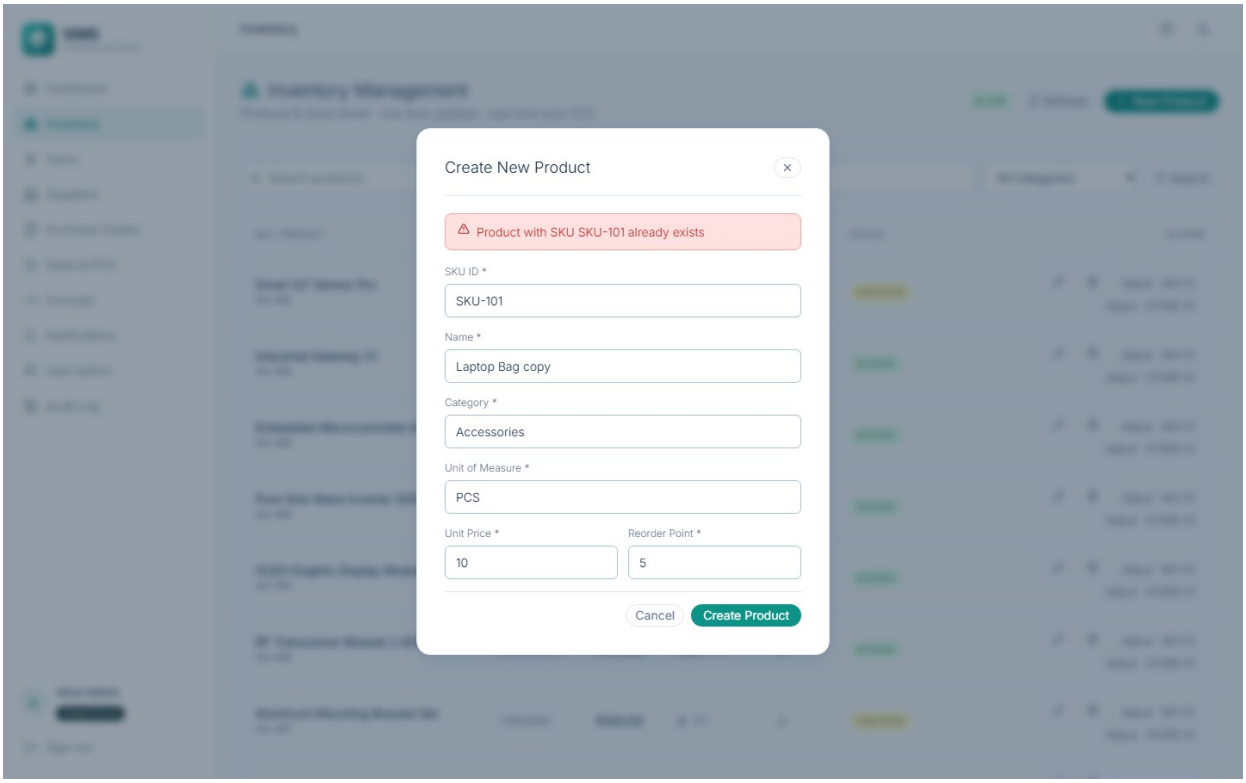


Fig. 37: Duplicate SKU: the second create is refused with a clear message (409)

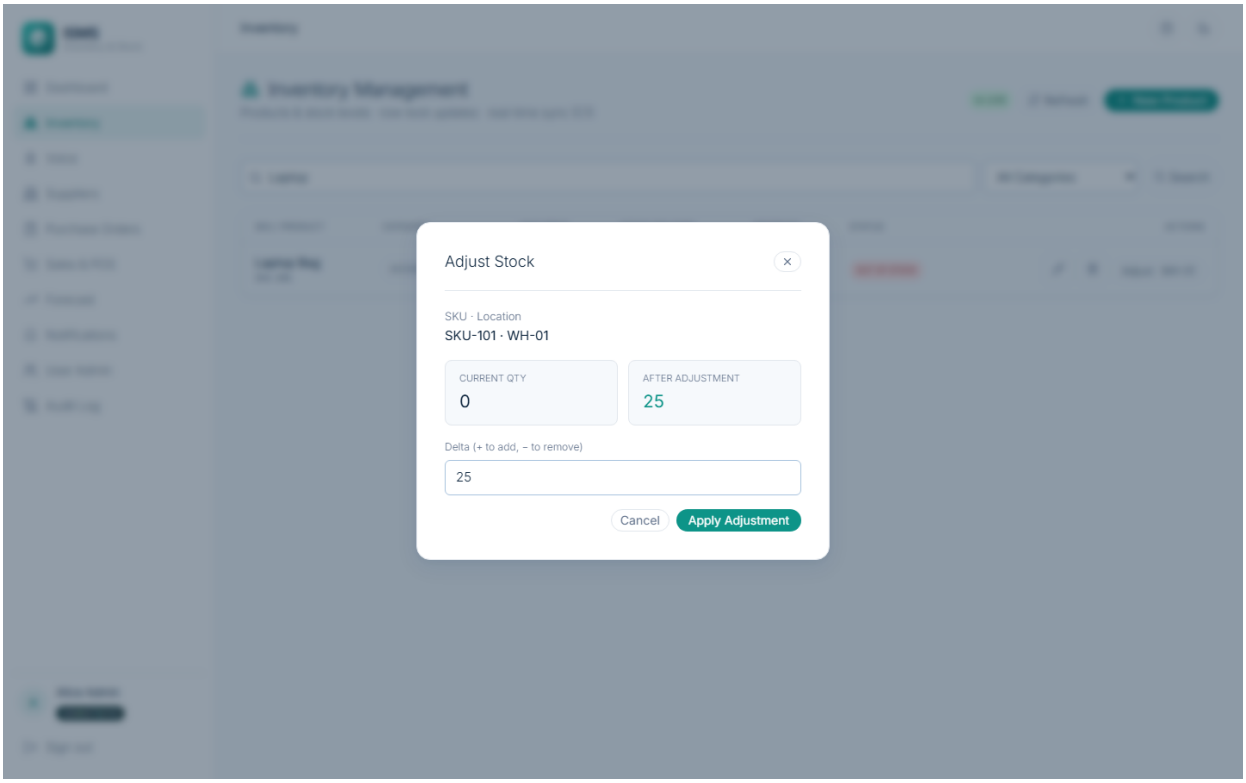


Fig. 38: Adjust stock: the dialog shows the current quantity and the quantity after the adjustment

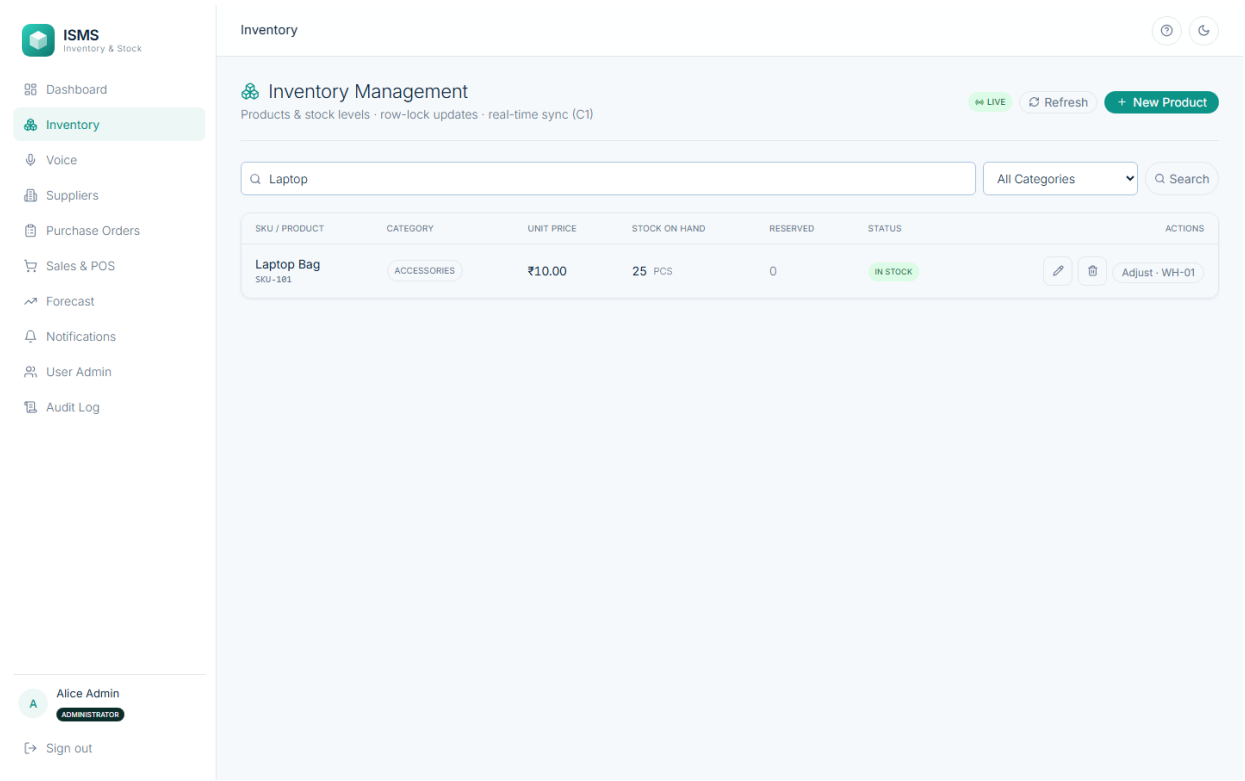


Fig. 39: Stock of SKU-101 is now 25 units (StockRecord.update_stock)

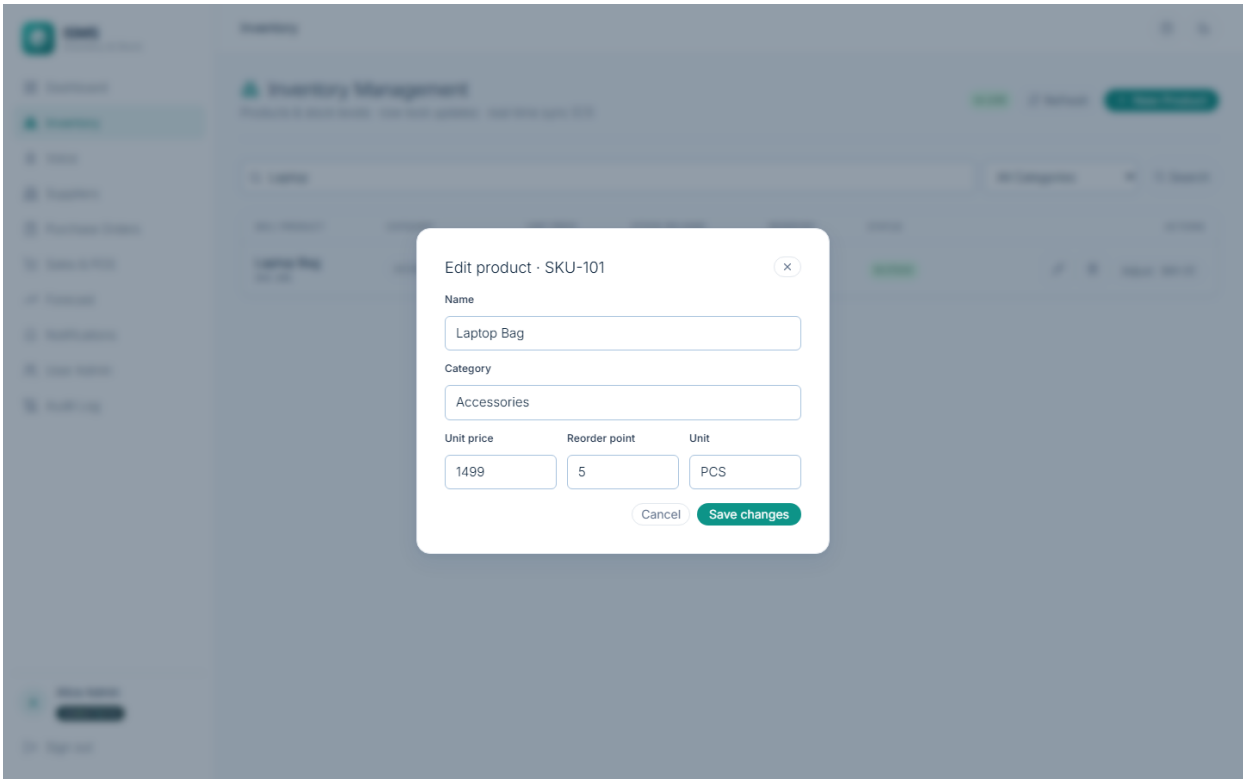


Fig. 40: Edit product: price changed to 1499 (Inventory.CRUD.Update)

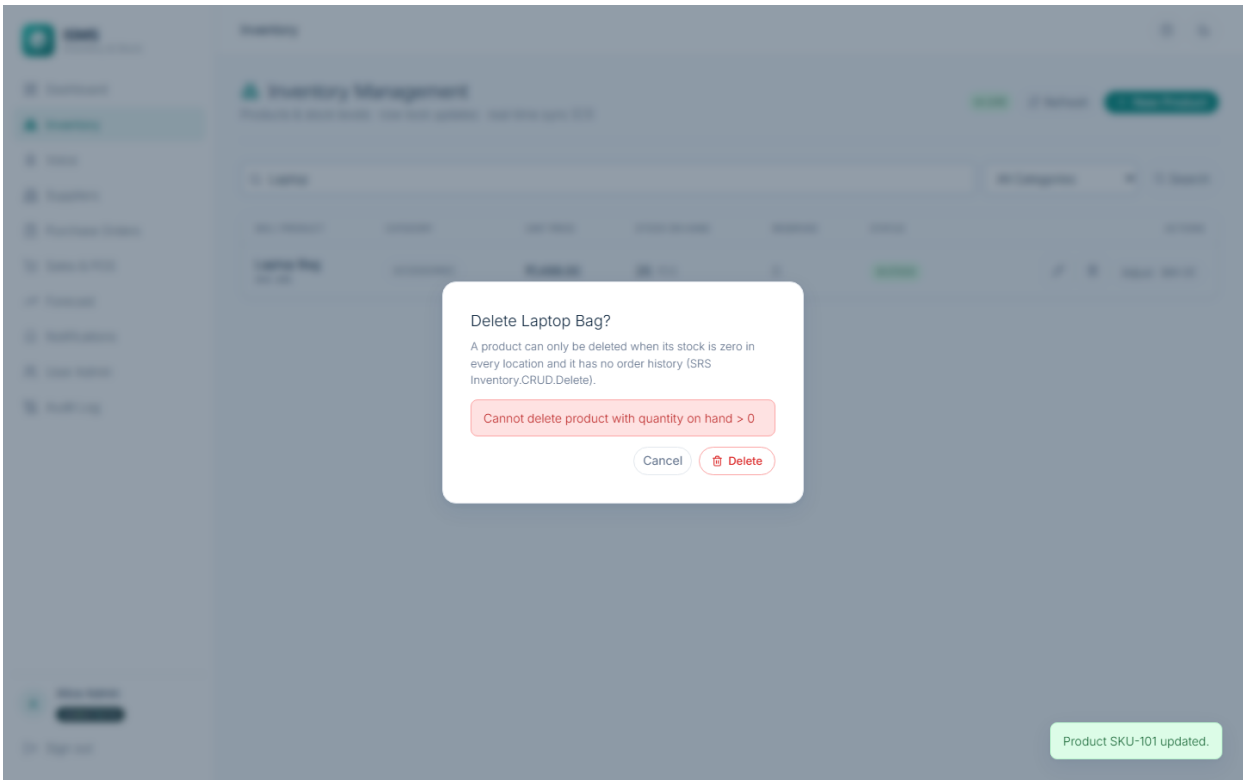


Fig. 41: Delete refused while stock exists: "Cannot delete product with quantity on hand > 0" (Inventory.CRUD.Delete)

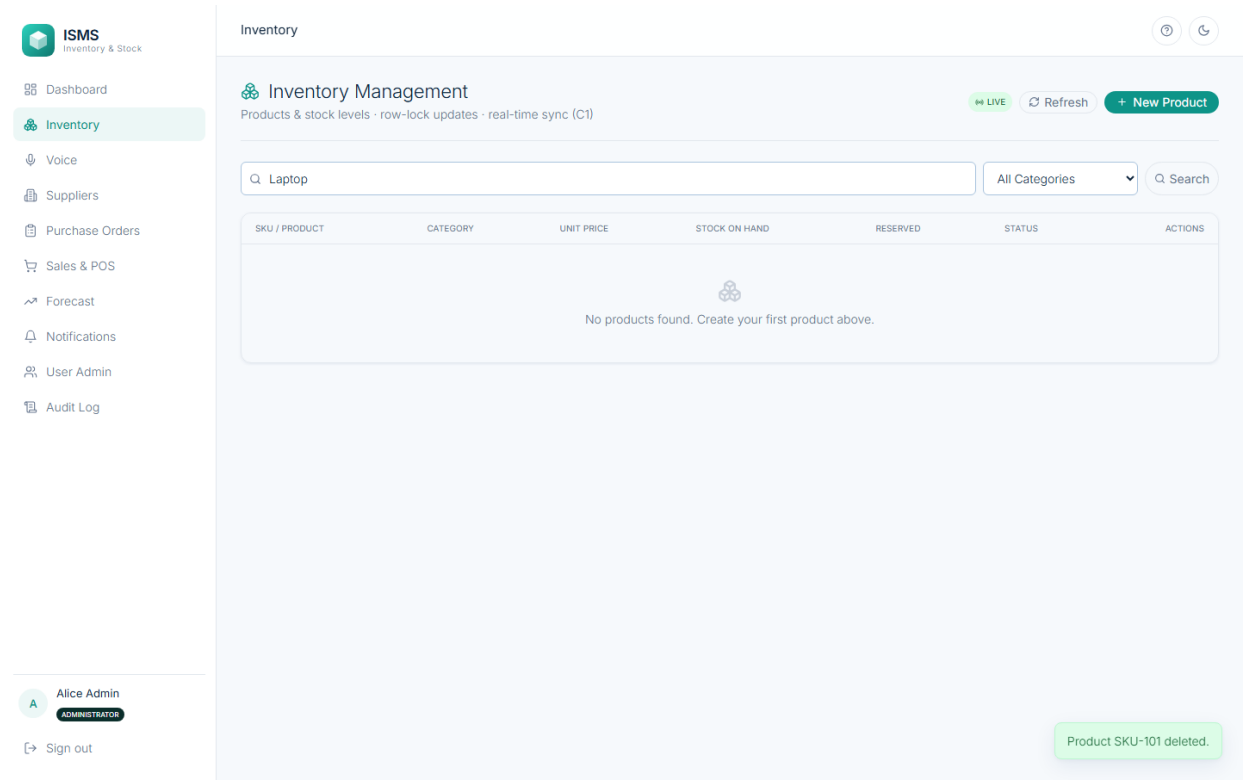


Fig. 42: After the stock is brought to zero, the same product can be deleted

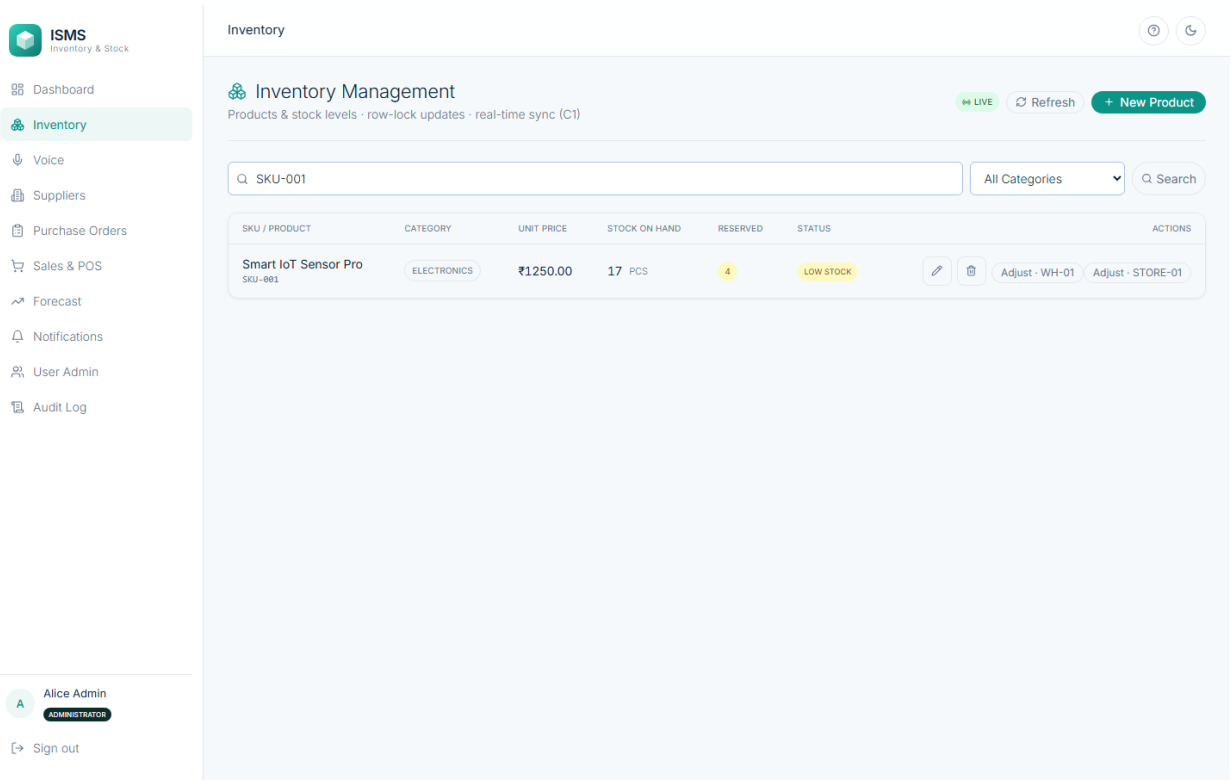


Fig. 43: Real-time sync, before: SKU-001 shows 17 units on one open screen

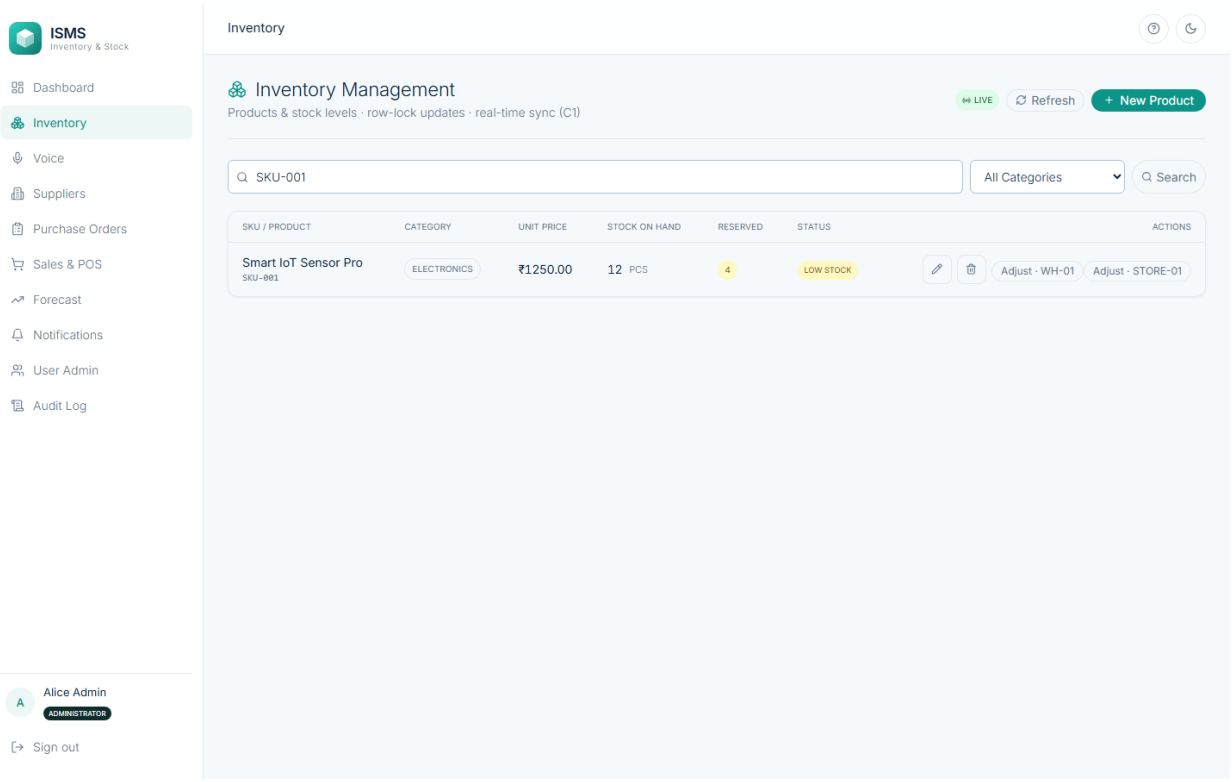


Fig. 44: Real-time sync, after: another user lowered WH-01 to 7 units; this screen now shows 12 (7 + 5) without a reload and the status turned Low Stock

PowerShell - Inventory rules: sync, low-stock alert, concurrency, BR-3

```
$ PUT /api/v1/stock/SKU-001/WH-01 {"quantity_on_hand":7}      (Manager; reorder point is 20)
HTTP 200 in 0.07s -> row updated, pushed to /ws/stock clients, low-stock check run
$ GET /api/v1/notifications                                   (Manager)
  {"type": "Low Stock", "channel": "E-mail", "message": "Low stock alert: SKU SKU-001 at WH-01 has 7 units remaining (reorder point: 20)", "status": "Sen
t"}
  {"type": "Low Stock", "channel": "E-mail", "message": "Low stock alert: SKU SKU-101 at WH-01 has 0 units remaining (reorder point: 5)", "status": "Sent"}

# Sync.Channel.Conflict: two simultaneous sales of the LAST unit (stock = 1)
$ POST /api/v1/sales/physical   (x2, in parallel)
  HTTP 201  order created
  HTTP 409  INSUFFICIENT_STOCK

# BR-3 and validation
$ PUT  /stock/LAST-1/WH-01 {quantity_on_hand:-3}    -> HTTP 400 VALIDATION_FAILED
$ POST /products (duplicate SKU-001)                -> HTTP 409 DUPLICATE_SKU
$ POST /products {"name":"no sku"}                  -> HTTP 400 VALIDATION_FAILED
$ DELETE /products/SKU-002 (stock on hand > 0)       -> HTTP 409 STOCK_EXISTS
$ POST /products as Sales Personnel                  -> HTTP 403 (RBAC)
```

Fig. 45: Business rules in the API: sync time, the low-stock alert, the last-unit race (one 201, one 409), BR-3 and validation errors, guarded delete and RBAC

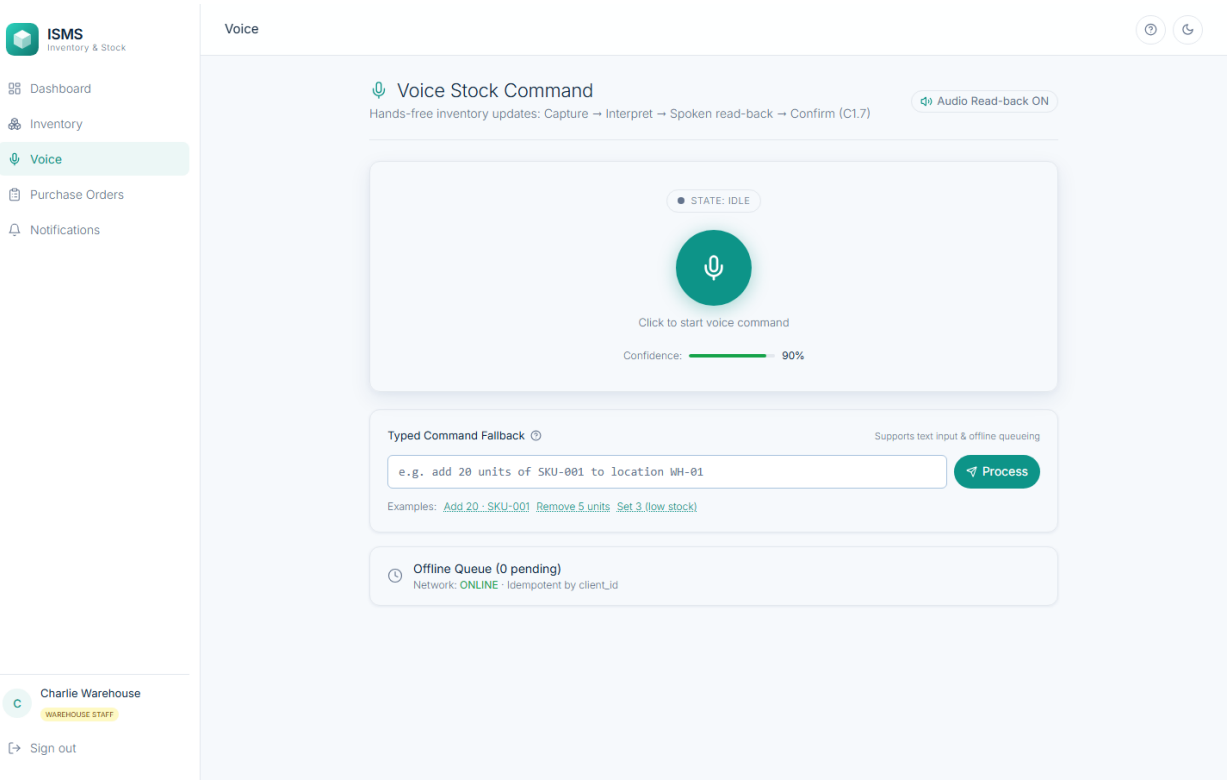


Fig. 46: Voice Stock Command (Warehouse Staff): microphone, state indicator (UI-3), typed fallback and offline queue

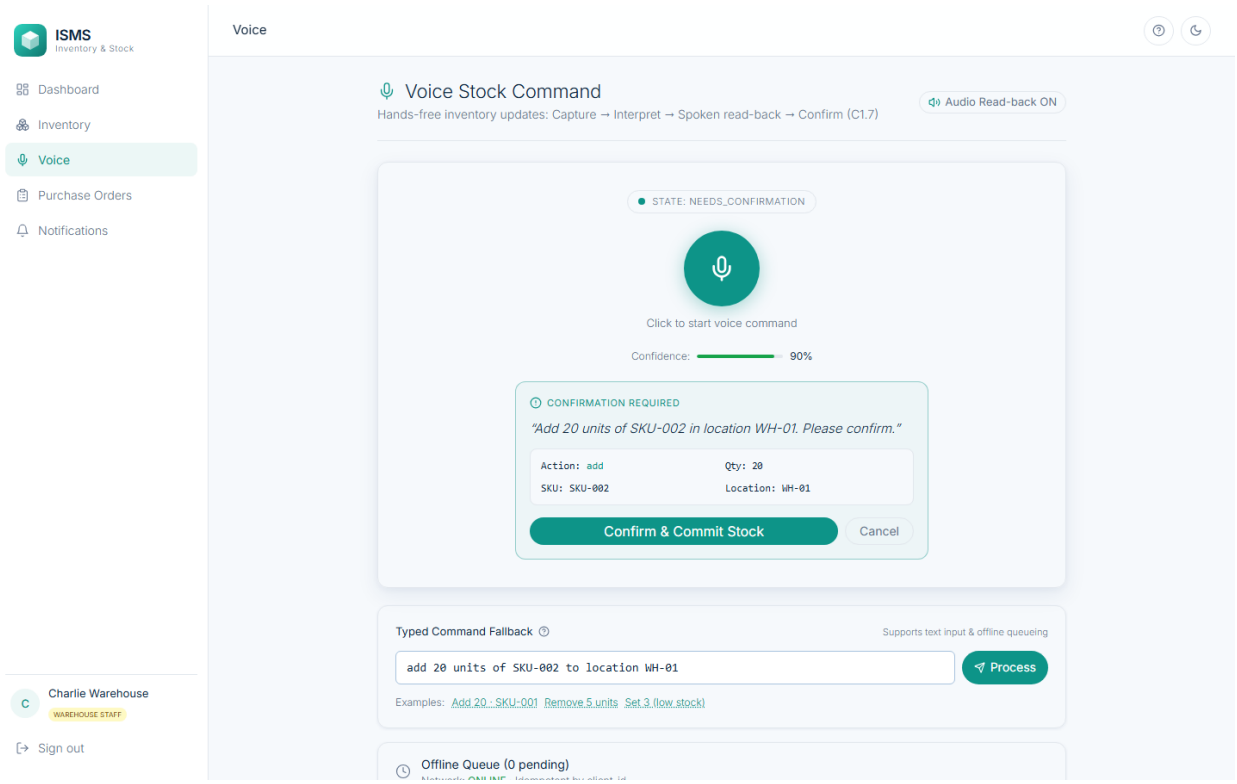


Fig. 47: Interpretation and read-back: "Add 20 units of SKU-002 in location WH-01. Please confirm." with action, quantity, SKU and location

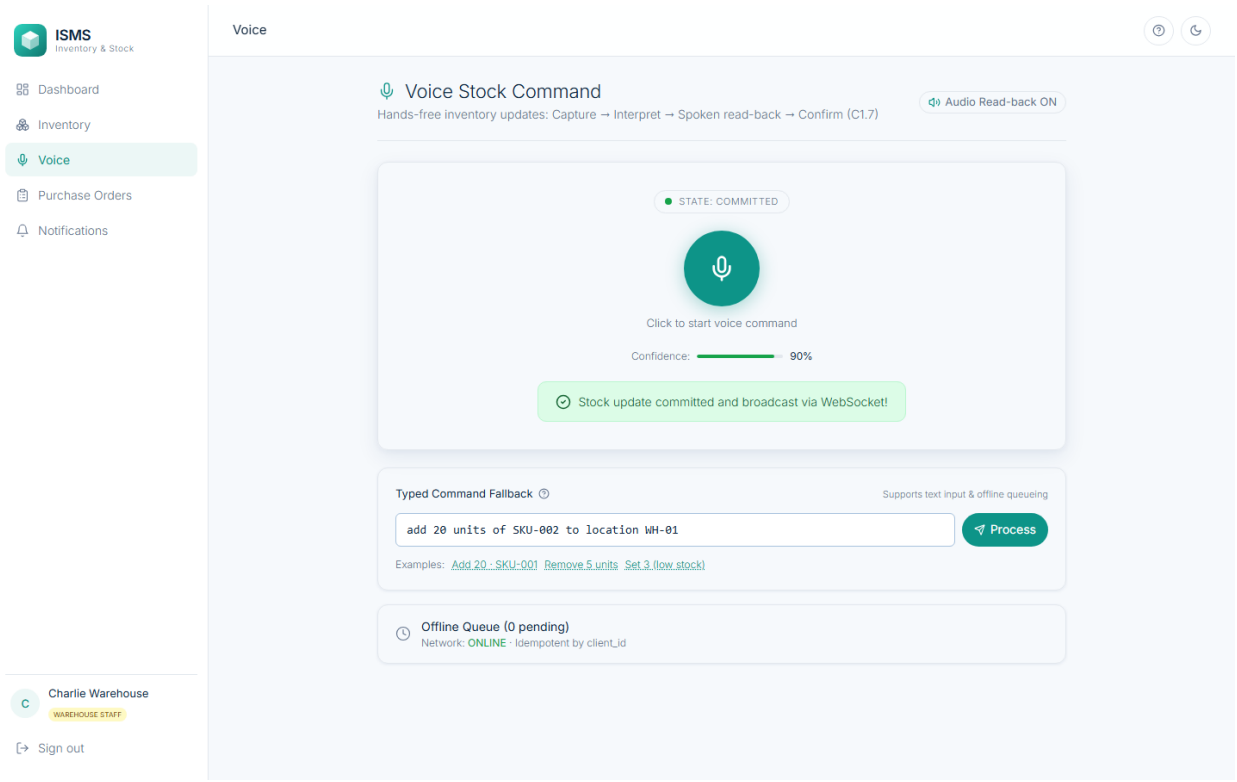


Fig. 48: After confirmation the stock update is committed and broadcast over WebSocket (state: COMMITTED)

5. Supporting Modules Implemented

To run the two core modules inside a complete system, the remaining SRS features were also implemented. They are summarised below with screenshots; their automated tests are listed in the Testing section.

5.1 Module 3: Supplier Management (SRS 4.3, DFD 2.0, D2) — Dev

Managers and Administrators create and edit supplier profiles, search them ranked by rating, and view each supplier's purchase-order history and on-time rate. The rating is recalculated only when a purchase order reaches Received or Cancelled (BR-4), and only an Administrator may deactivate a supplier, and only when no purchase order is open (BR-5).

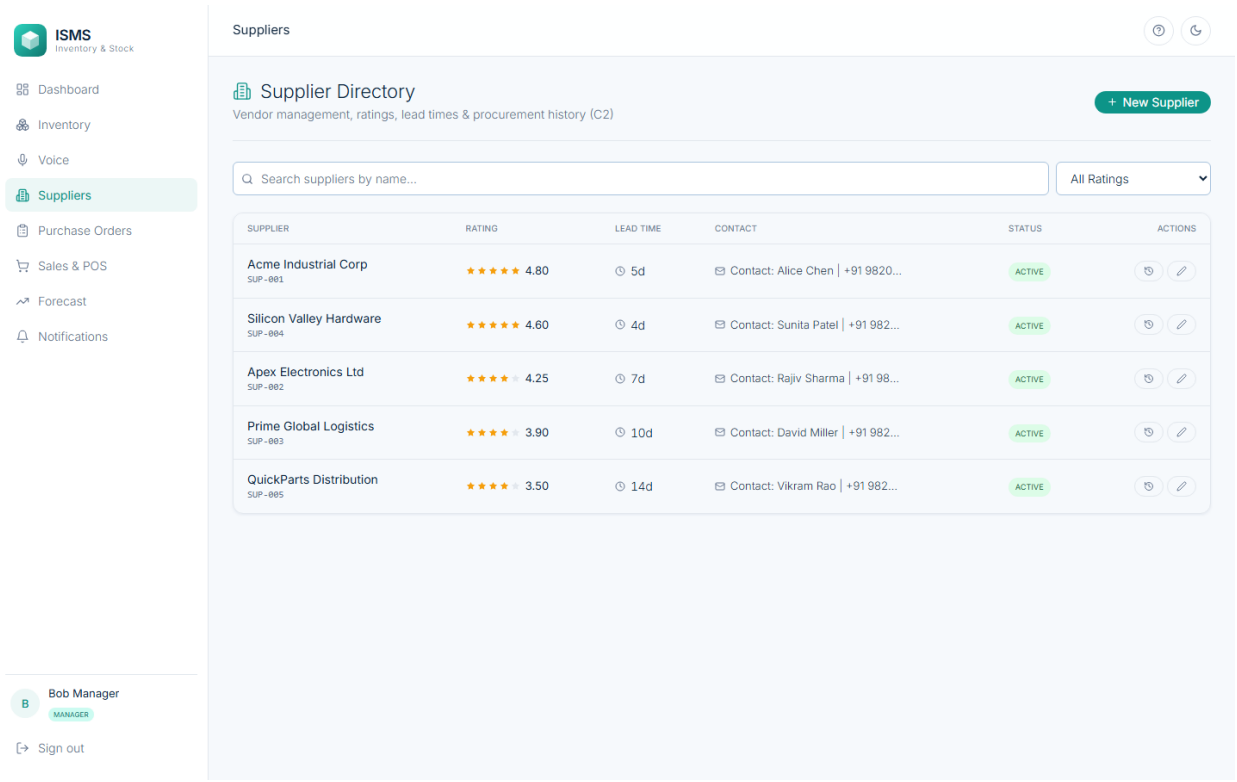


Fig. 49: Suppliers: rating, lead time, contact and status

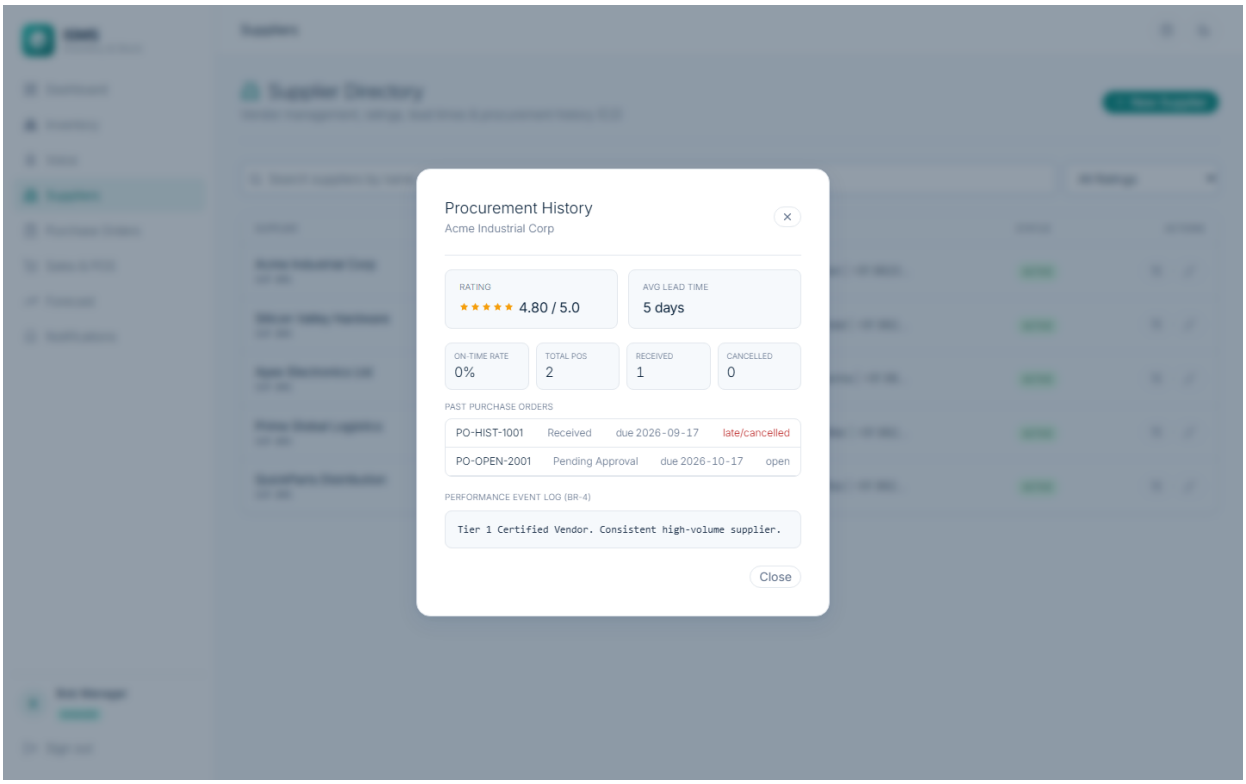


Fig. 50: Supplier performance history: rating, on-time rate and past purchase orders (Supplier.Performance.View)

5.2 Module 4: Purchase Order Management (SRS 4.4, DFD 6.0, D7, SD-2) — Stavan

A purchase order moves through six states: Draft, Pending Approval, Approved, Partially Received, Received and Cancelled. Only a Manager or Administrator may create, approve or reject (BR-1); a Manager's approval is limited by a configurable amount and every PO must keep the supplier within its credit limit (BR-2); a rejection returns the PO to Draft with the reason sent to the creator. Warehouse Staff confirm receipt per line, partial receipts keep the rest open, and each receipt raises stock.

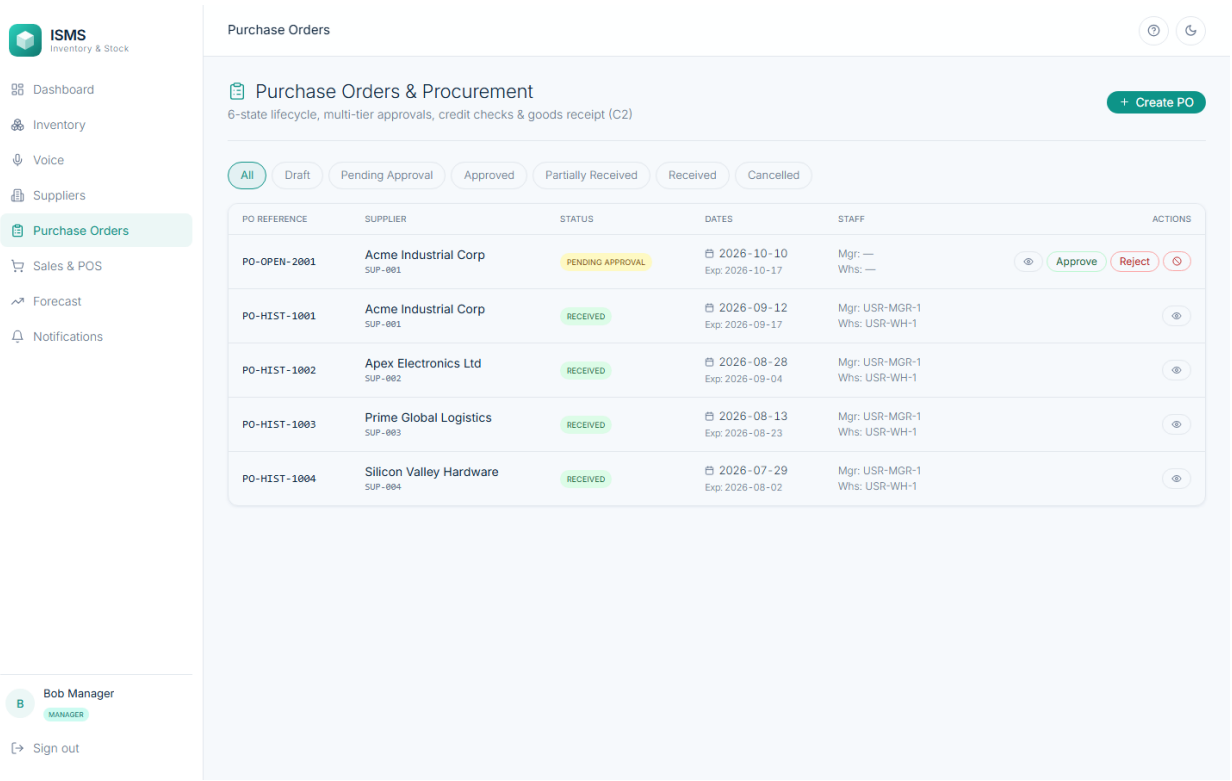


Fig. 51: Purchase orders with status, supplier, dates and the actions allowed for the signed-in role

PowerShell - Purchase order lifecycle, Sequence 2 and business rules BR-1 / BR-2

```
POST /purchase-orders  (2 lines, supplier SUP-001)        HTTP 201  status=Draft
POST /purchase-orders/PO-3EC8F62B/submit                  HTTP 200  status=Pending Approval
POST .../reject  {reason}   (BR-1, back to Draft)         HTTP 200  status=Draft
POST .../submit                                           HTTP 200  status=Pending Approval
POST .../approve                                          HTTP 200  status=Approved
POST .../receive  (partial: 6 of 10 + 10 of 10)           HTTP 200  status=Partially Received
POST .../receive  (remaining 4)                           HTTP 200  status=Received

POST /purchase-orders  (value above the manager limit)    HTTP 201  status=Draft
POST .../submit                                           HTTP 200  status=Pending Approval
POST .../approve  as Manager (BR-2 approval limit)        HTTP 403  APPROVAL_LIMIT_EXCEEDED
POST .../approve  as Administrator                        HTTP 200  status=Approved
POST /purchase-orders as Warehouse Staff (BR-1)           HTTP 403  FORBIDDEN
```

Fig. 52: Sequence SD-2 and BR-1/BR-2 through the API: reject back to Draft, partial then full receipt, manager refused above the limit while the Administrator may approve

5.3 Module 5: Sales Tracking & Order Management (SRS 4.5, DFD 3.0, D5, SD-3) — Dev

Sales Personnel log physical sales at the POS terminal (barcode-wedge input or quick-add). The stock is checked and deducted atomically, a PDF invoice with items, quantities, prices and tax is generated, and returns restore stock. Marketplace orders arrive as webhooks whose HMAC-SHA256 signature is verified; repeated events change nothing. If stock is insufficient the order is rejected, kept as Rejected and the Manager is notified.

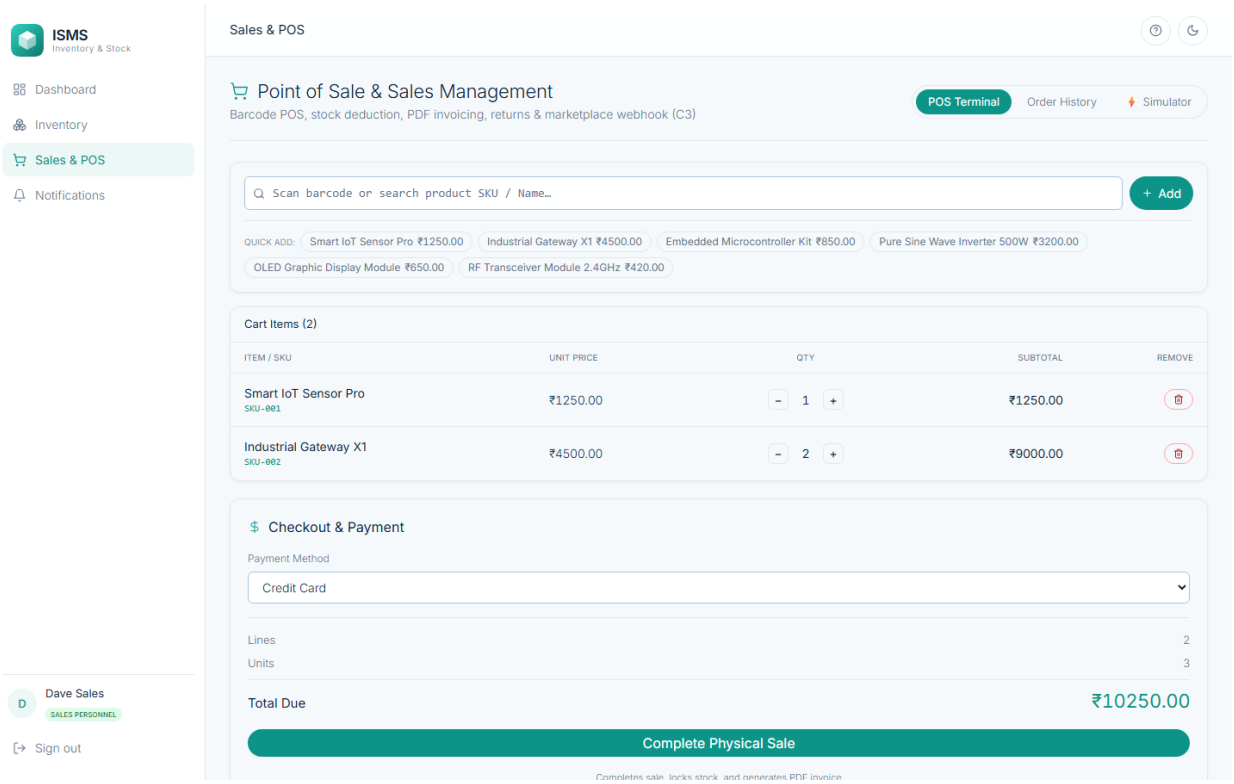


Fig. 53: POS terminal: cart built with quick-add, payment method and total due

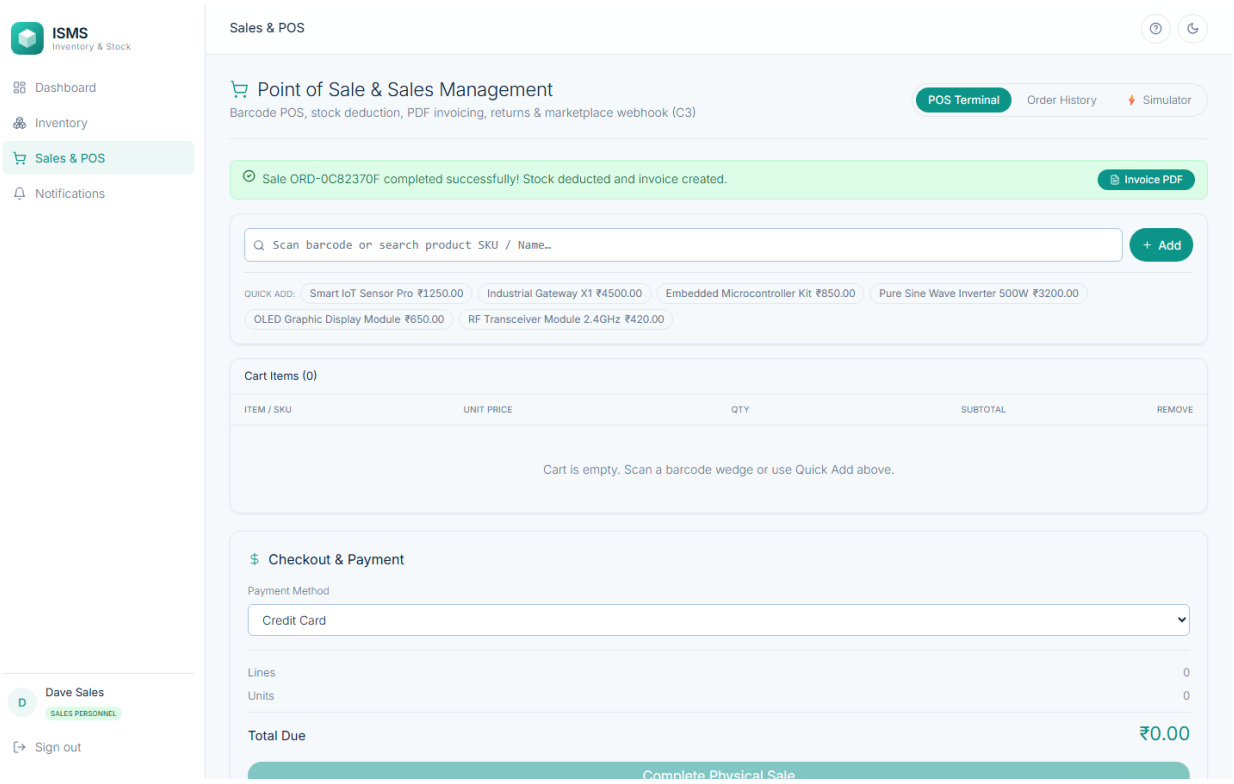


Fig. 54: Sale completed: stock deducted and the invoice is available as a PDF

INVENTORY & STOCK MANAGEMENT SYSTEM (ISMS)

Tax Invoice & Delivery Receipt

Order ID: ORD-0C82370F	Date: 2026-10-12
Channel: physical	**Status: Completed**
Customer: Walk-in Customer	Payment Ref: PAY-AEFABFFF

Item Description	Location	Qty	Unit Price	Total
Smart IoT Sensor Pro (SKU-001)	WH-01	1	1250.00	1250.00
Industrial Gateway X1 (SKU-002)	WH-01	2	4500.00	9000.00

Subtotal: 10250.00
Tax (18%): 1845.00
Grand Total: 12095.00

Fig. 55: The generated PDF invoice (OpenPDF)

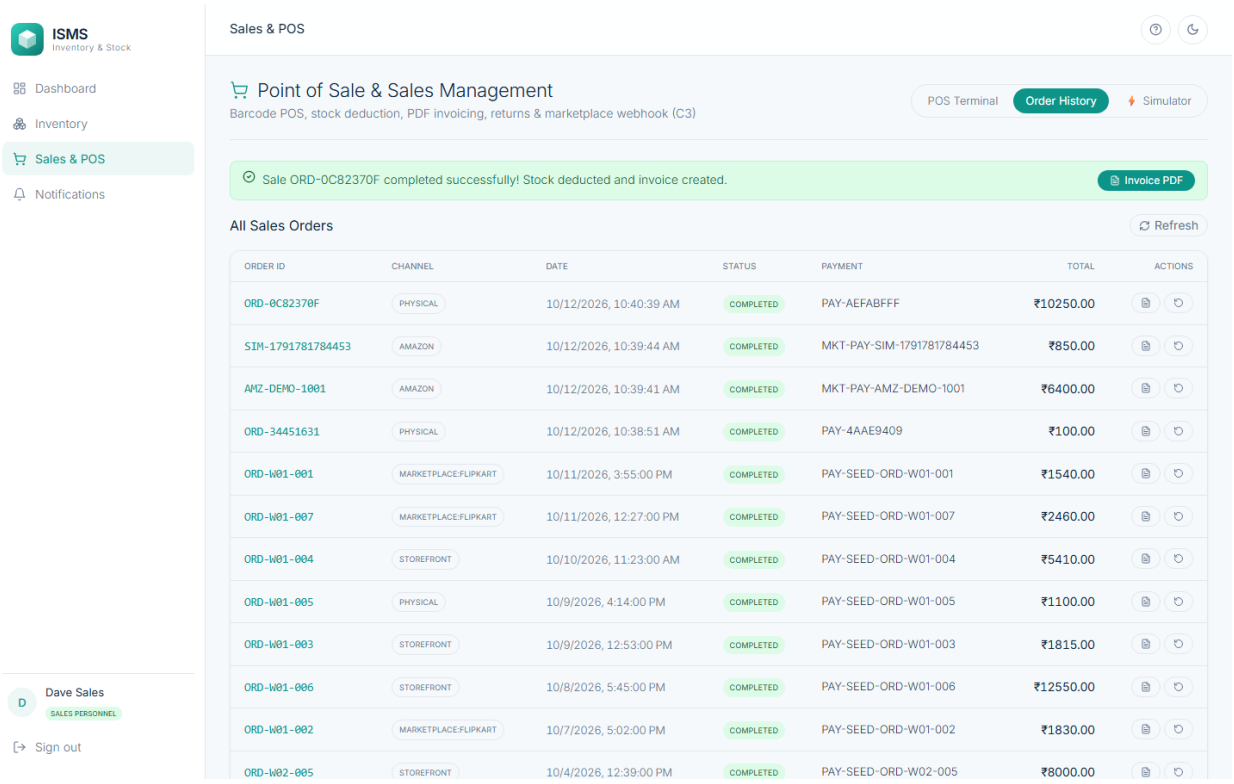


Fig. 56: Order history with channel, status, payment reference and total; invoice and return actions per order

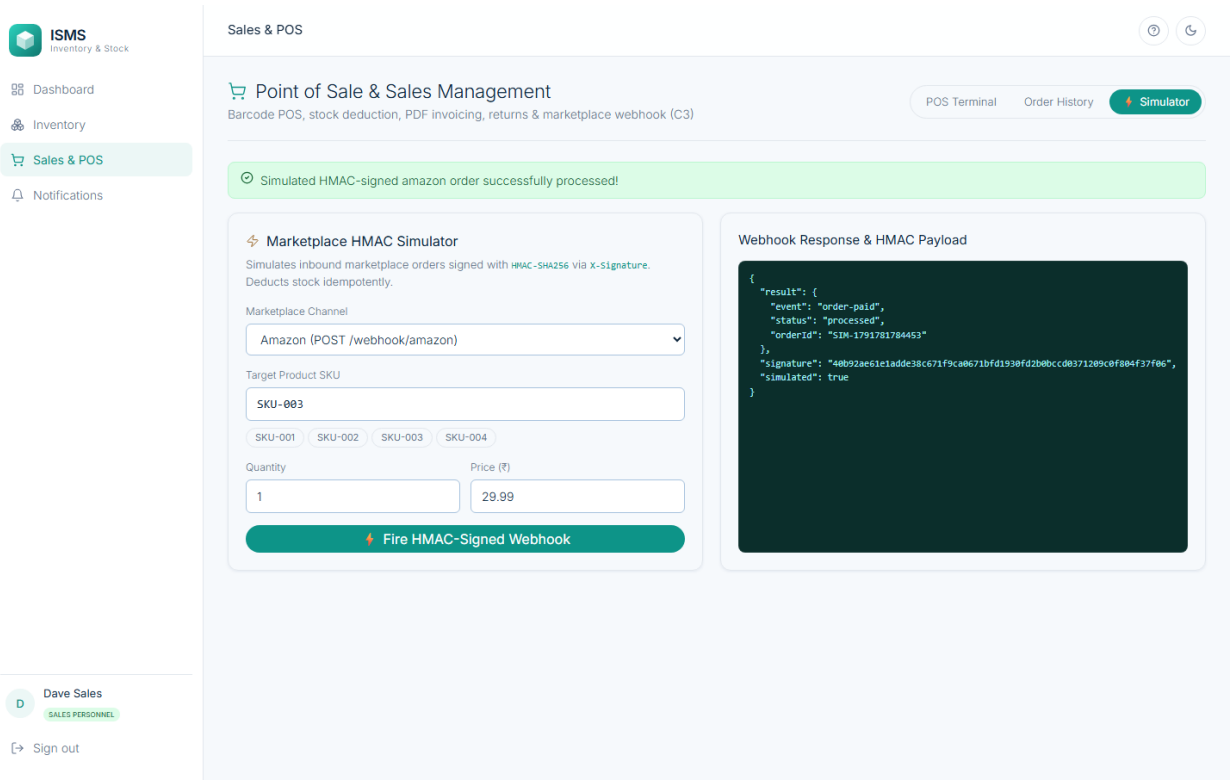


Fig. 57: Marketplace simulator: an HMAC-signed order is posted to the webhook and the response is shown

5.4 Module 6: Predictive Analytics & AI Forecasting (SRS 4.6, DFD 5.0, D6) — Stavan

A Holt-Winters additive model (with a seasonal-naive fallback for short histories) forecasts weekly demand per SKU. The page shows the forecast, recommended reorder quantity (from supplier lead time), one-click draft purchase orders, forecast accuracy (predicted versus actual, fed by every sale) and revenue and stock analytics.

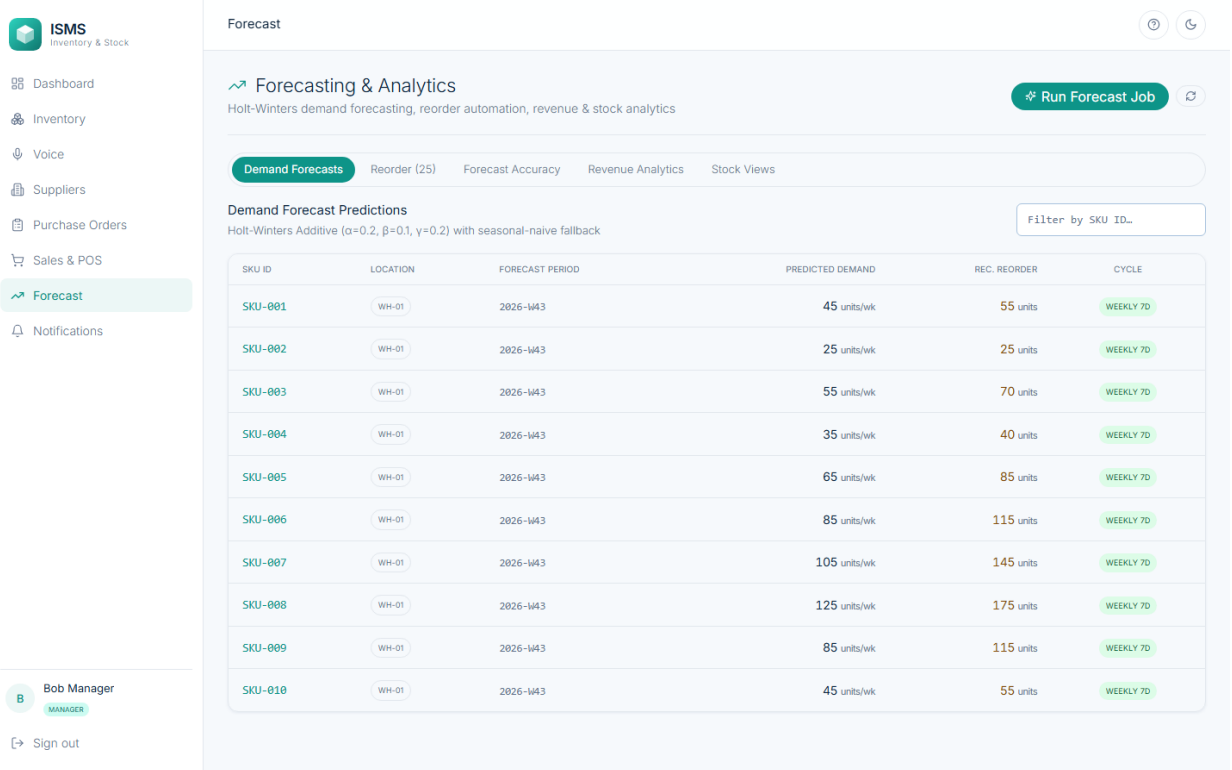


Fig. 58: Demand forecast per SKU with the recommended reorder quantity and the detected cycle

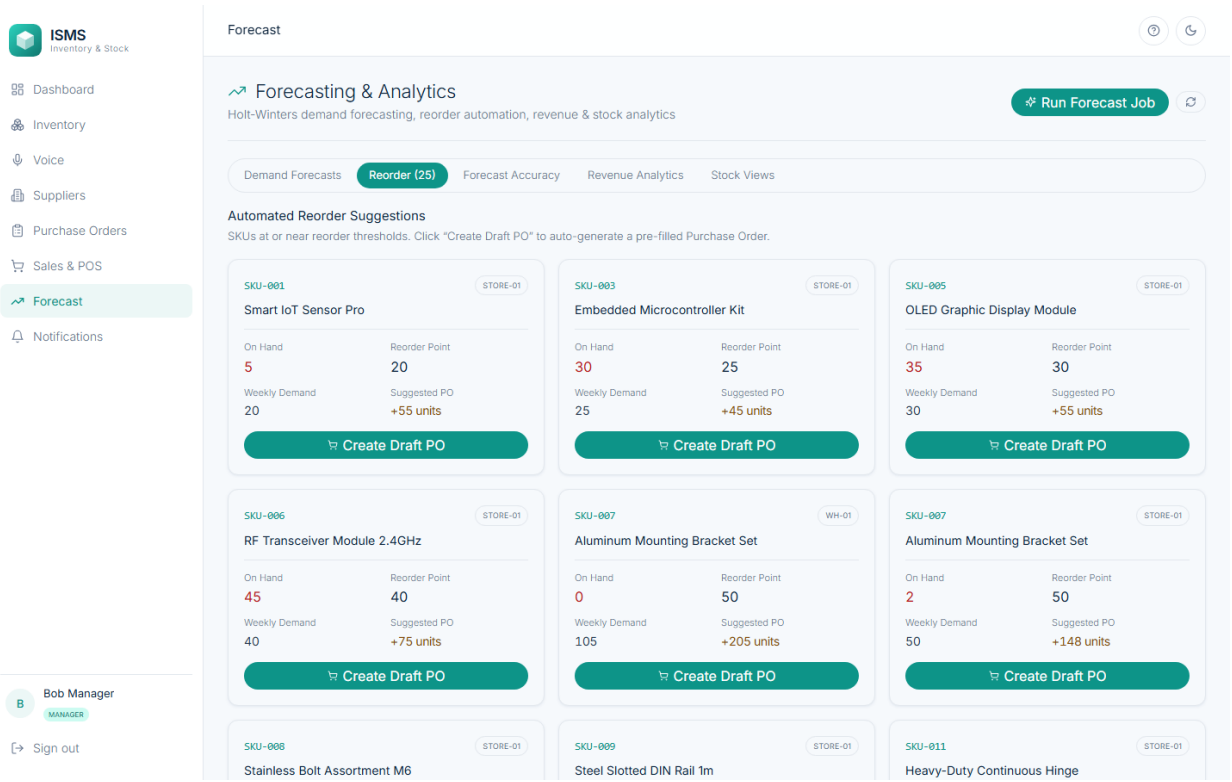


Fig. 59: Reorder suggestions: SKUs near their reorder point with a one-click "Create Draft PO"

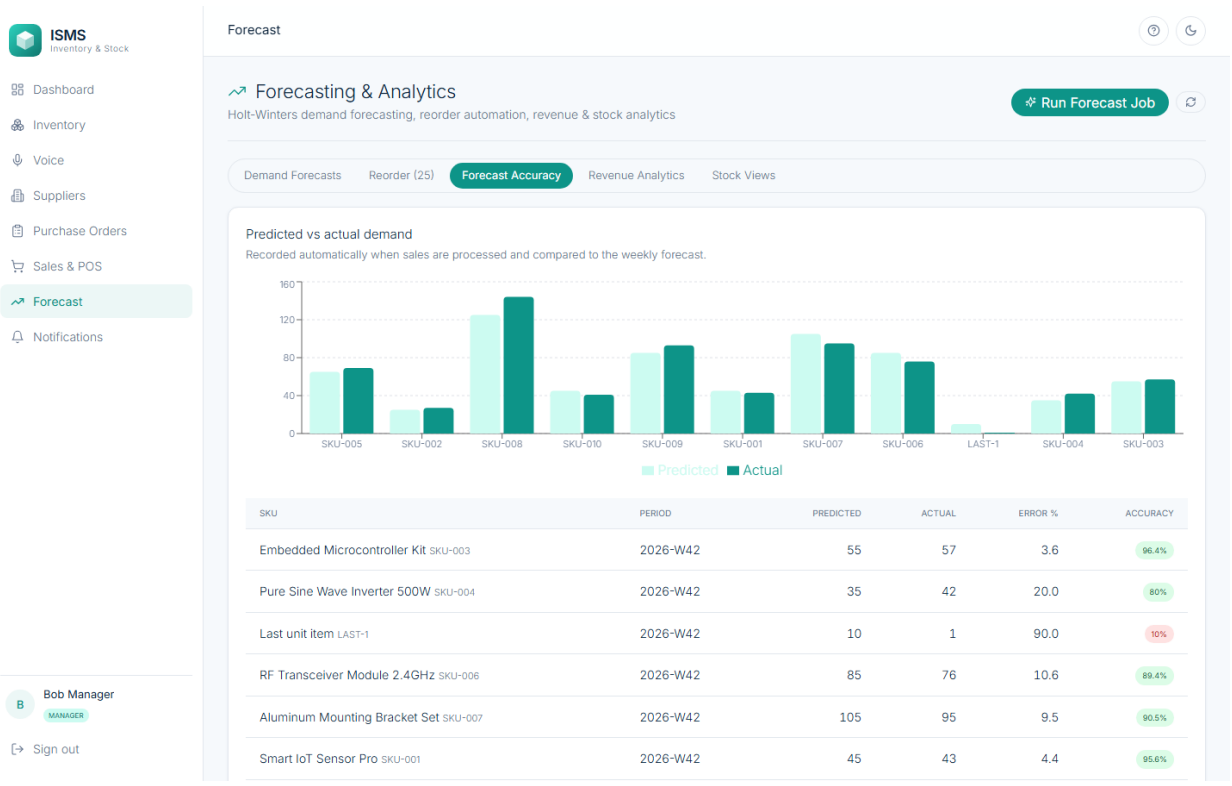


Fig. 60: Forecast accuracy: predicted versus actual demand (Forecast.Accuracy.Track)

5.5 Module 7: Multimodal Notification System (SRS 4.7, DFD 7.0, D8) — Dev

Low-stock and shipping-delay alerts are logged for every recipient and channel (e-mail, SMS, audio) and shown live in the inbox, where they can be marked as read. Each user chooses which alert types reach which channels. The notification engine itself is shared with Module 2 (it is called by every stock change).

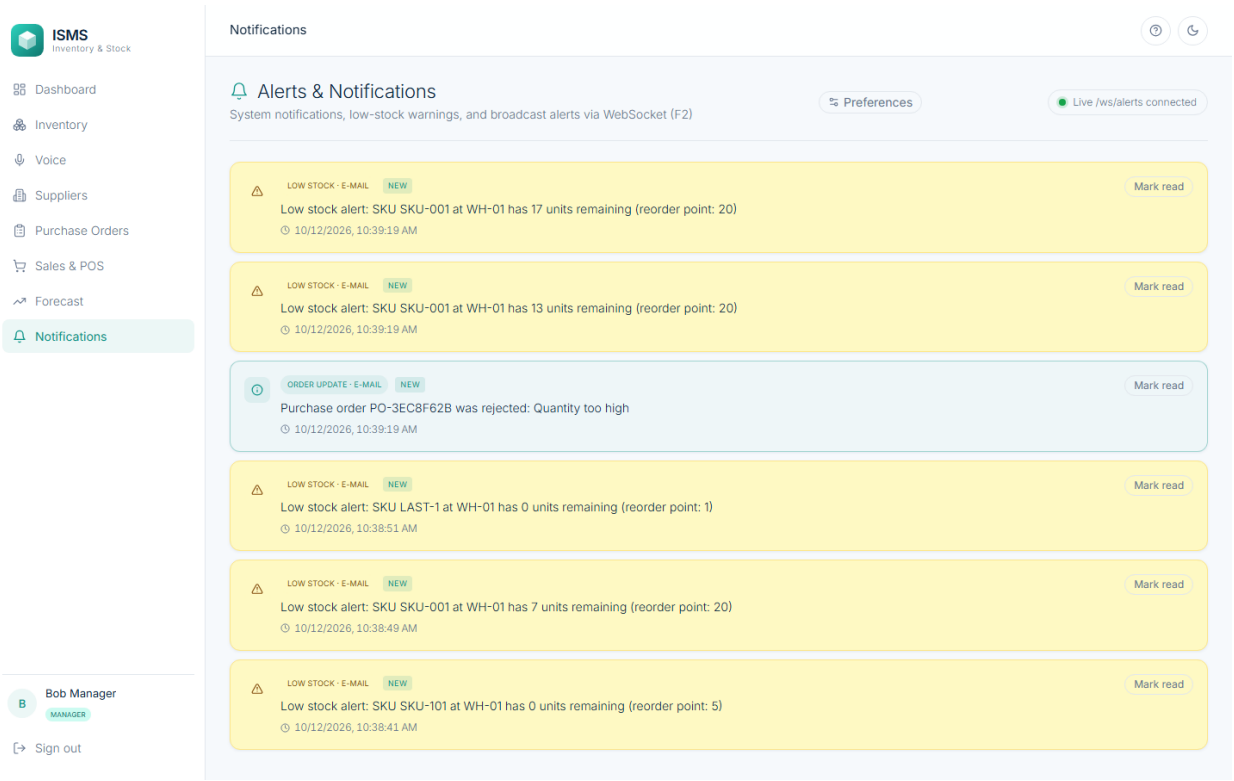


Fig. 61: Notification inbox: low-stock alerts and a PO-rejection notice with NEW badges and "Mark read"; the live /ws/alerts connection is shown

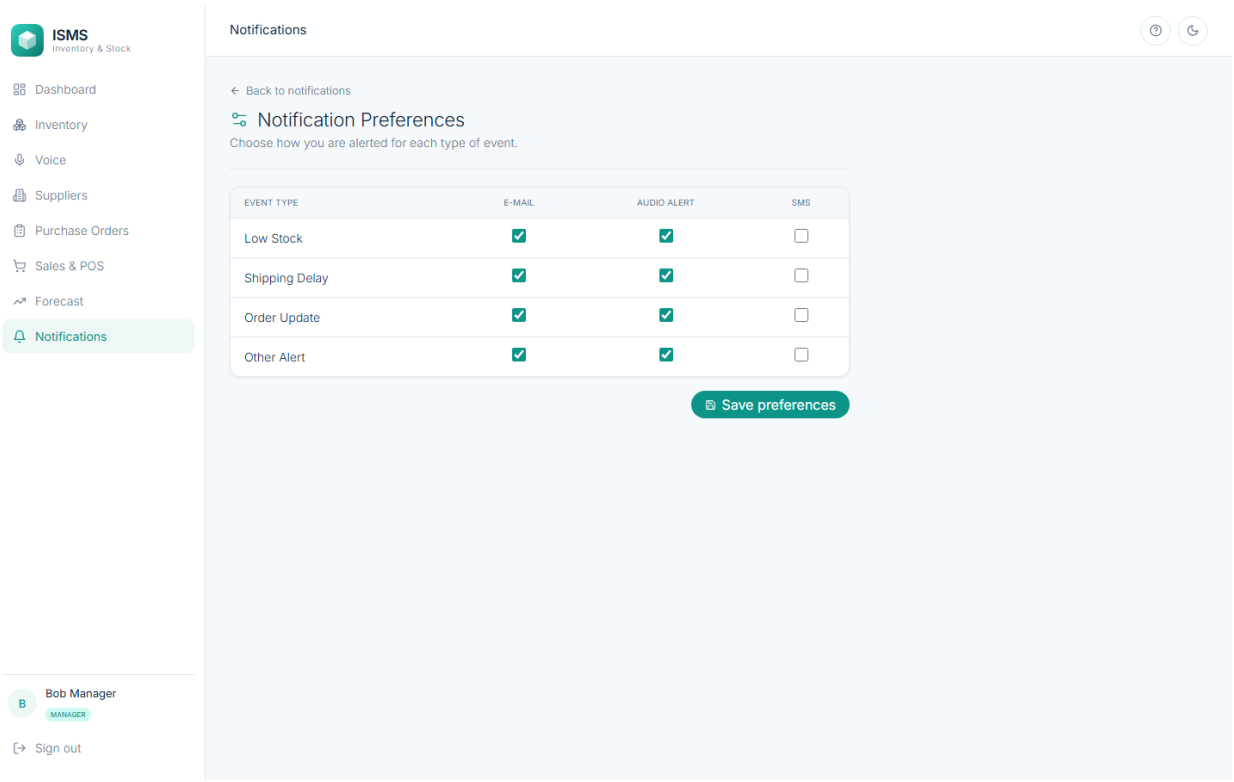


Fig. 62: Notification preferences per alert type and channel (Notify.Preferences.Configure)

5.6 Module 8: Platform Hardening (the rest) — Stavan

The remaining work makes the system demonstrable and verifiable: a demo-data seeder (5 suppliers, 31 SKUs, purchase orders and 443 sales over 52 weeks), non-functional tests (login lockout, session renewal, stock-sync latency, voice throughput), the two class-diagram tests, a

load script, a 53-check end-to-end smoke script, the traceability matrix and the module-wise hand-off plan.

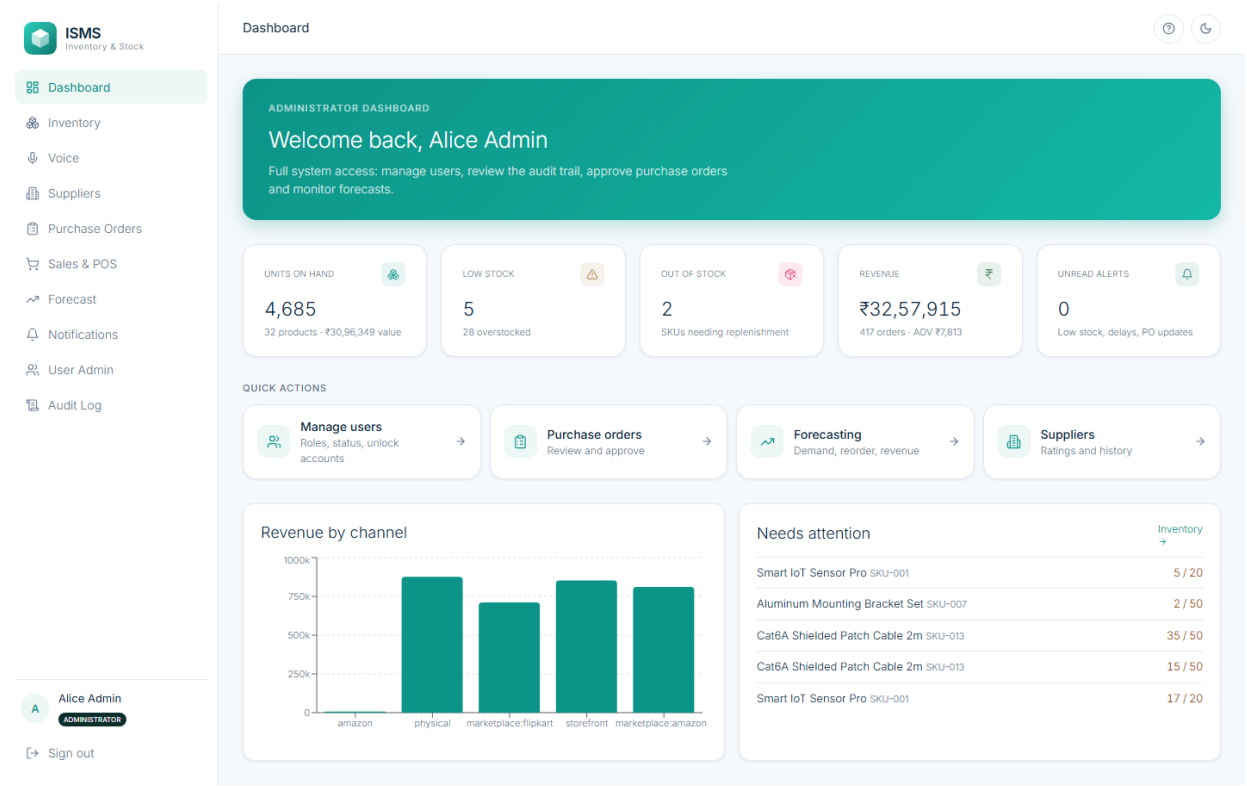


Fig. 63: Dashboard on the seeded demo data: units, low and out-of-stock counts, revenue, revenue by channel and the low-stock list

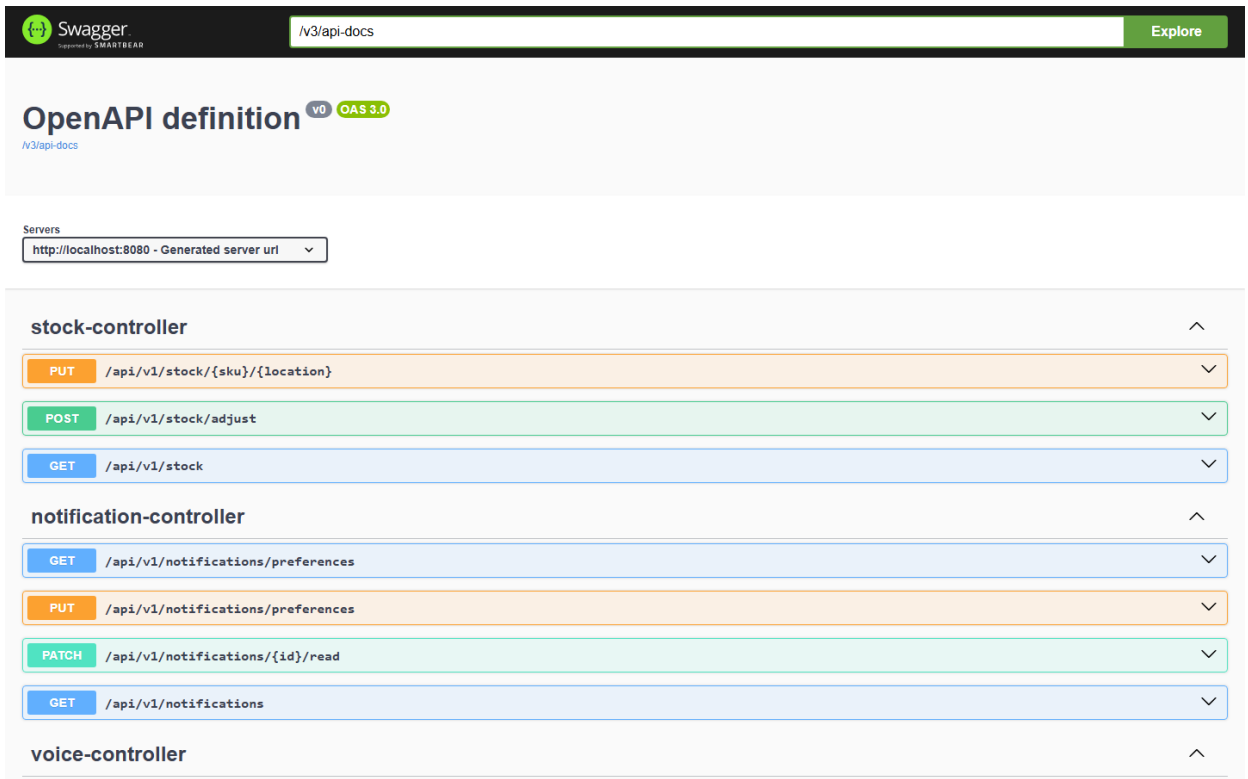


Fig. 64: OpenAPI / Swagger UI listing every REST endpoint of the seven modules

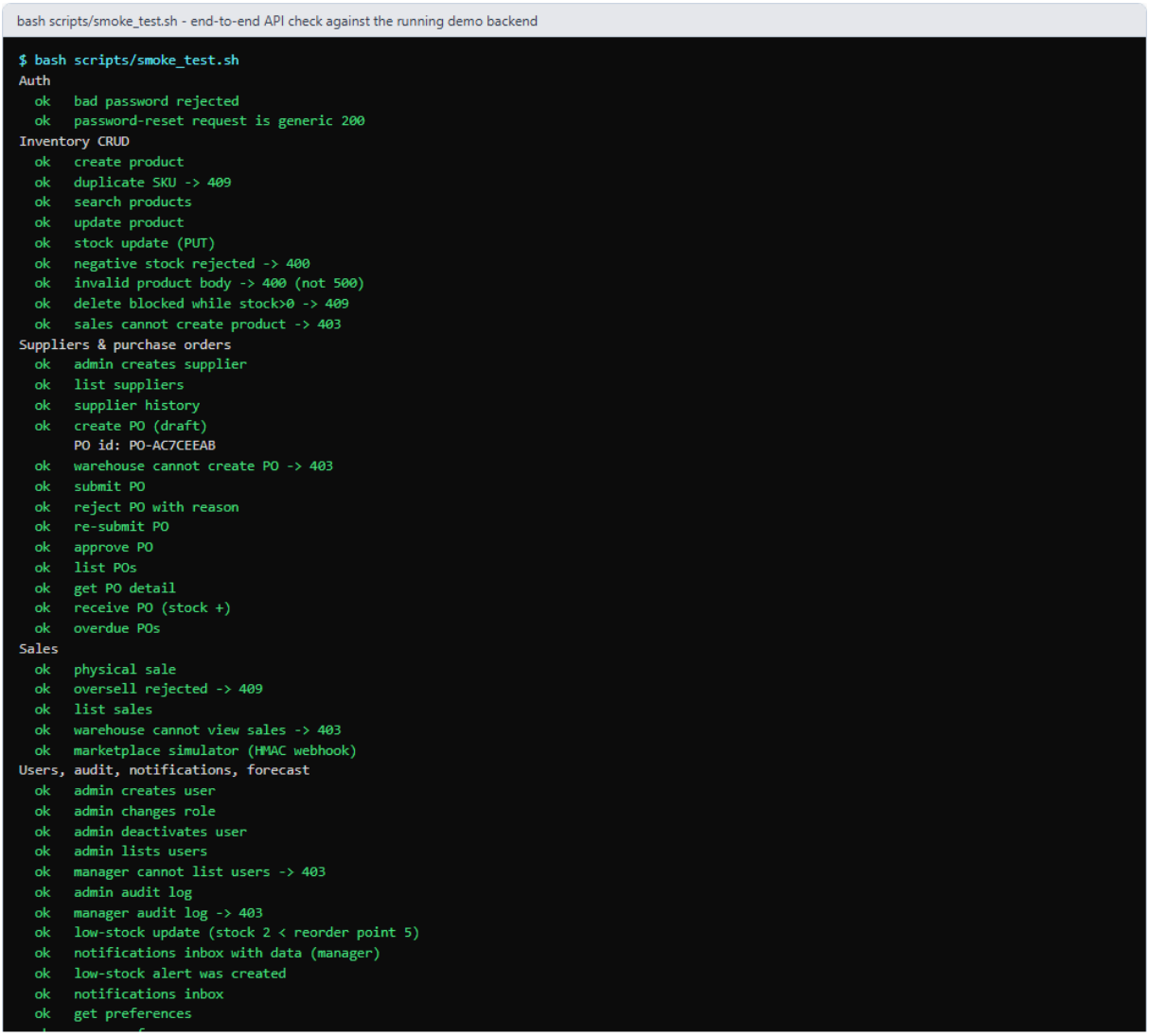


Fig. 65: scripts/smoke_test.sh against the running demo backend: 53 end-to-end checks across all modules and roles, all passing

Testing:

Testing was done at three levels:

- **Automated unit and integration tests:** JUnit 5, Spring Boot Test and MockMvc drive the real REST API against a real PostgreSQL database.
- **Concurrency tests:** simultaneous stock deductions, and six simultaneous sales of the last unit.
- **Manual system testing in the browser and a live API smoke test:** the screenshots above and `scripts/smoke_test.sh`.

All 70 backend tests pass, together with the frontend test (type-check, production build and Vitest) and the 53 live API checks. Of the 70 backend tests, **12 cover Module 1** (AuthIntegrationTest 5, AdminTest 3, RbacMatrixTest 1, BcryptTest 1, HealthControllerTest 1, SchemaMatchesSpecTest 1) and **9 cover Module 2** (StockConcurrencyTest 2, VoiceInterpreterTest 4, Seq1InventoryTest 2, SalesConcurrencyTest 1); the 4 non-functional tests (lockout, session renewal, stock-sync latency, voice throughput) cover both. The remaining 45 cover Modules 3–8 and the class diagram.

Test Cases: Module 1 (Authentication & Role-Based Access Control)

ID	Test case	Input / steps	Expected result	Actual result	Status
TC-1.1	Sign in with each role	USR-ADMIN-1, USR-MGR-1, USR-WH-1, USR-SALES-1, password Password123!	200, token and role; role-specific menu	All four roles signed in; menus differ by role (Fig. 13, 25)	Pass
TC-1.2	Wrong password	USR-SALES-1 with an invalid password	401, generic message; LOGIN_FAILED audited	“Invalid user ID/email or password” (Fig. 11, 24)	Pass
TC-1.3	Lockout boundary (4th vs 5th failure)	5 wrong passwords for one user in a row	Attempts 1–4: 401. Attempt 5: 403 ACCOUNT_LOCKED	4 × 401 then 403 (Fig. 20; automated)	Pass
TC-1.4	Locked account, correct password	Correct password after the lock	403 ACCOUNT_LOCKED	Refused (Fig. 21)	Pass
TC-1.5	Lock is persisted and visible	Open User Admin after the lock	Status Locked in the directory; unlock action available	Locked pill and highlighted unlock button (Fig. 22; automated: testLockIsPersistedAndAdminCanUnlock)	Pass
TC-1.6	Administrator unlocks	Click unlock for the locked user	Status active; correct password works again	Active again, login returns 200 (Fig. 23)	Pass
TC-1.7	Reset request, unknown vs known e-mail	POST password-reset/request for both	Same generic 200; an e-mail only for the known address	Identical response; one e-mail with a 30-minute link (Fig. 28, 29)	Pass
TC-1.8	Reset with a valid token	New password via the e-mailed link	Password changed; login with the new password succeeds	Password changed, login 200 (Fig. 30, 31)	Pass
TC-1.9	Reset token reused	Submit the same token again	400 TOKEN_ALREADY_USED	400 returned (negative test)	Pass
TC-1.10	No token / forged token	GET /users without a token; with “not-a-jwt”	401 both	401 and 401 (Fig. 27; automated)	Pass
TC-1.11	Wrong role on the API	Sales token on GET /users and /admin/audit-log	403	403 and 403 (Fig. 27; RbacMatrixTest)	Pass
TC-1.12	Wrong role in the UI	Sales user opens /users in the browser	“Access Restricted” page, no data	Shown (Fig. 26)	Pass
TC-1.13	Sliding session	Any authenticated request	Response carries a refreshed token	X-Refreshed-Token present (Fig. 27; testSlidingTokenHeader, NfrTest)	Pass

ID	Test case	Input / steps	Expected result	Actual result	Status
TC-1.14	Create user, inline validation	Submit the empty form; then a valid form	Field messages and no request; then the user is created and audited	Messages shown, then user created (Fig. 15, 16)	Pass
TC-1.15	Assign a role	Change Demo Staff to Sales Personnel	Role updated; ROLE_ASSIGNED audited	Updated (Fig. 17, 18, 24; AdminTest.testAssignRole)	Pass
TC-1.16	Audit log access	Open the audit log as Administrator, then as another role	Events listed for the Administrator only (403 otherwise)	All events listed (Fig. 24)	Pass
TC-1.17	System settings are read-only	Open the Settings tab	Limits shown, nothing editable	Eight values shown (Fig. 19)	Pass
TC-1.18	Database schema matches the design	Compare the live schema with schema.md	No difference	SchemaMatchesSpecTest passes	Pass

Test Cases: Module 2 (Voice-Enabled Inventory & Omnichannel Sync)

ID	Test case	Input / steps	Expected result	Actual result	Status
TC-2.1	List and search products	Open Inventory; search “Sensor”	All products with stock; filter returns the match	31 products listed; one match (Fig. 33, 34)	Pass
TC-2.2	Create a product	SKU-101 “Laptop Bag”, price 10, reorder point 5	201; initial stock record of 0	Created, stock 0 (Fig. 35, 36)	Pass
TC-2.3	Duplicate SKU	Create SKU-101 again	409 DUPLICATE_SKU	Refused (Fig. 37, 45)	Pass
TC-2.4	Missing required field	POST /products with only a name	400 VALIDATION_FAILED (not 500)	400 returned (Fig. 45)	Pass
TC-2.5	Adjust stock	Set SKU-101 at WH-01 to 25	Stock 25; audited	25 units (Fig. 38, 39)	Pass
TC-2.6	Edit a product	Change the unit price to 1499	Price updated	Updated (Fig. 40)	Pass
TC-2.7	Delete with stock (boundary: stock > 0)	Delete SKU-101 while it holds 25 units	409 STOCK_EXISTS	Refused with the message (Fig. 41)	Pass
TC-2.8	Delete at zero stock (boundary: stock = 0)	Set stock to 0, then delete	Deleted	Deleted (Fig. 42)	Pass
TC-2.9	Negative quantity (BR-3)	PUT stock = −3	400, stock unchanged	400 returned (Fig. 45; DB CHECK as second guard)	Pass
TC-2.10	Real-time sync within 5 s (PE-2)	Keep the page open; another user changes WH-01 to 7	The open screen updates without reload	17 → 12 shown live; API call took 0.06 s (Fig. 43, 44; NfrTest.testStockSyncLatency)	Pass
TC-2.11	Low-stock alert branch of SD-1	Stock below the reorder point; then above	Alert + audio alert; “no alert” when above	Alert created (Fig. 45; testSequence1WithLowStockAlert, testSequence1NoAlertBranch)	Pass
TC-2.12	Last-unit conflict	Two simultaneous sales of the last unit	Exactly one succeeds, the other 409	201 + 409 (Fig. 45; testLastUnitConflict)	Pass
TC-2.13	Stress: six simultaneous sales of one unit	6 threads, stock 1	1 order, stock ends at 0	1 order, stock 0 (SalesConcurrencyTest)	Pass
TC-2.14	Concurrent deductions never negative	Many parallel deductions	Stock never below 0	As expected (StockConcurrencyTest)	Pass
TC-2.15	Voice command, numeric	“add 20 units of SKU-002 to location WH-01”	action add, qty 20, SKU-002, WH-01; read-back text	Interpreted and read back (Fig. 47; testStandardCommandNumeric)	Pass
TC-2.16	Voice command, spoken number	“add twenty units …”	Quantity 20	20 (testWordNumbers)	Pass
TC-2.17	Confirm a command	Press Confirm & Commit Stock	Stock raised by 20; broadcast; state COMMITTED	Committed (Fig. 48)	Pass

ID	Test case	Input / steps	Expected result	Actual result	Status
TC-2.18	Low confidence (boundary 0.75)	Confidence below the threshold	status low_confidence: repeat or use the form	As expected (testLowConfidenceThreshold) ; at or above → needs_confirmation	Pass
TC-2.19	100 concurrent voice commands (PE-1)	100 parallel interpretations	Done in under 3.0 s	Passed (NfrTest.testConcurrentVoiceInterpretationThroughput)	Pass
TC-2.20	Role restriction	Sales Personnel posts a new product	403	403 (Fig. 45)	Pass

Test Cases: Modules 3–8 (summary)

Module	Main behaviours verified	Automated tests	Result
3 Supplier	Create/search, rating only after Received/Cancelled (BR-4), deactivation blocked with open POs (BR-5), history	Seq2PurchaseOrderTest (deactivate), smoke test	Pass
4 Purchase Orders	6-state lifecycle, illegal transitions rejected, approval limit, credit limit, reject to Draft, partial receipt, overdue alert	PoStateMachineTest 3, Seq2PurchaseOrderTest 7	Pass
5 Sales	Physical sale, invoice PDF, return, webhook (valid, missing, invalid, replayed), insufficient stock keeps a REJECTED order and notifies the Manager	Seq3SalesTest 4, WebhookSecurityTest 4, SalesConcurrencyTest 1	Pass
6 Forecasting	Forecast pipeline call order, reorder suggestions and PO payload, stock analytics	ForecastTest 3	Pass
7 Notifications	Low-stock and shipping-delay triggers, inbox with data, preferences	Seq1/Seq2 tests, smoke test	Pass
8 Platform	Class diagram (12 classes, 43 operations, 9 relationships), NFRs, schema	ClassContractTest 12, ClassDiagramStructureTest 12, NfrTest 4	Pass

Automated Test Execution

Module 1 - Authentication & RBAC: automated tests (JUnit 5 + MockMvc + PostgreSQL)

```
$ ./mvnw test -Dtest=AuthIntegrationTest,RbacMatrixTest,AdminTest,BcryptTest,HealthControllerTest,SchemaMatchesSpecTest

com.isms.AuthIntegrationTest   (5 tests, 3.12s)
    ok    testAccountLockoutAfterFiveFailedAttempts
    ok    testSlidingTokenHeader
    ok    testPasswordResetFlow
    ok    testLoginAllFourRoles
    ok    testLockIsPersistedAndAdminCanUnlock

com.isms.RbacMatrixTest   (1 tests, 0.25s)
    ok    testUserAdminAccessMatrix

com.isms.AdminTest   (3 tests, 9.36s)
    ok    testUserCrudAndUnlock
    ok    testListUsers
    ok    testAssignRole

com.isms.BcryptTest   (1 tests, 0.11s)
    ok    generateAndVerifyHash

com.isms.HealthControllerTest   (1 tests, 0.00s)
    ok    healthReturnsUp

com.isms.SchemaMatchesSpecTest   (1 tests, 0.04s)
    ok    testSchemaMatchesSpec

Tests run: 12, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Fig. 66: Module 1 automated tests: 12 / 12 passed (each test method listed)

Module 2 - Voice-Enabled Inventory & Omnichannel Sync: automated tests

```
$ ./mvnw test -Dtest=StockConcurrencyTest,VoiceInterpreterTest,Seq1InventoryTest,SalesConcurrencyTest,NfrTest

com.isms.StockConcurrencyTest   (2 tests, 0.36s)
    ok    testConcurrentStockDeductionsNeverNegative
    ok    testLastUnitConflict

com.isms.VoiceInterpreterTest   (4 tests, 0.01s)
    ok    testWordNumbers
    ok    testHighConfidenceConfirmation
    ok    testLowConfidenceThreshold
    ok    testStandardCommandNumeric

com.isms.Seq1InventoryTest   (2 tests, 0.17s)
    ok    testSequence1NoAlertBranch
    ok    testSequence1WithLowStockAlert

com.isms.SalesConcurrencyTest   (1 tests, 1.53s)
    ok    lastUnitIsSoldOnce

com.isms.NfrTest   (4 tests, 0.05s)
    ok    testConcurrentVoiceInterpretationThroughput
    ok    testStockSyncLatency
    ok    testFailedLoginLockout
    ok    testSlidingSessionTokenRenewal

Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Fig. 67: Module 2 automated tests (with the non-functional tests): 13 / 13 passed

./mvnw test - complete backend regression suite

```
$ ./mvnw test

Tests run: 3, Failures: 0, Errors: 0, Skipped: 0 -- in com.isms.AdminTest
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0 -- in com.isms.AuthIntegrationTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- in com.isms.BcryptTest
Tests run: 12, Failures: 0, Errors: 0, Skipped: 0 -- in com.isms.ClassContractTest
Tests run: 12, Failures: 0, Errors: 0, Skipped: 0 -- in com.isms.ClassDiagramStructureTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0 -- in com.isms.ForecastTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- in com.isms.HealthControllerTest
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0 -- in com.isms.NfrTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0 -- in com.isms.PoStateMachineTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- in com.isms.RbacMatrixTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- in com.isms.SalesConcurrencyTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- in com.isms.SchemaMatchesSpecTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 -- in com.isms.Seq1InventoryTest
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0 -- in com.isms.Seq2PurchaseOrderTest
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0 -- in com.isms.Seq3SalesTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 -- in com.isms.StockConcurrencyTest
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0 -- in com.isms.VoiceInterpreterTest
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0 -- in com.isms.WebhookSecurityTest
Tests run: 70, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
Total time:  20.619 s
```

Fig. 68: Complete backend regression suite (./mvnw test): 70 / 70 tests passed, BUILD SUCCESS

Frontend: type-check, production build and unit test

```
$ npx tsc --noEmit
(no type errors)

$ npm run build
✓ 2436 modules transformed.
dist/index.html                  0.79 kB │ gzip:   0.48 kB
dist/assets/index-nJzg6vwI.css   23.86 kB │ gzip:   5.46 kB
dist/assets/index-B_3mXzzU.js   749.43 kB │ gzip: 203.42 kB
✓ built in 9.03s

$ npm test
 ✓ src/App.test.tsx (1 test) 2ms
 Test Files  1 passed (1)
      Tests  1 passed (1)
```

Fig. 69: Frontend: TypeScript type-check, production build and unit test all pass

Conclusion:

Thus, the two core modules of the Inventory and Stock Management System, **Authentication & Role-Based Access Control** and **Voice-Enabled Inventory & Omnichannel Synchronization**, were successfully implemented and tested, together with the five other SRS feature modules and the platform module. The implementation follows the SRS, use-case, class, sequence, activity and DFD models from the earlier experiments: all 12 classes, 43 operations and 9 relationships of the class diagram exist in the code and are verified by tests, and the call order of SD-1, SD-2

and SD-3 is asserted automatically. Login, the persisted account lockout, password reset, role-based access, the audit log, product CRUD with guarded delete, row-locked stock updates, the last-unit conflict rule, real-time synchronisation and confirmed voice commands all work correctly, including their failure paths. 38 documented test cases for the two core modules, 70 automated backend tests and 53 live API checks pass, which confirms that the modules meet their functional requirements and business rules. The work was divided between the two team members in alternating modules (Dev: 1, 3, 5, 7; Stavan: 2, 4, 6, 8) and every stage was verified to build on its own.